

Robot Perception and Learning

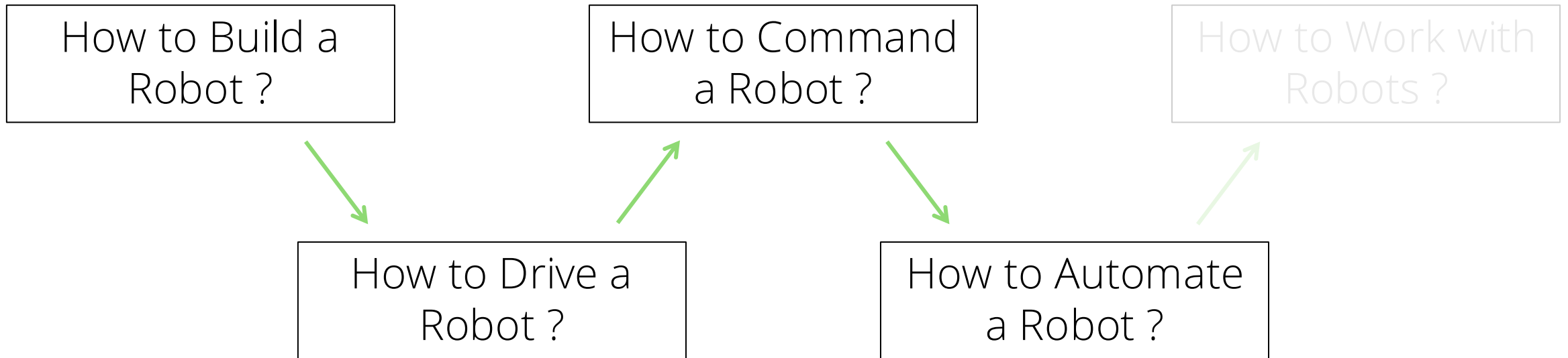
Imitation learning

Tsung-Wei Ke

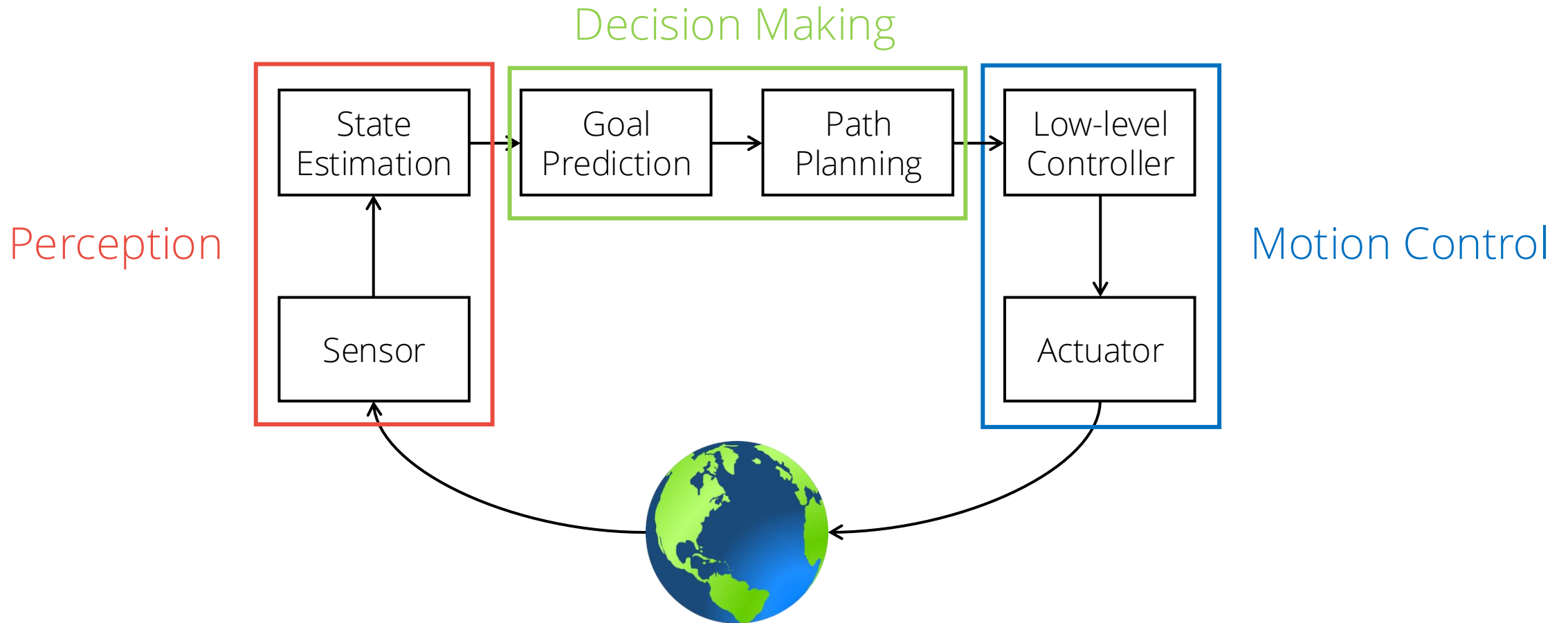
Fall 2026



Recap: Robotics is a System

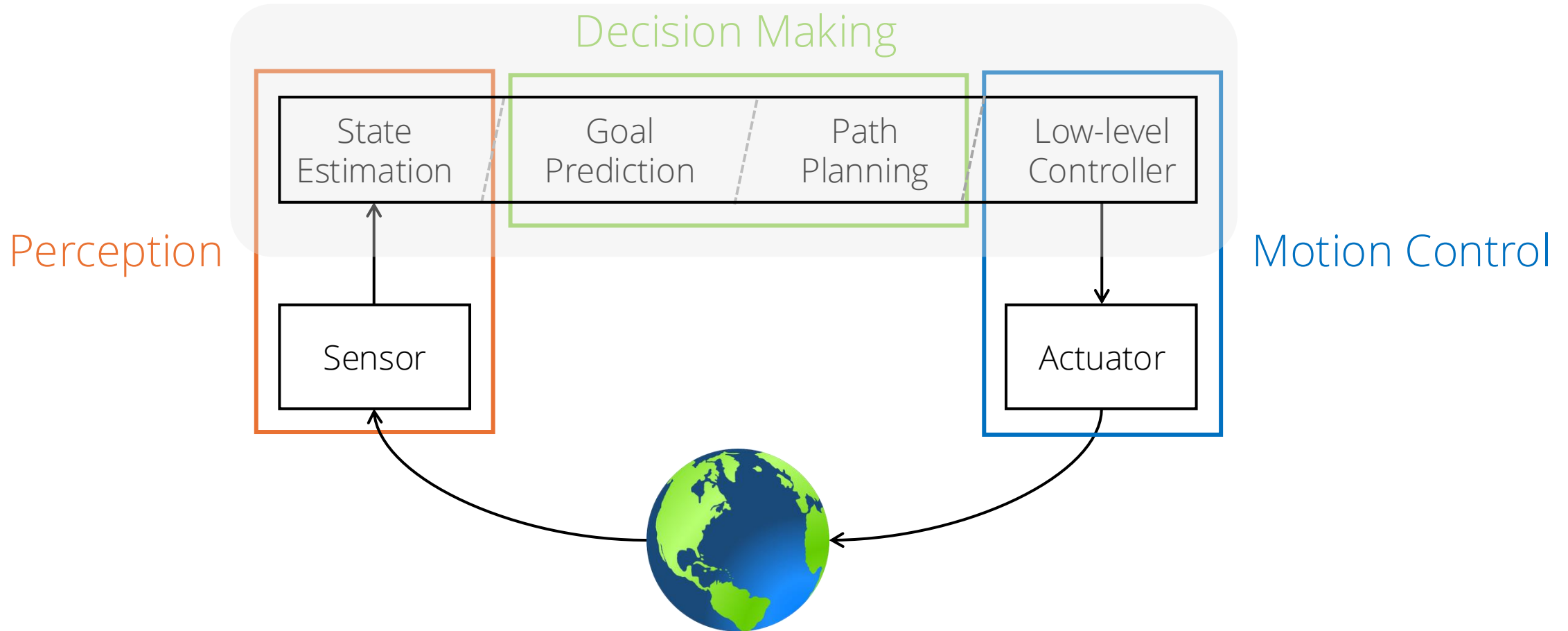


We Discussed Three Individual Components in Robotics

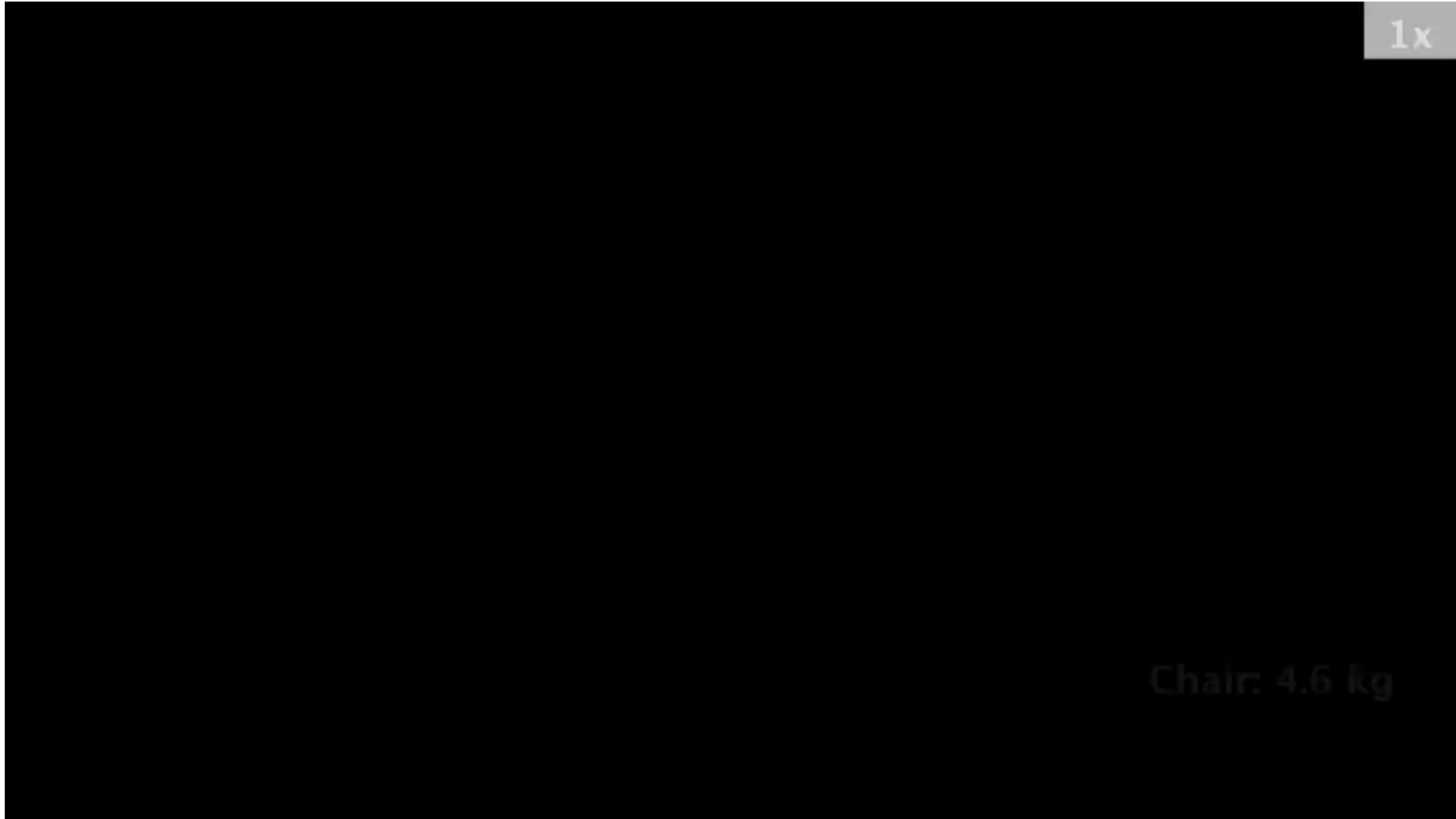


Robot Learning: Learning an End-to-end Framework for These Components

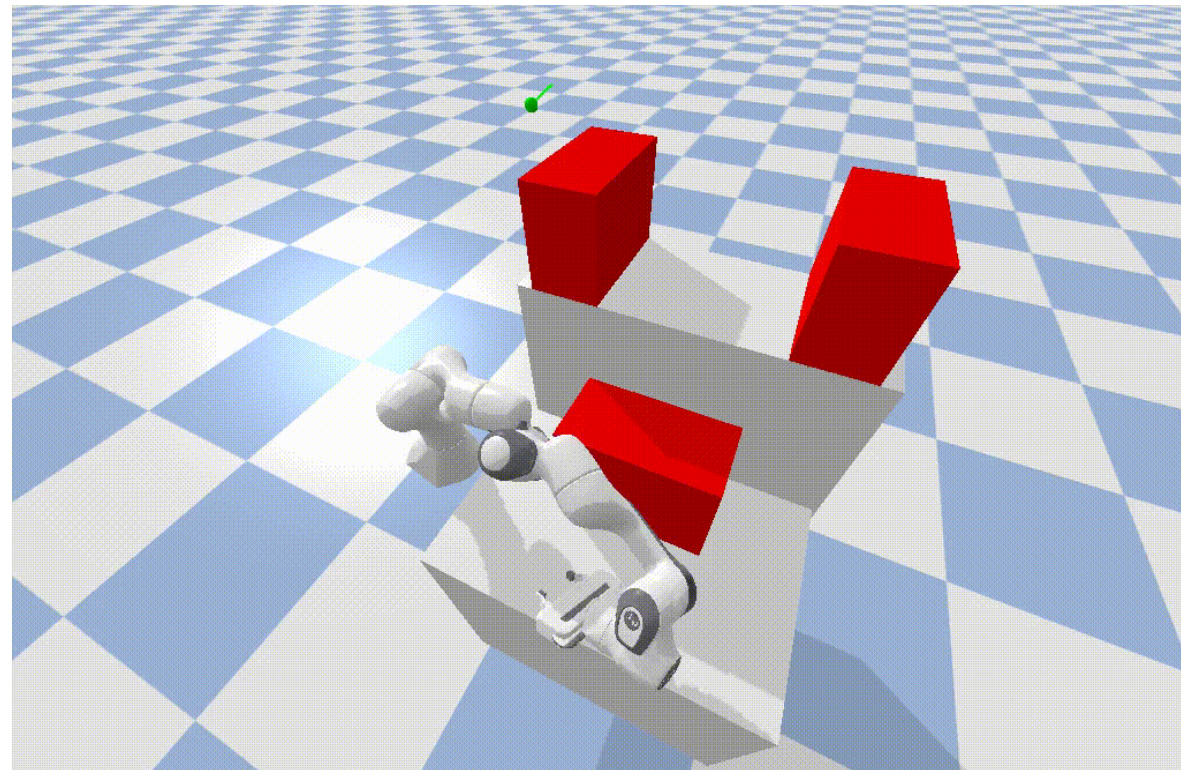
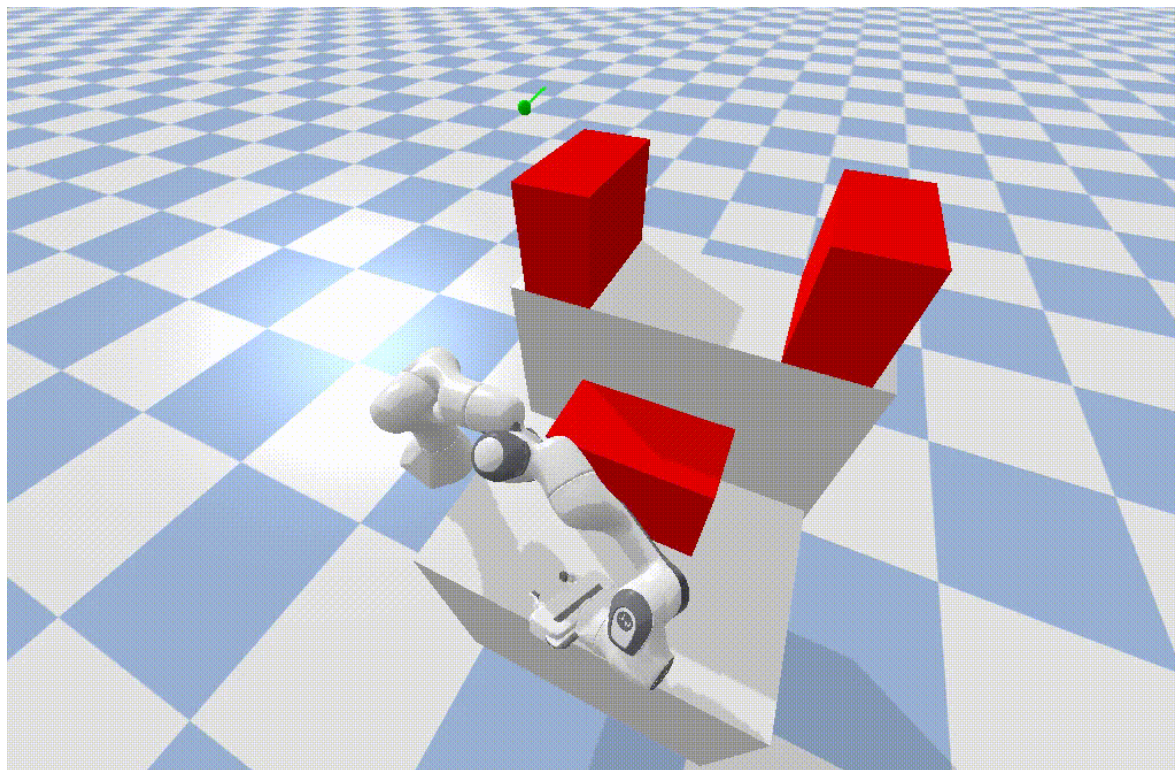
Robot Learning



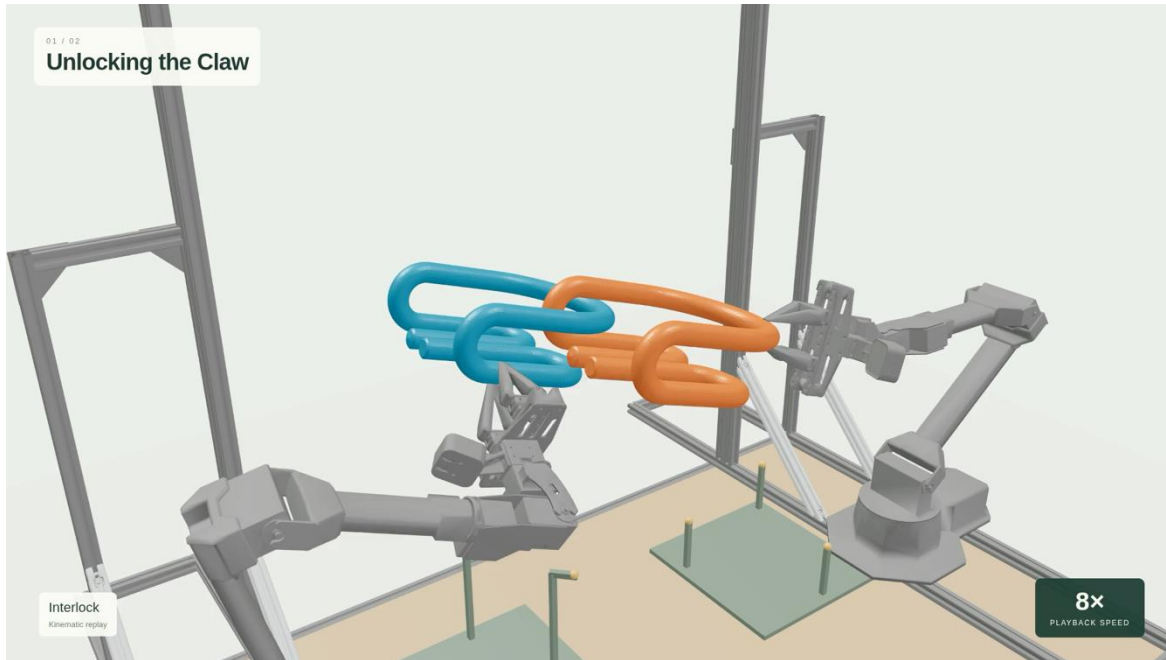
Robot Learning for Low-level Controller



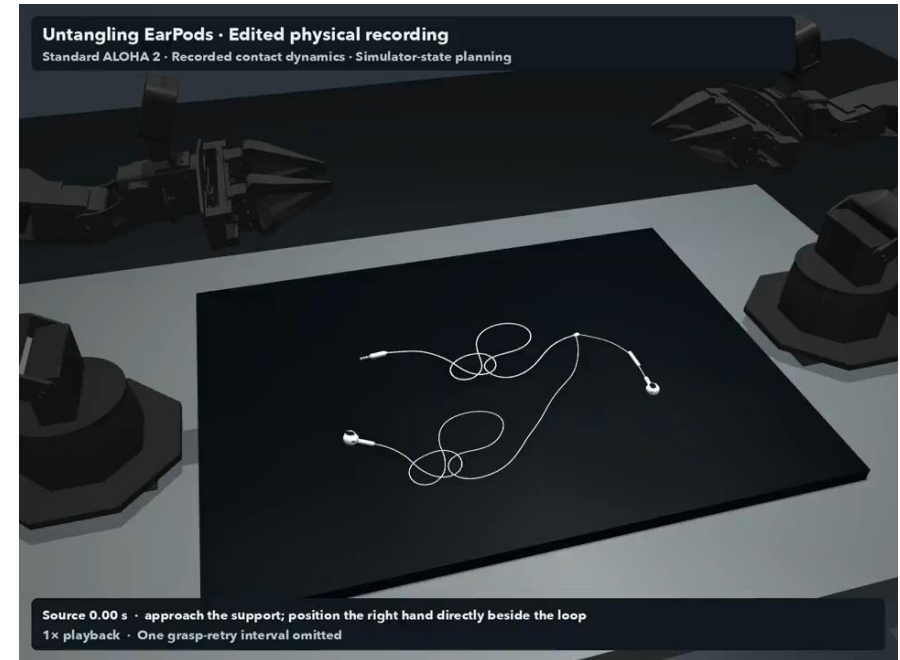
Robot Learning for Path Planning



Robot Learning for Goal (End-Effector Pose) Prediction

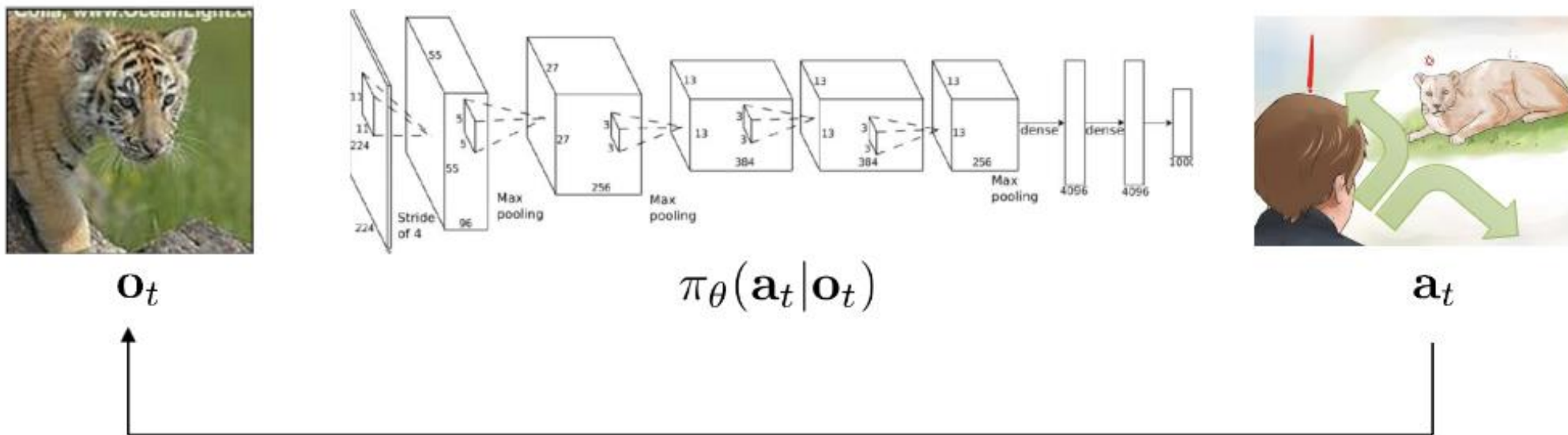


Video source: https://x.com/qineng_wang/status/2099893504658866561



Video source: <https://x.com/dimentary/status/2100431859759054882>

Policy Learning: Learn a Model that Predicts Actions from Observations



$\mathbf{s}_t$ – state

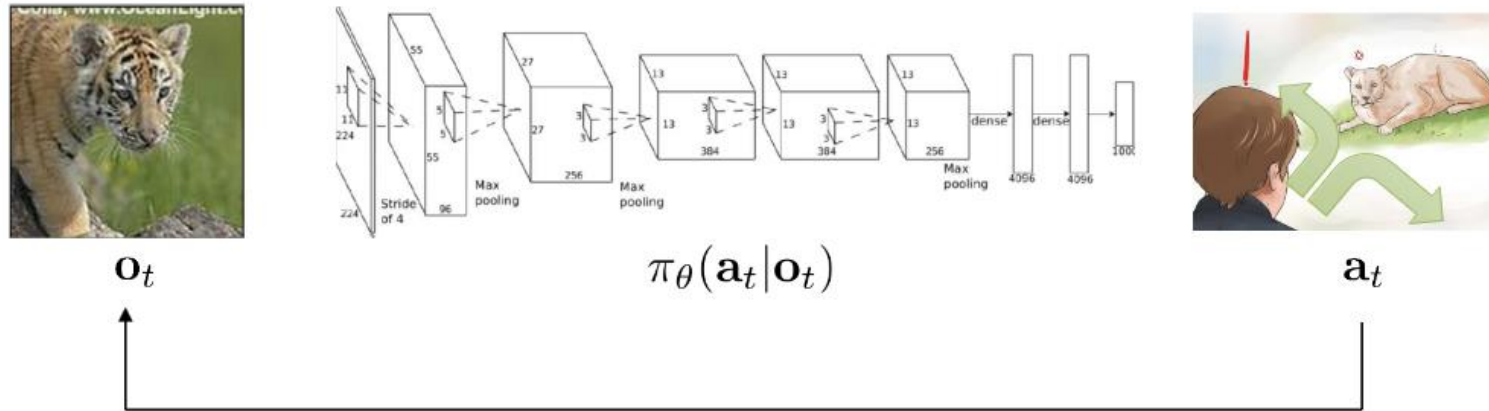
$\mathbf{o}_t$ – observation → Obtained by your sensors

$\mathbf{a}_t$ – action → End-effector 6DoF pose / joint angles / torques?

$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$ – policy

$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)

Policy Learning vs. Sequential Labeling



$\mathbf{s}_t$ – state

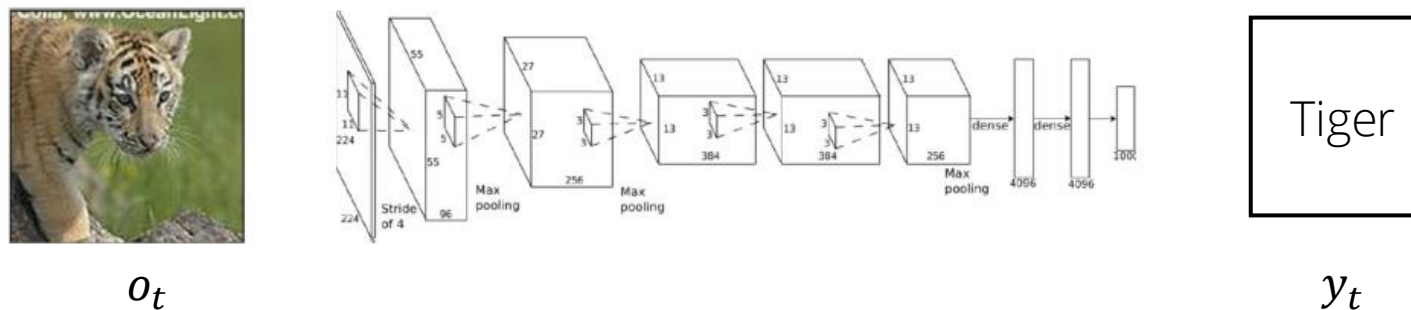
$\mathbf{o}_t$ – observation

$\mathbf{a}_t$ – action

$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$ – policy

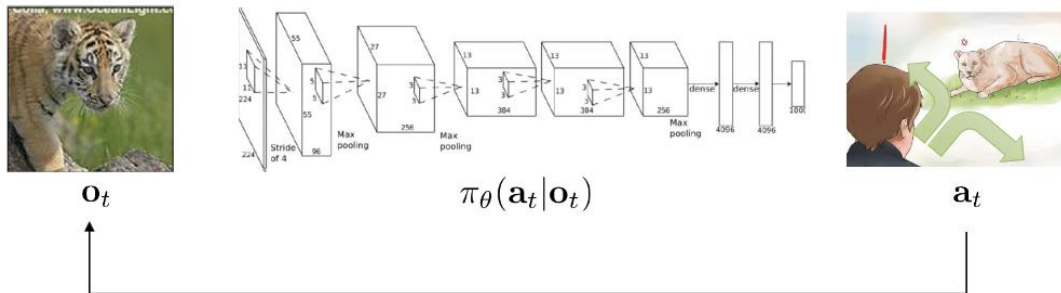
$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)

Sequential labeling



Policy Learning vs. Sequential Labeling

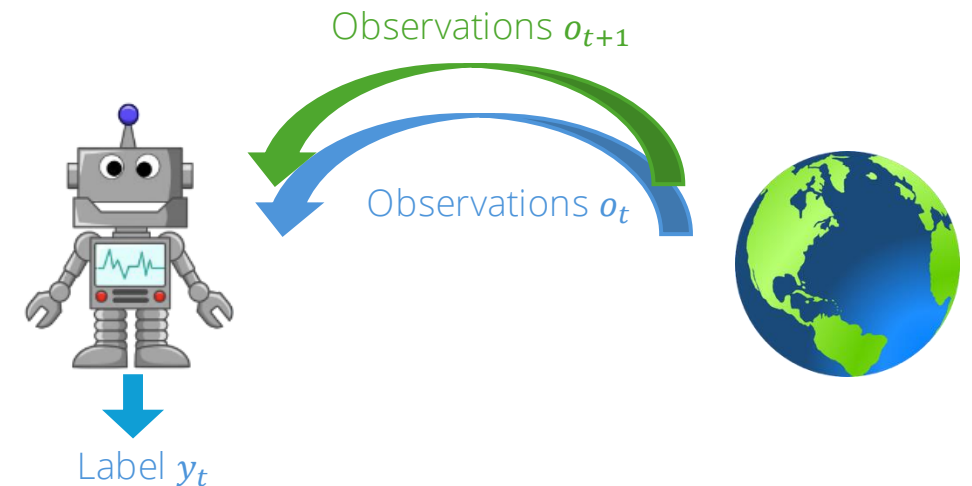
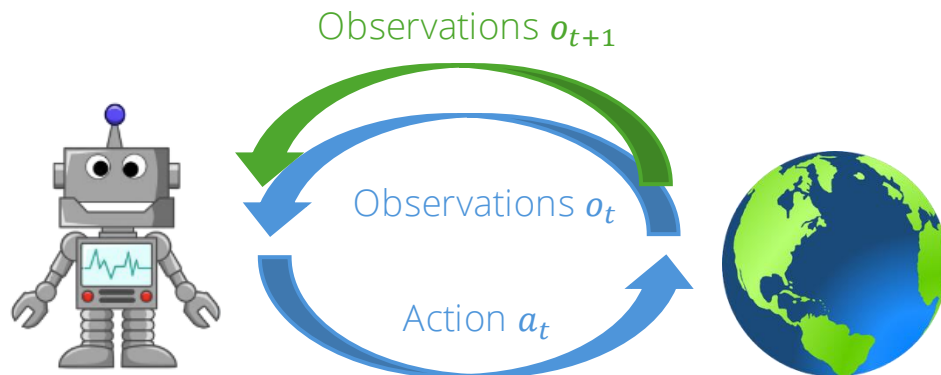
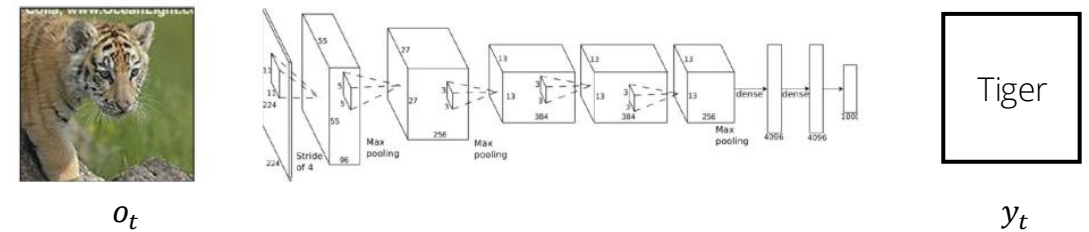
Policy learning



$\mathbf{s}_t$ – state
 $\mathbf{o}_t$ – observation
 $\mathbf{a}_t$ – action

$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$ – policy
 $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)

Sequential labeling



Learning to Act by Cloning Behaviors



Collect Action Labels by Teleoperation



https://youtu.be/9-9udQtO1PY?si=AZCr7n_9V41BA2o9

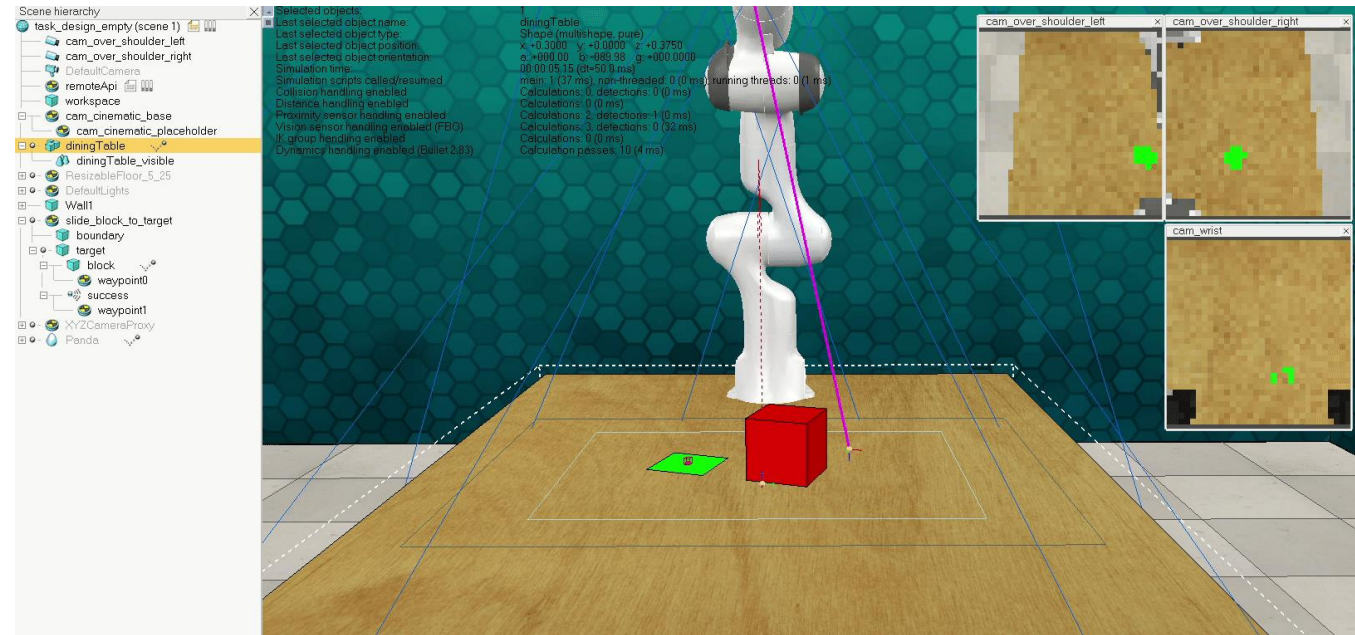


<https://youtu.be/Bhg3uOx9ZPw?si=et7L0endzGvGPJz->

Collect Action Labels by Goal-Pose Annotations and Motion Planners



CuRoBo: CUDA Accelerated Robot Library



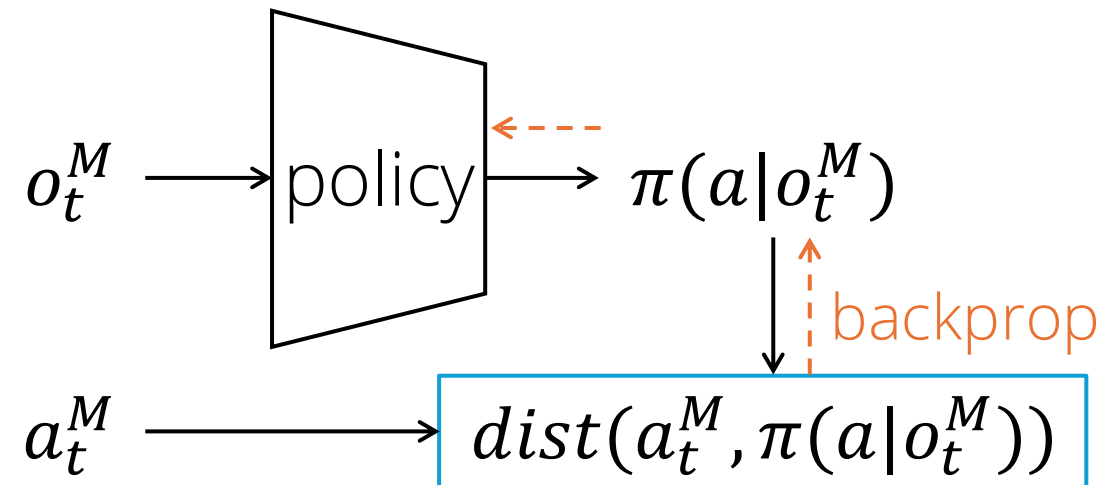
RLBench: Robot Learning Benchmark. James et al.

Behavior Cloning: Learning to Act by Imitation

Step 1: Collect demonstrations

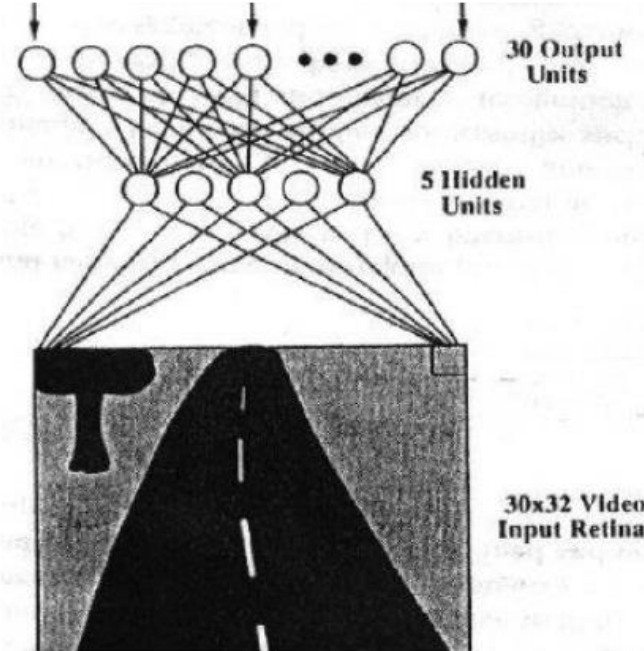
$$\begin{array}{c} o_1^1, a_1^1, o_2^1, a_2^1, \dots, o_T^1, a_T^1 \\ o_1^2, a_1^2, o_2^2, a_2^2, \dots, o_T^2, a_T^2 \\ \dots \\ o_1^N, a_1^N, o_2^N, a_2^N, \dots, o_T^N, a_T^N \end{array}$$

Step 2: Train the policy to imitate



Autonomous Driving with Imitation Learning

ALVINN: Autonomous Land Vehicle In a Neural Network
1989



Autonomous Driving with Imitation Learning



Imitation Learning Looks Great in General



However, It Doesn't Always Work...



Co-Variate Shift Problem

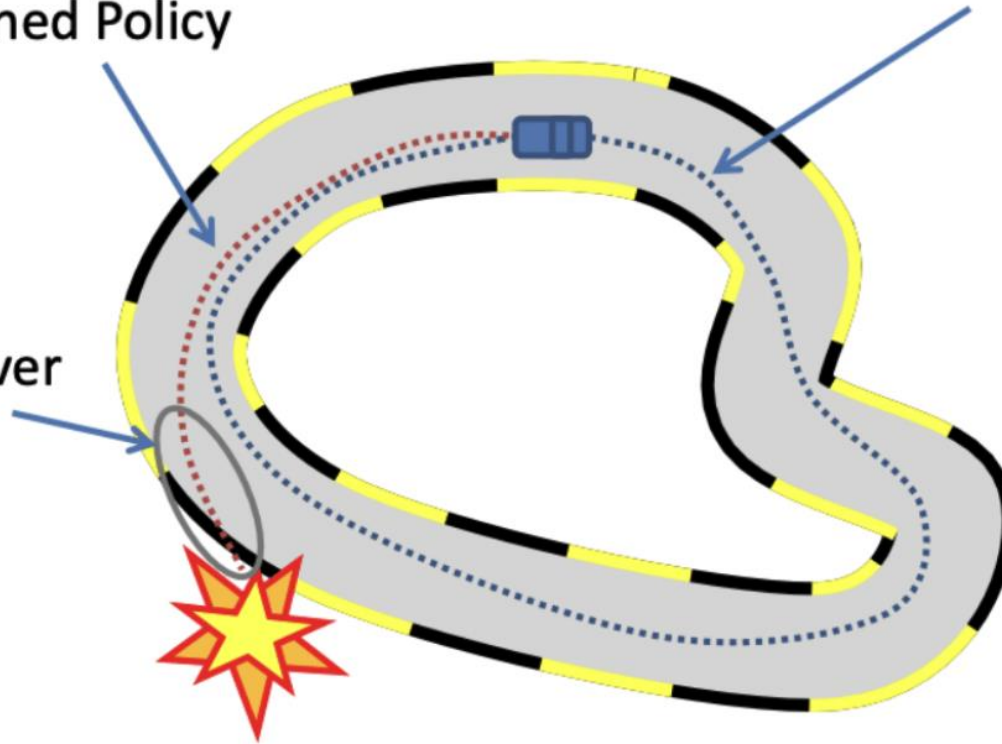
Distributional Shift Problem

Learned Policy

Expert trajectory

No data on
how to recover

Any solution?



The “cloned” policy can go out-of-distribution!

Co-Variate Shift Problem

Distributional Shift Problem

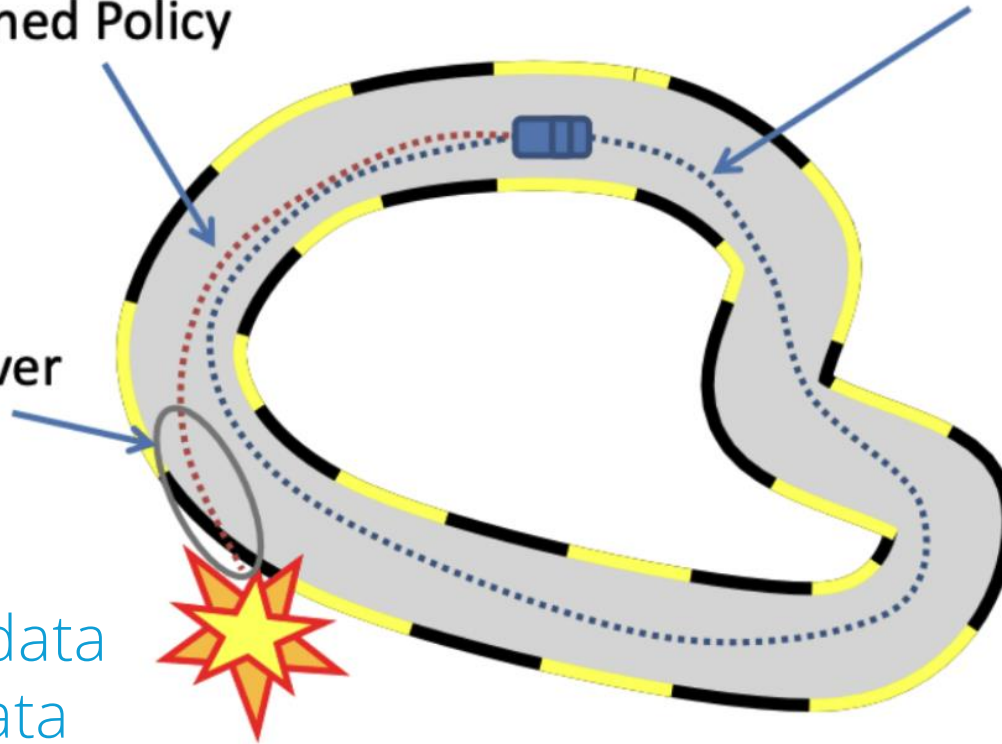
Learned Policy

Expert trajectory

No data on
how to recover

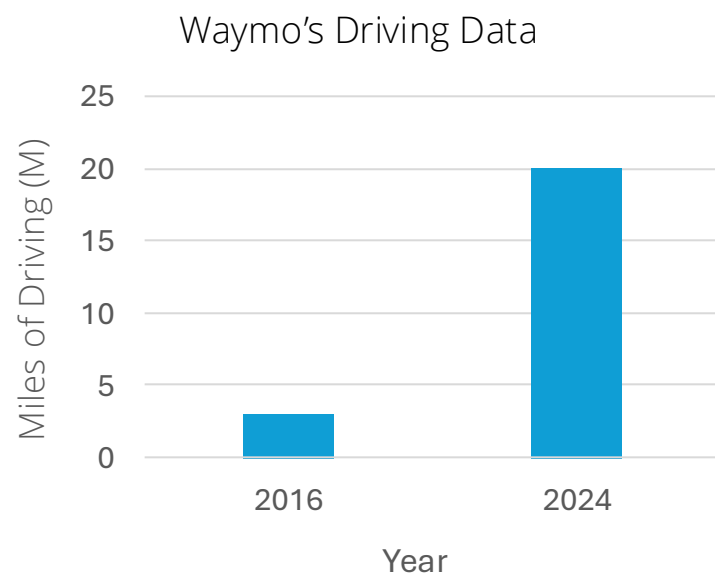
Solution:

1. Collect more diverse data
2. Collect “recovering” data by augmentation



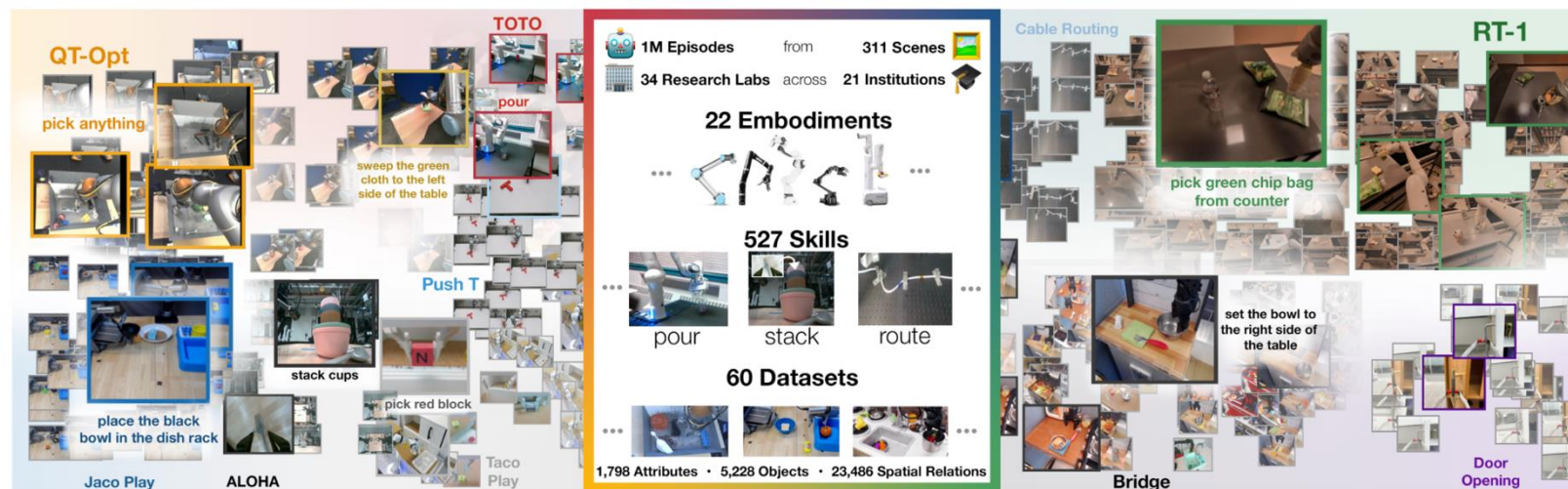
The “cloned” policy can go out-of-distribution!

Solution 1: Collect More Demonstration Data

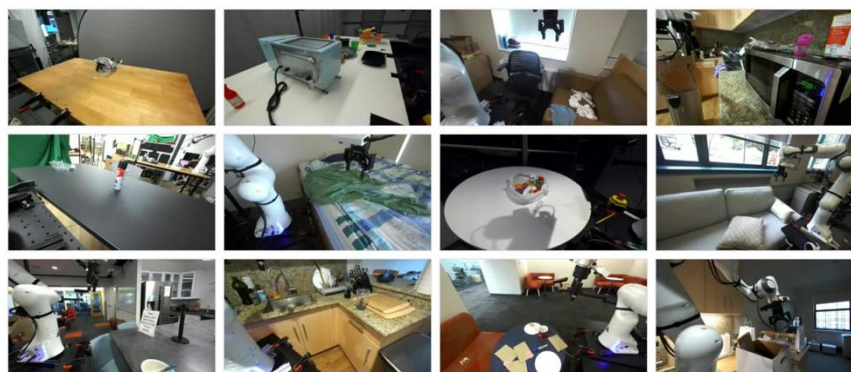


<https://waymo.com/waymo-driver/>

Open X-Embodiment



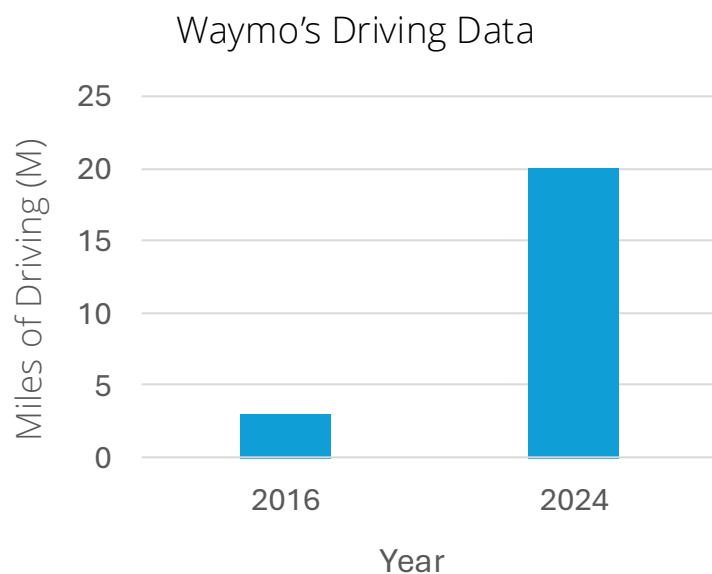
DROID



RoboCasa



Solution 1: Collect More Demonstration Data

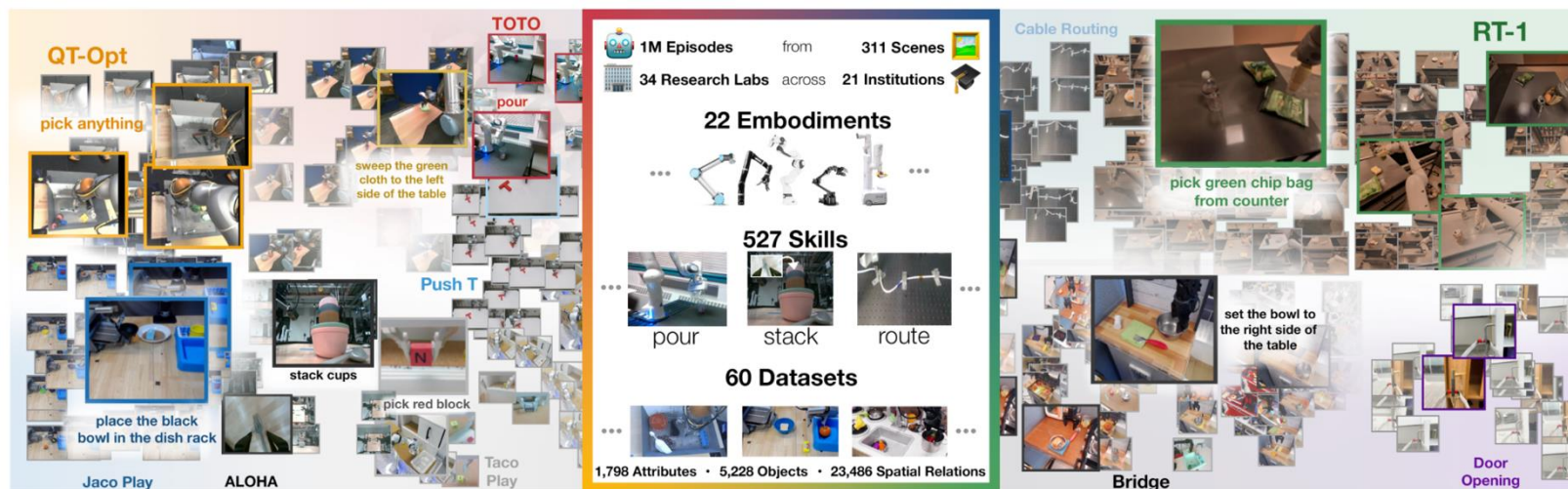


<https://waymo.com/waymo-driver/>

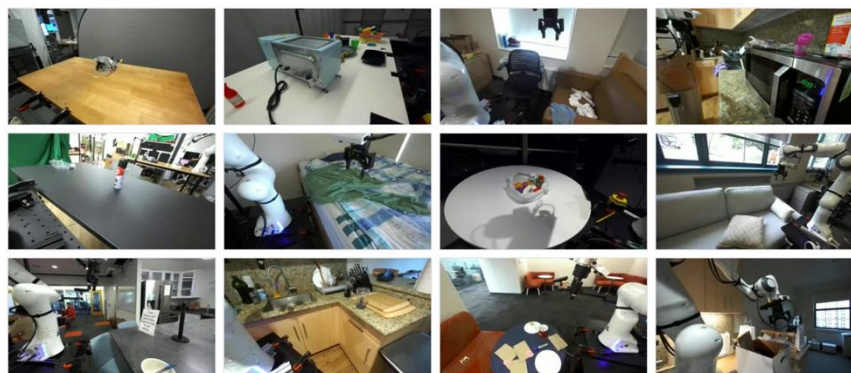
Problems:

1. Reproducibility: How to replay corner cases?
2. Cost: How much does it take to cover all/most distributional drift?

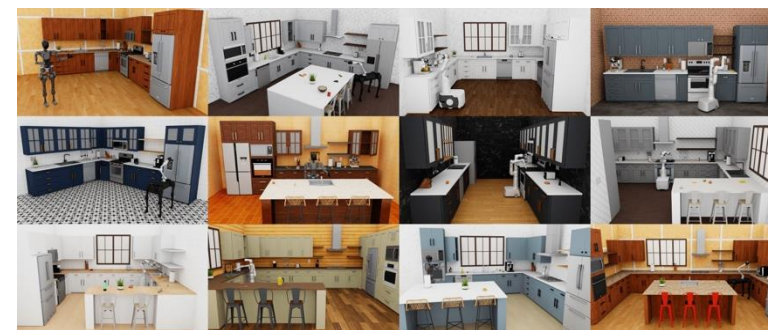
Open X-Embodiment



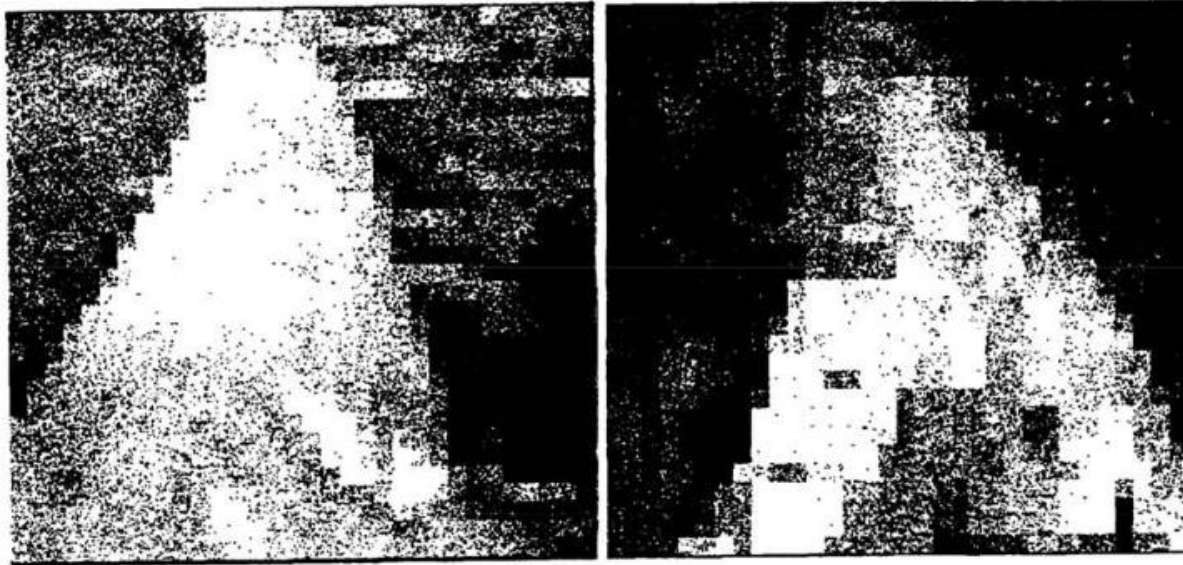
DROID



RoboCasa



Solution 1: Collect More Demonstration Data in Simulation



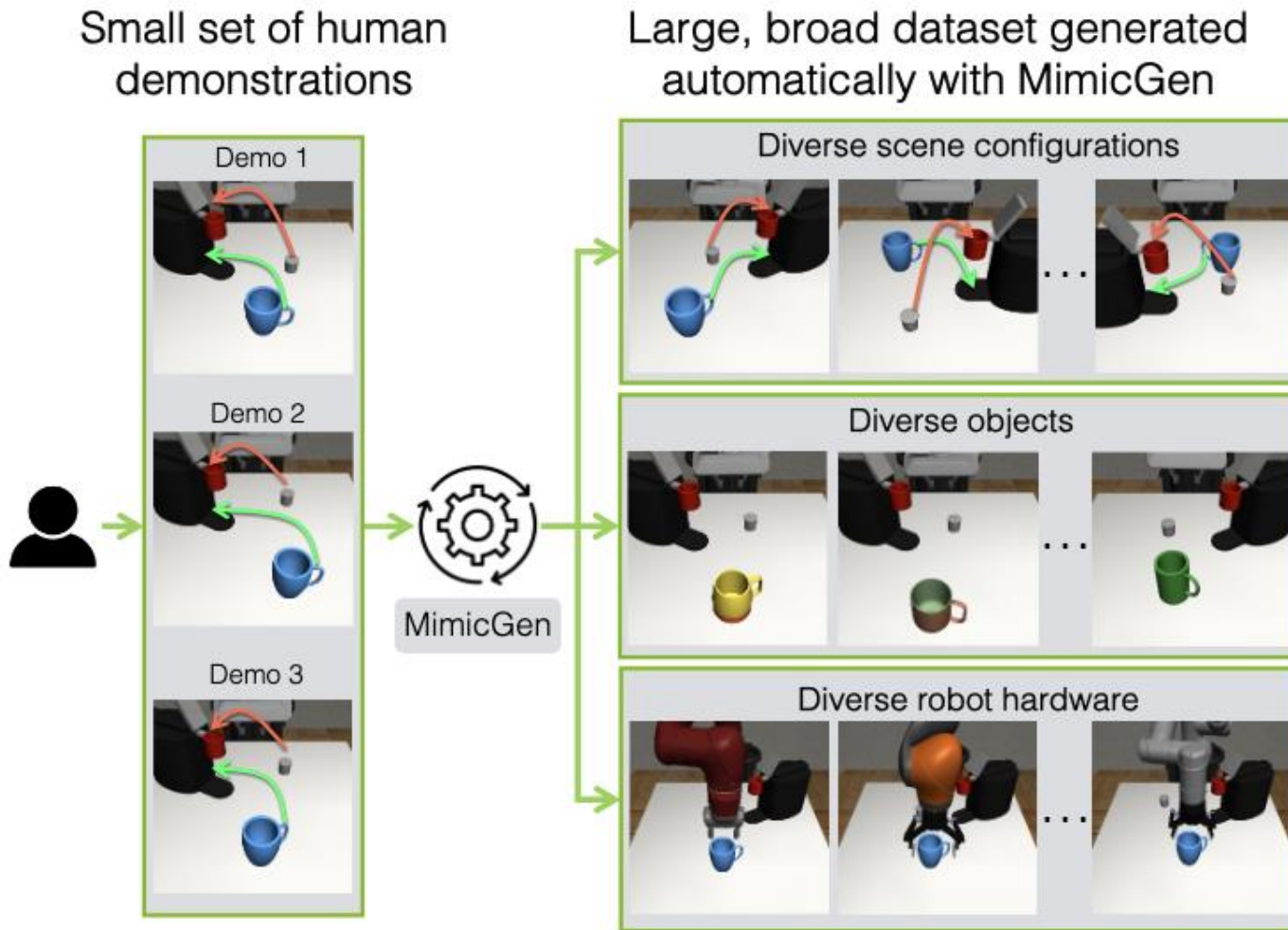
Real Road Image

Simulated Road Image

- Simulate road images and the corresponding ground-truth steering angle
- Online adaptation to human-driver steering angle control
- Neural-network based policies

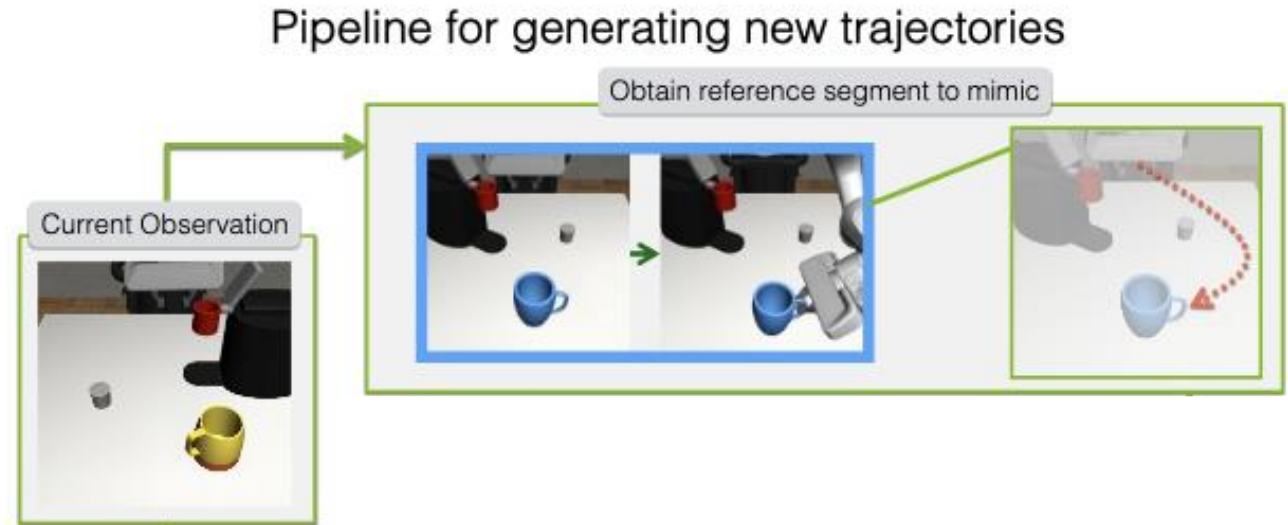
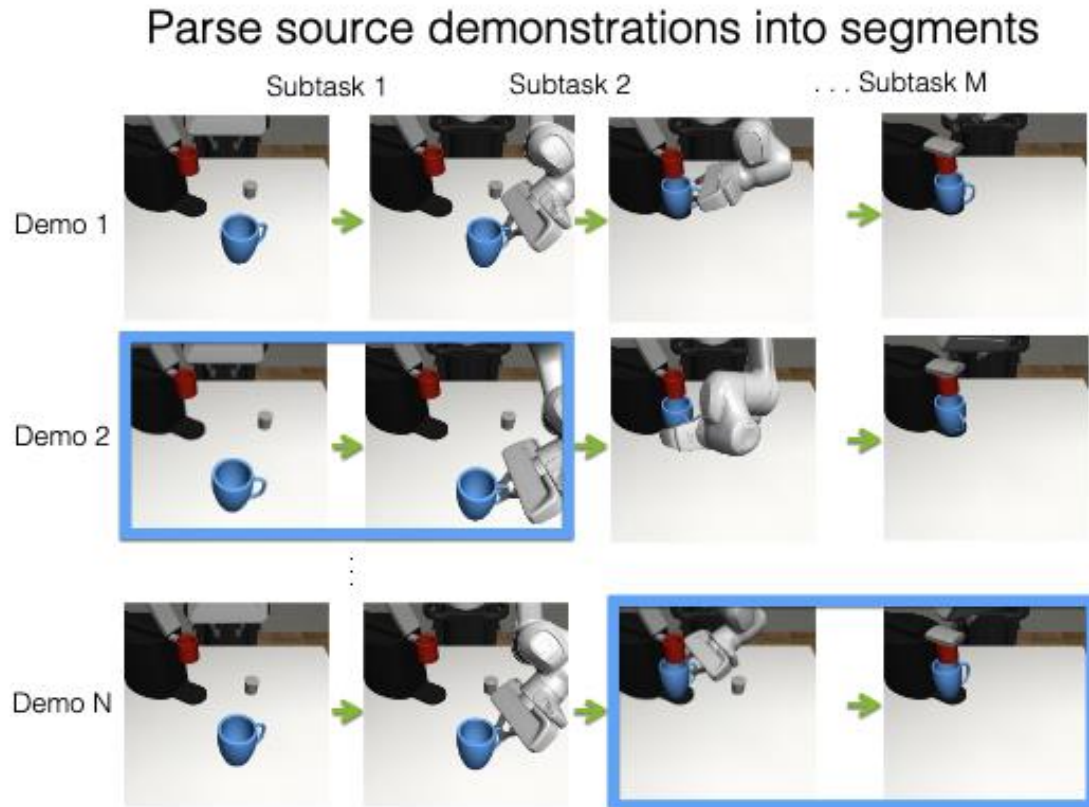
ALVINN: An Autonomous Land Vehicle in a Neural Network. D. A. Pomerleau.

Solution 1: Collect More Demonstration Data by Augmentation

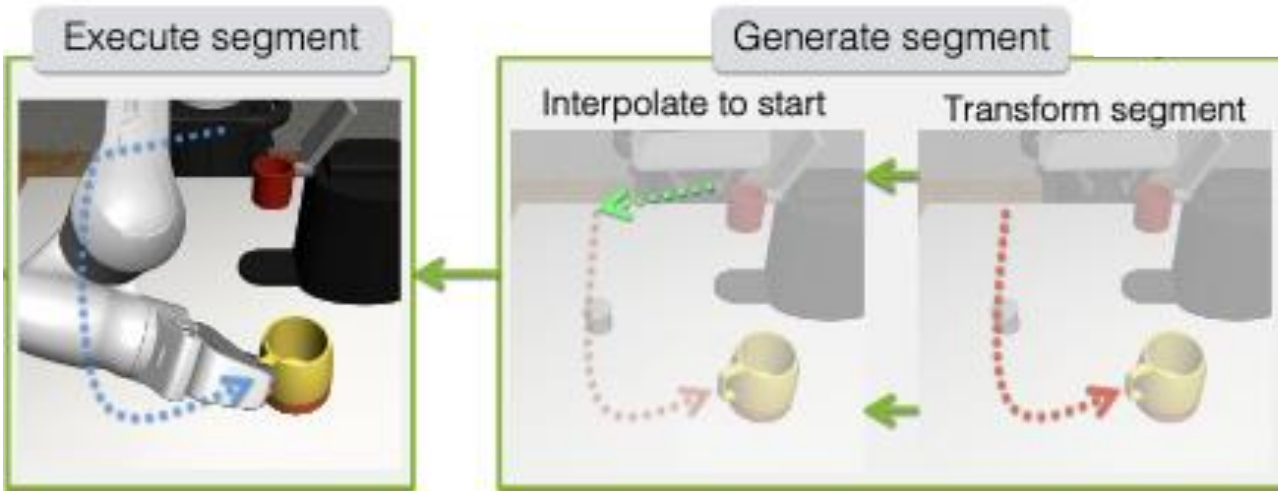


- Data augmentation based on geometrical priors:
 - Robot actions remain invariant to the object frame
- Pipeline overview:
 1. Sample a start state of a task
 2. Retrieve subtasks from demonstrations and adapt each subtask to the current scene
 3. Execute the synthetic demos. Add to training dataset if the execution succeed.

Step 1: Segment Demonstrations into Subtasks



Step 2: Adapt the Subtask Segment to the New Scene



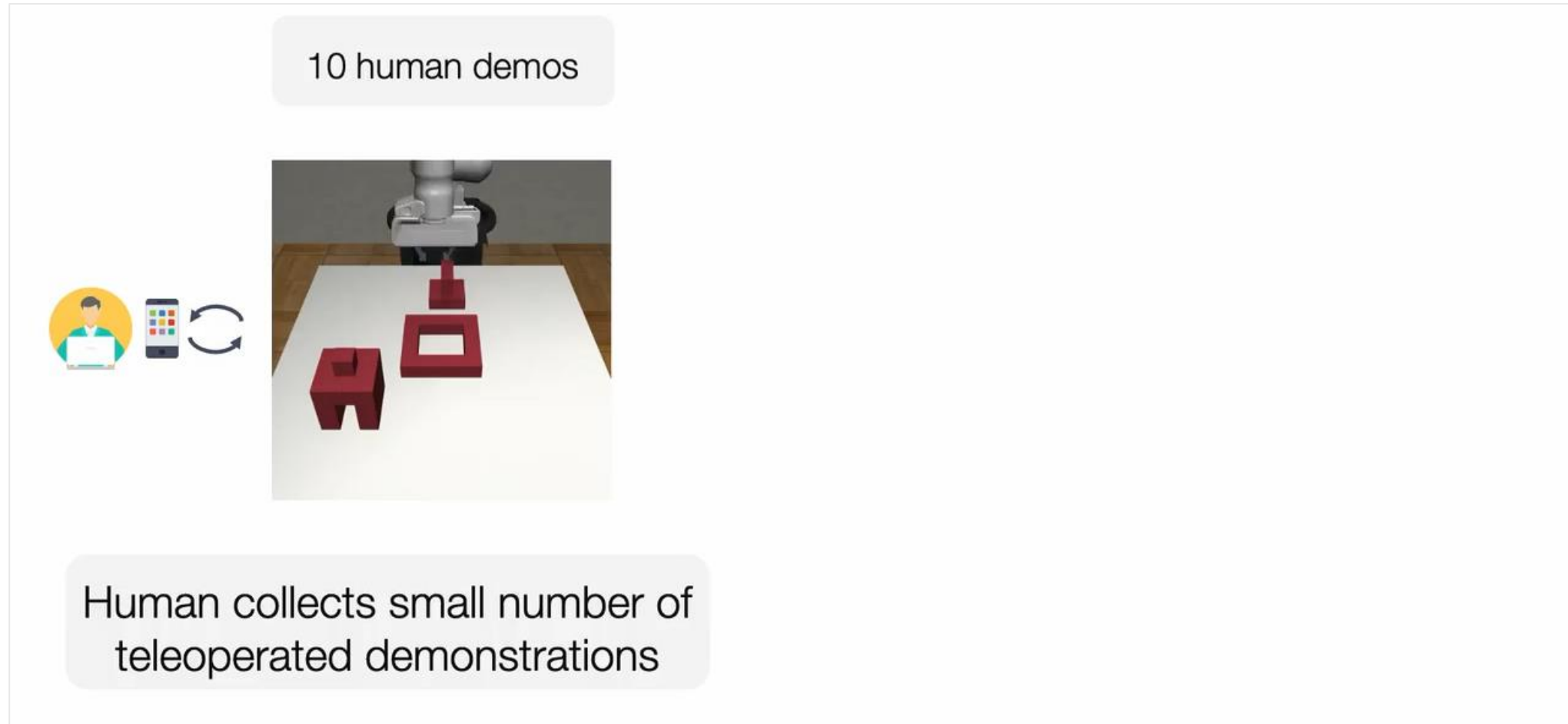
- Let $\mathbf{T}_W^{\mathbf{C}_t}$ denote the end-effector pose $\mathbf{C}_t$ with respect to world frame $\mathbf{W}$
- Let $\mathbf{T}_W^{\mathbf{O}}$ denote the object pose $\mathbf{O}$ with respect to world frame $\mathbf{W}$
- Given a new scene with $\mathbf{T}_W^{\mathbf{O}'}$, geometrical priors:

$$\mathbf{T}_W^{\mathbf{C}_t'} = \mathbf{T}_W^{\mathbf{O}} \left(\mathbf{T}_W^{\mathbf{O}'} \right)^{-1} \mathbf{T}_W^{\mathbf{C}_t}$$

Why?

- $\mathbf{T}_O^{\mathbf{C}_t'} = \left(\mathbf{T}_W^{\mathbf{O}'} \right)^{-1} \mathbf{T}_W^{\mathbf{C}_t'}$
- $\mathbf{T}_O^{\mathbf{C}_t} = \left(\mathbf{T}_W^{\mathbf{O}} \right)^{-1} \mathbf{T}_W^{\mathbf{C}_t}$

Solution 1: Collect More Demonstration Data by Augmentation

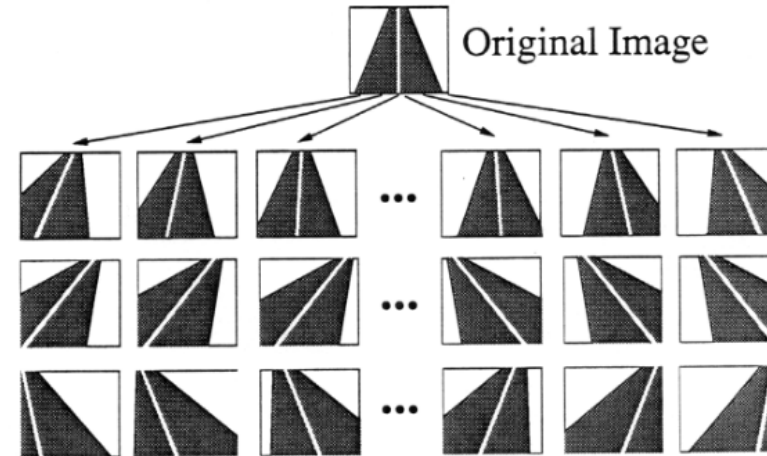


Problems: the policy still doesn't how to correct / recover from mistakes...
How to collect corrective data?

Solution 2: Collect Recovering Data by Augmentation

Simulating the Steering Errors

- Let humans drive as best they can.
- Increase training set variety by *artificially* shifting and rotating the video images, so that the vehicle appears at different orientations relative to the road.



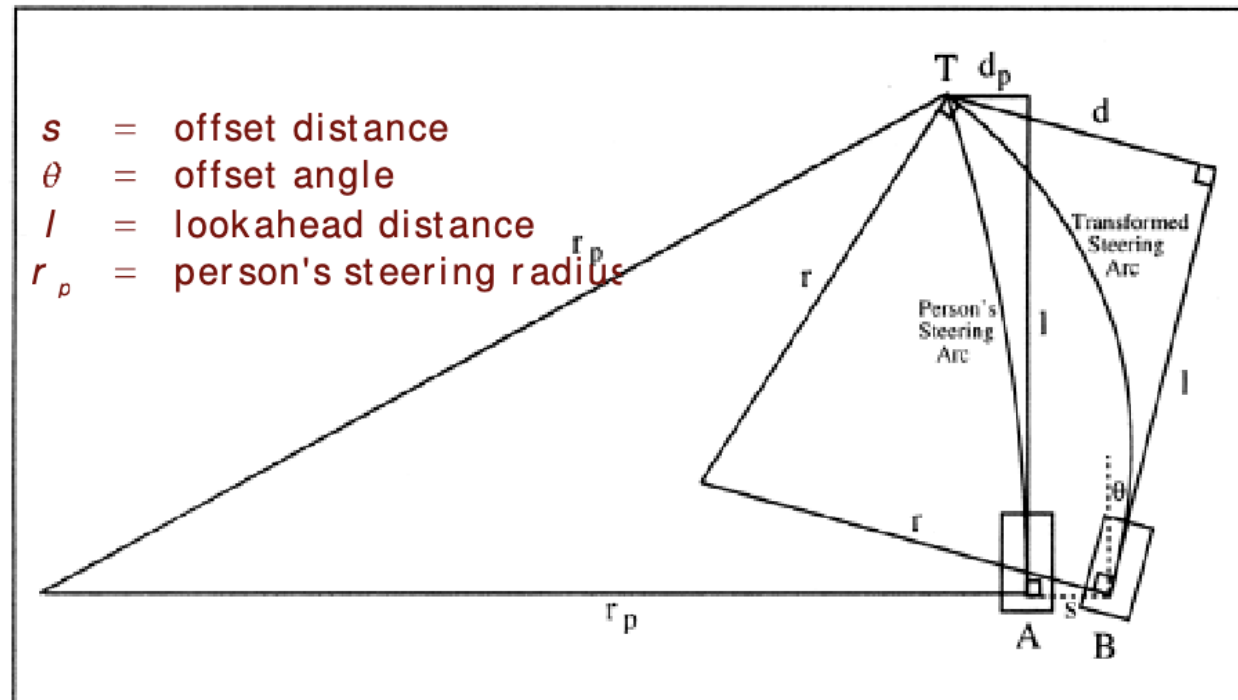
- **Generate 14 random shift/rotations for each image.**
- A simple steering model is used to predict how a human driver would react to each transformation.



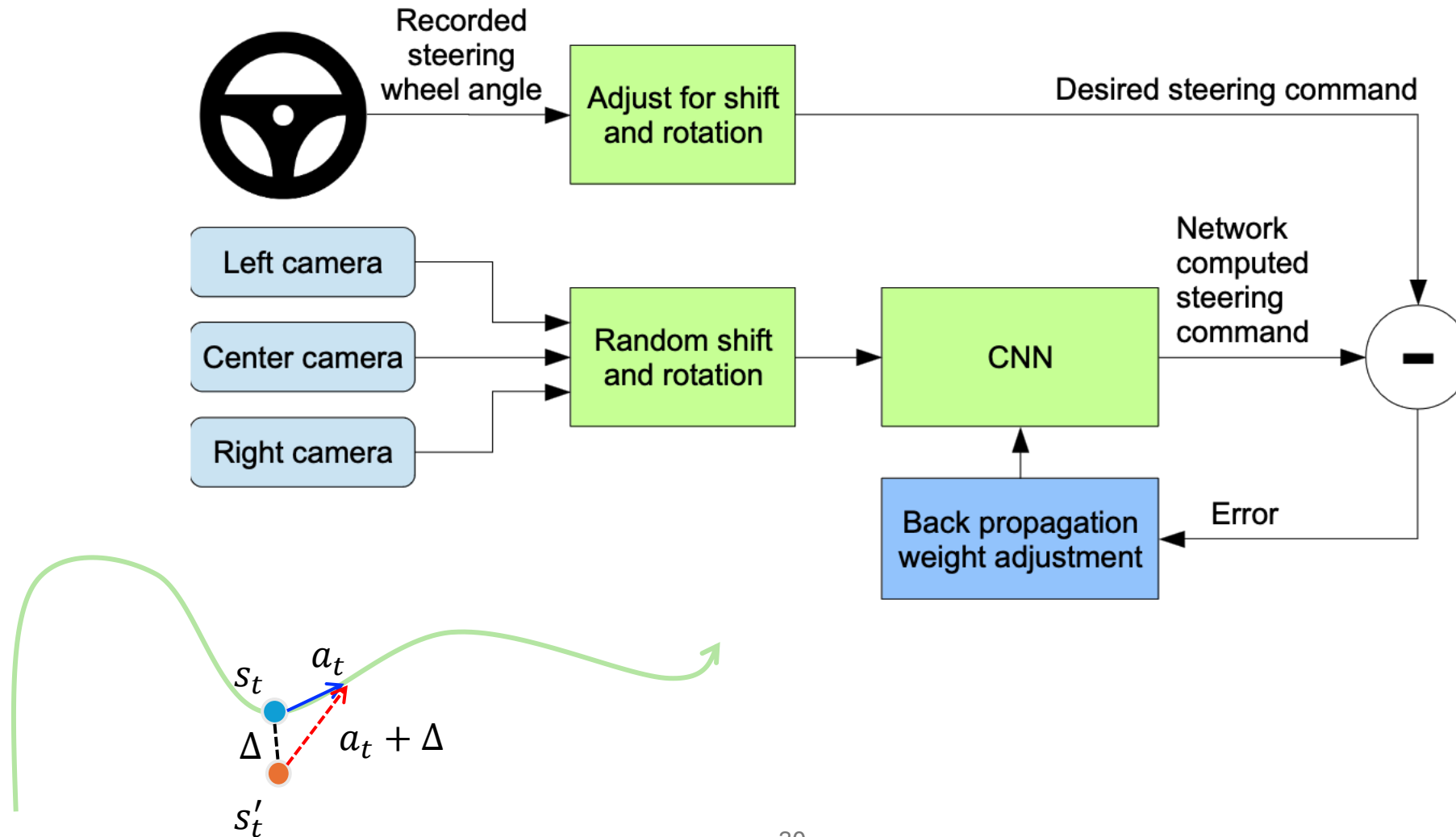
Solution 2: Collect Recovering Data by Augmentation

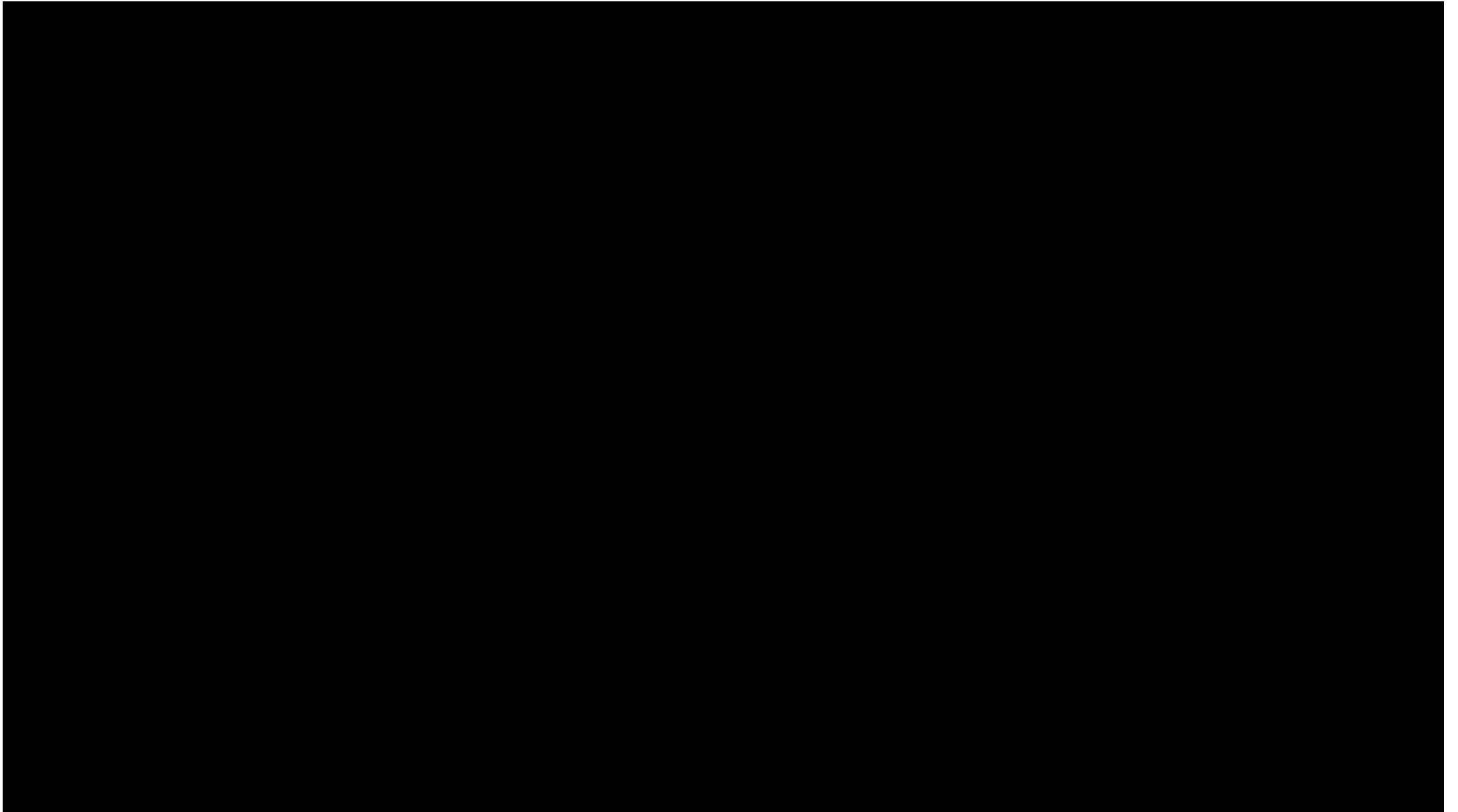
Estimating Correct Steering Direction

- “Pure pursuit” steering model generates a fairly good estimate of what the human driver would do.

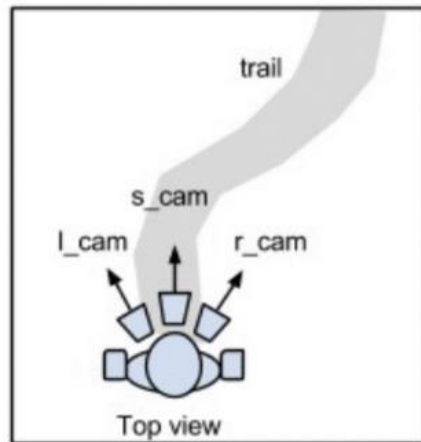
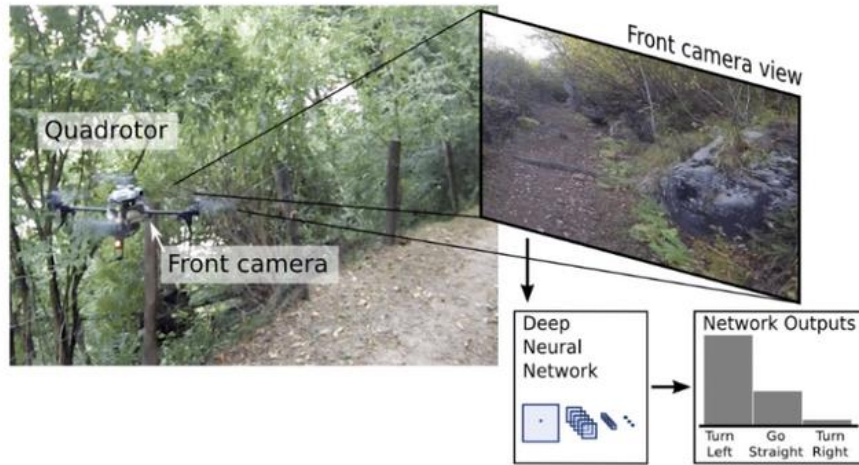


Solution 2: Collect Recovering Data by Augmentation





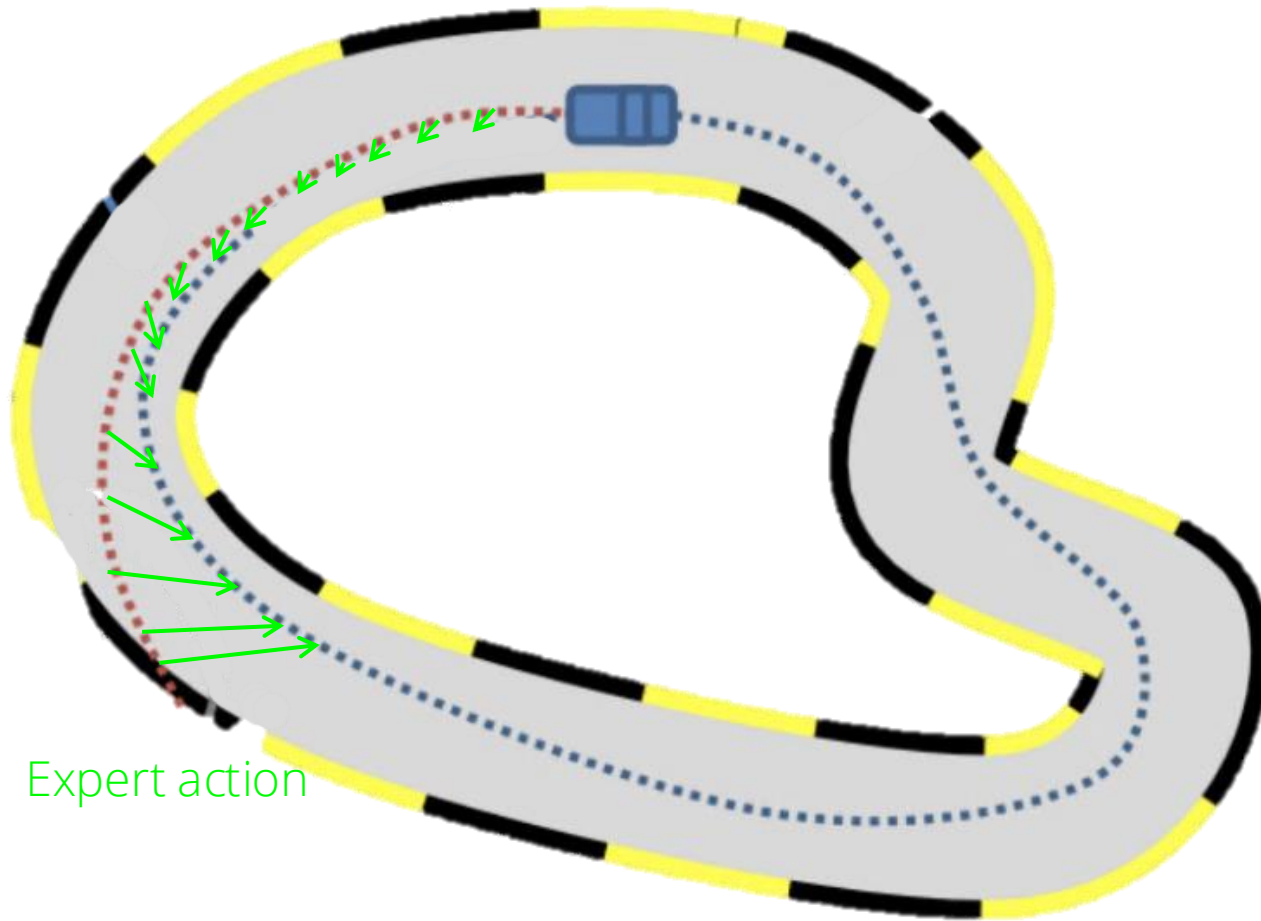
Solution 2: Collect Recovering Data by Augmentation



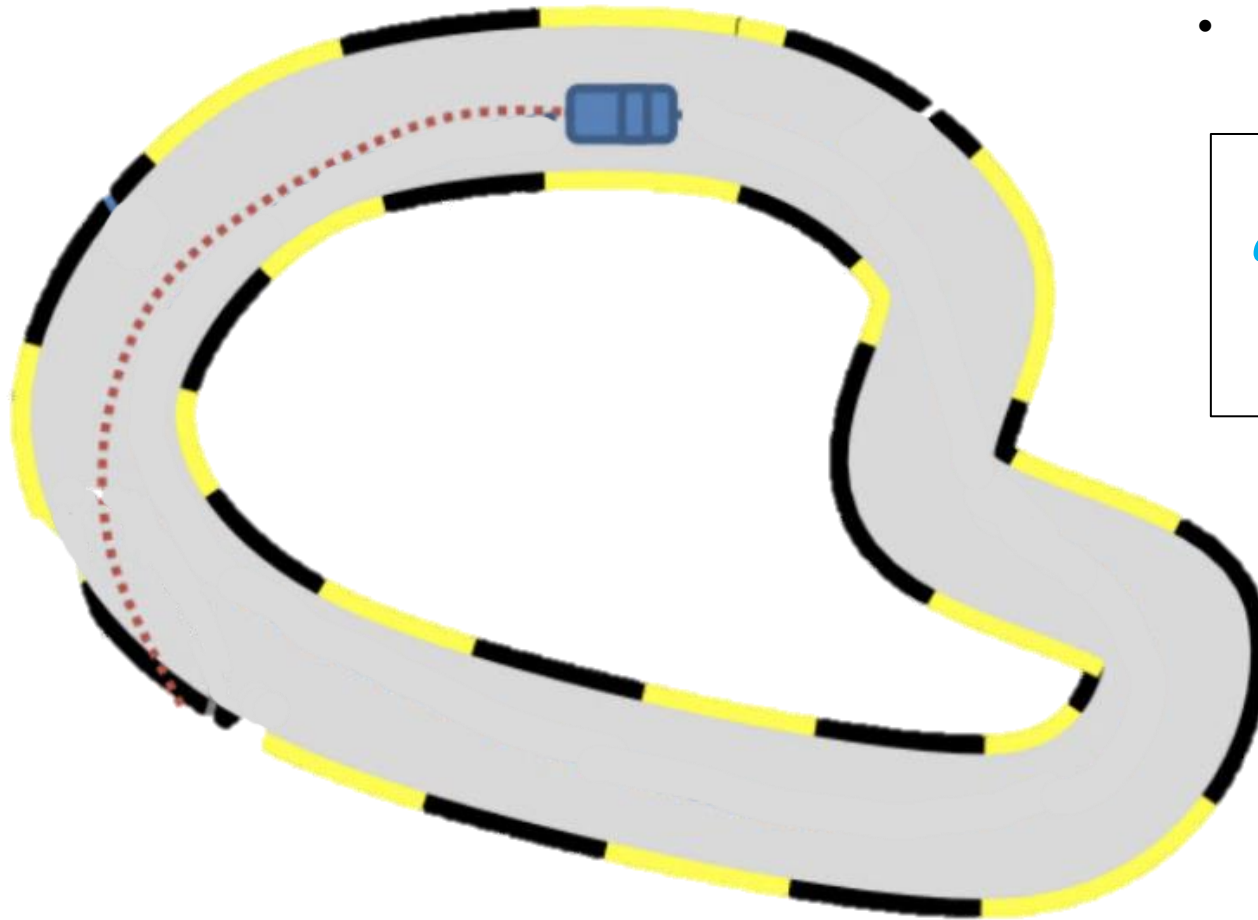
What are the Problems?

- Is geometric augmentation of the data always feasible?
 - How to recover from collision
 - How to correct erroneous manipulation of deformable objects
- Does geometric augmentation of the data suffice?

Solution 2: Collect Recovering Data with Expert in the Loop



Solution 2: Collect Recovering Data with Expert in the Loop



- Collect new experience by deploying trained policies:

$$\begin{aligned} & o_1^S, \hat{a}_1^S, o_2^S, \hat{a}_2^S, \dots, o_T^S, \hat{a}_T^S \\ & o_1^U, \hat{a}_1^U, o_2^U, \hat{a}_2^U, \dots, o_T^U, \hat{a}_T^U \\ & \dots \\ & o_1^V, \hat{a}_1^V, o_2^V, \hat{a}_2^V, \dots, o_T^V, \hat{a}_T^V \end{aligned}$$

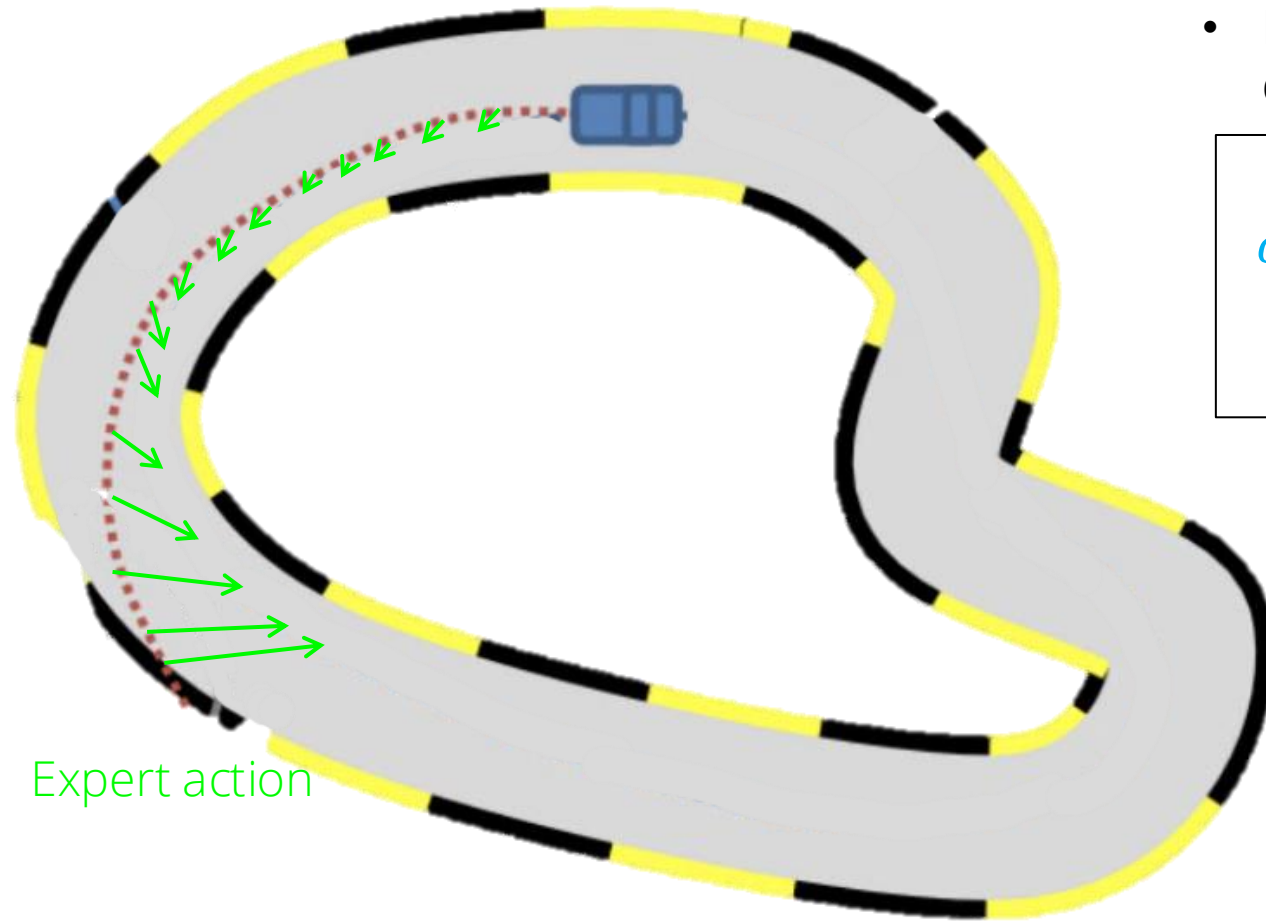
o_t^n : seen observation

a_t^n : seen action

o_t^n : unseen observation

$\hat{a}_t^n$: predicted action

Solution 2: Collect Recovering Data with Expert in the Loop



- Label new experience by querying experts:

$$\begin{array}{l} o_1^S, a_1^S, o_2^S, a_2^S, \dots, o_T^S, a_T^S \\ o_1^U, a_1^U, o_2^U, a_2^U, \dots, o_T^U, a_T^U \\ \dots \\ o_1^V, a_1^V, o_2^V, a_2^V, \dots, o_T^V, a_T^V \end{array}$$

o_t^n : seen observation

a_t^n : seen action

o_t^n : unseen observation

a_t^n : expert action

DAGGER

Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π .
for $i = 1$ **to** N **do**
 Let $\pi_i = \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i$.
 Sample T -step trajectories using π_i .
 Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i
 and actions given by expert.
 Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$.
 Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.
end for
Return best $\hat{\pi}_i$ on validation.

Algorithm 3.1: DAGGER Algorithm.

New dataset:

$$\begin{array}{c} o_1^1, a_1^1, o_2^1, a_2^1, \dots, o_T^1, a_T^1 \\ o_1^2, a_1^2, o_2^2, a_2^2, \dots, o_T^2, a_T^2 \\ \dots \\ o_1^N, a_1^N, o_2^N, a_2^N, \dots, o_T^N, a_T^N \end{array}$$

+

$$\begin{array}{c} o_1^S, a_1^S, o_2^S, a_2^S, \dots, o_T^S, a_T^S \\ o_1^U, a_1^U, o_2^U, a_2^U, \dots, o_T^U, a_T^U \\ \dots \\ o_1^V, a_1^V, o_2^V, a_2^V, \dots, o_T^V, a_T^V \end{array}$$

DAGGER



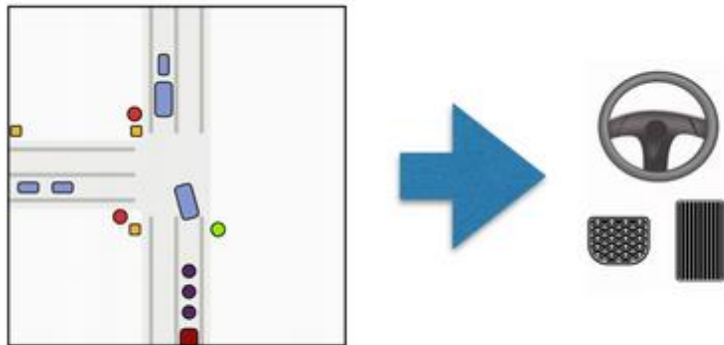
4x
Recovery behavior #1 - Too far away



Experts can be Non-Human but Privileged Learned Policies

DAGGER: from a privileged teaching agent to an agent that drives from images.

privileged agent

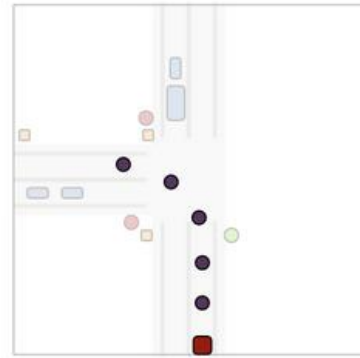
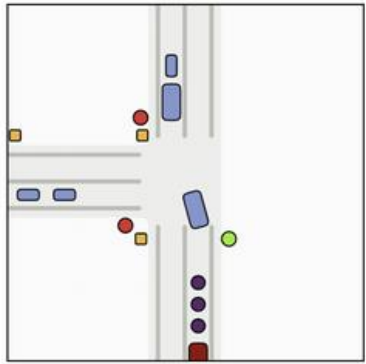


sensorimotor agent



Learning by Cheating, Chen et al., 2019

Privileged Agent vs. Sensorimotor Agent



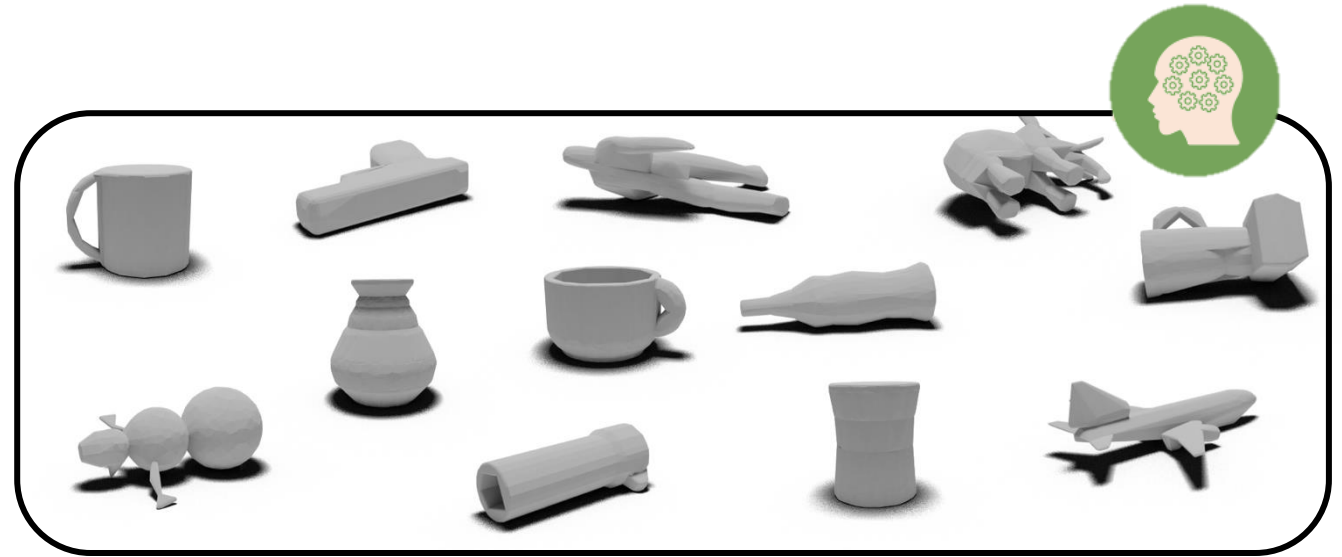
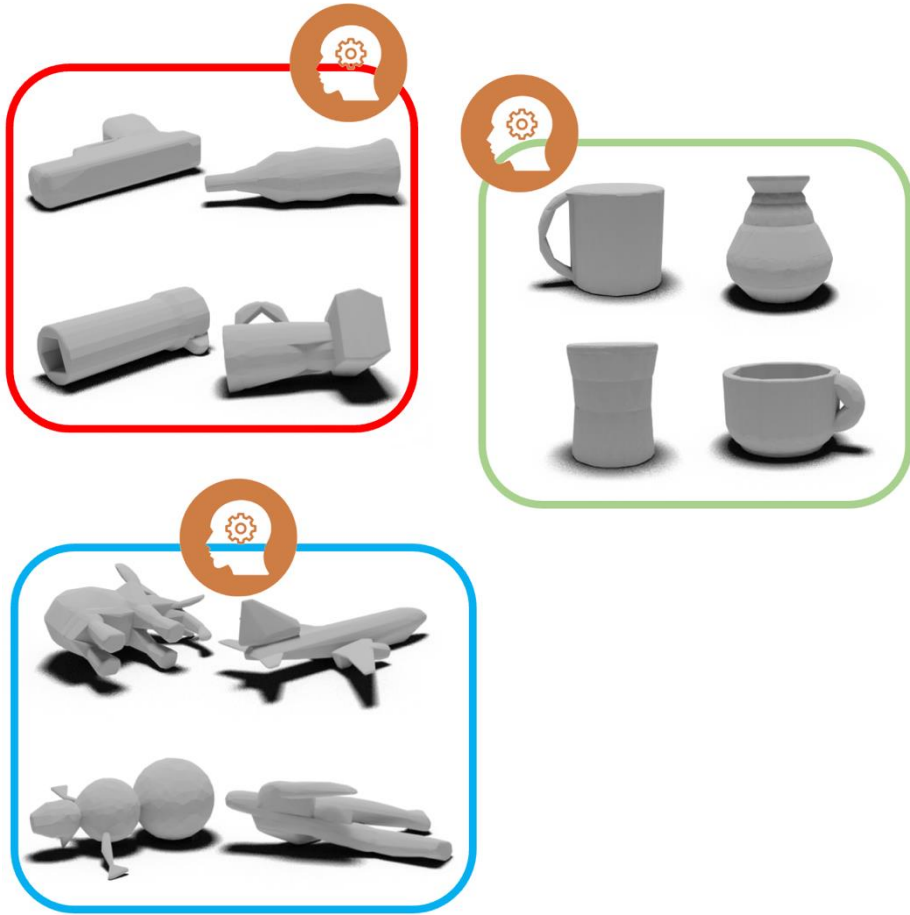
Privileged Agent:

- Fully (perfectly) observes the world state
- Predicts high-level control (e.g. future waypoints for the car to follow)

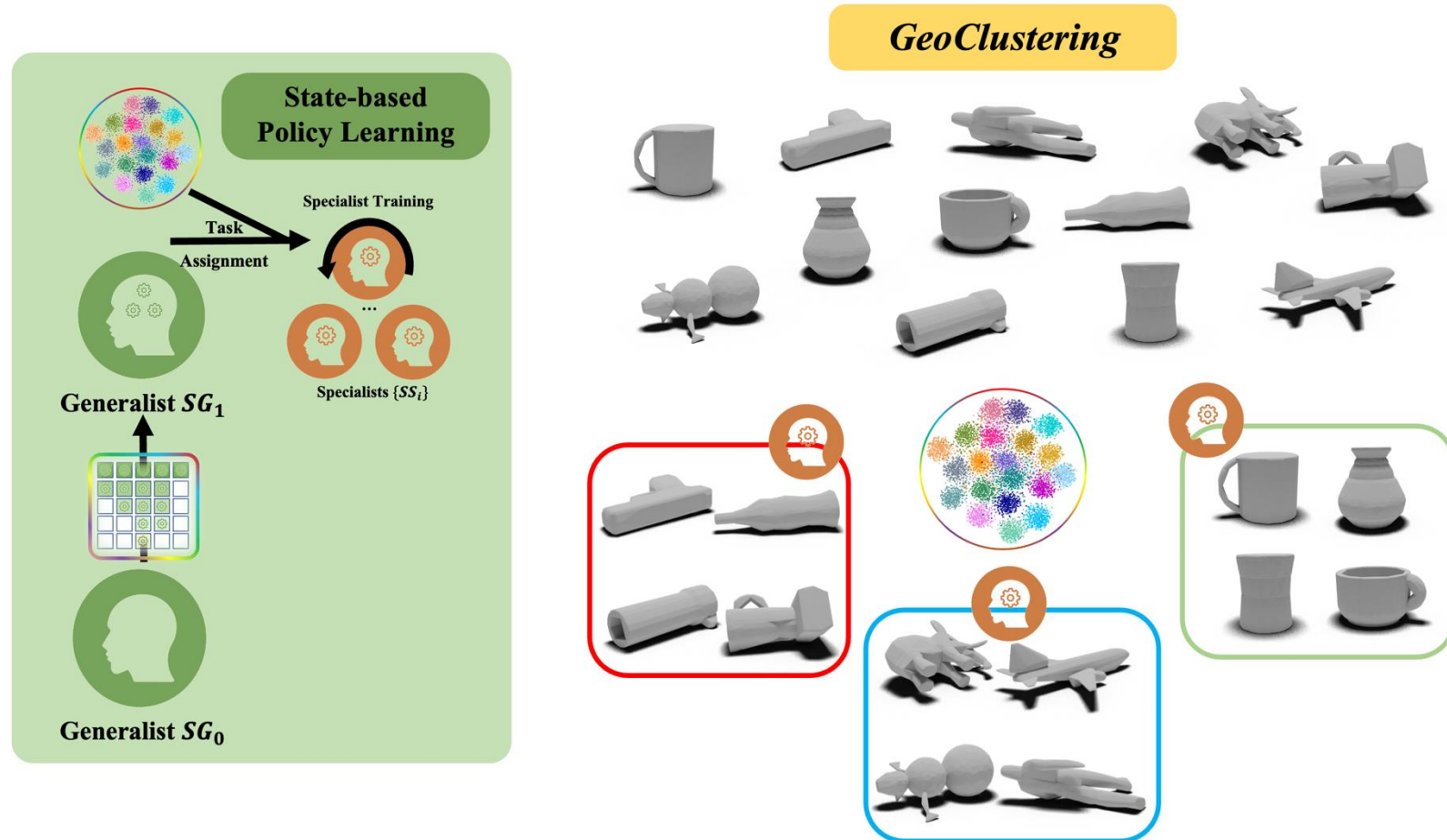
Sensorimotor Agent:

- Partially (imperfectly) observes the world state
- Predicts low-level control (e.g. steering command)

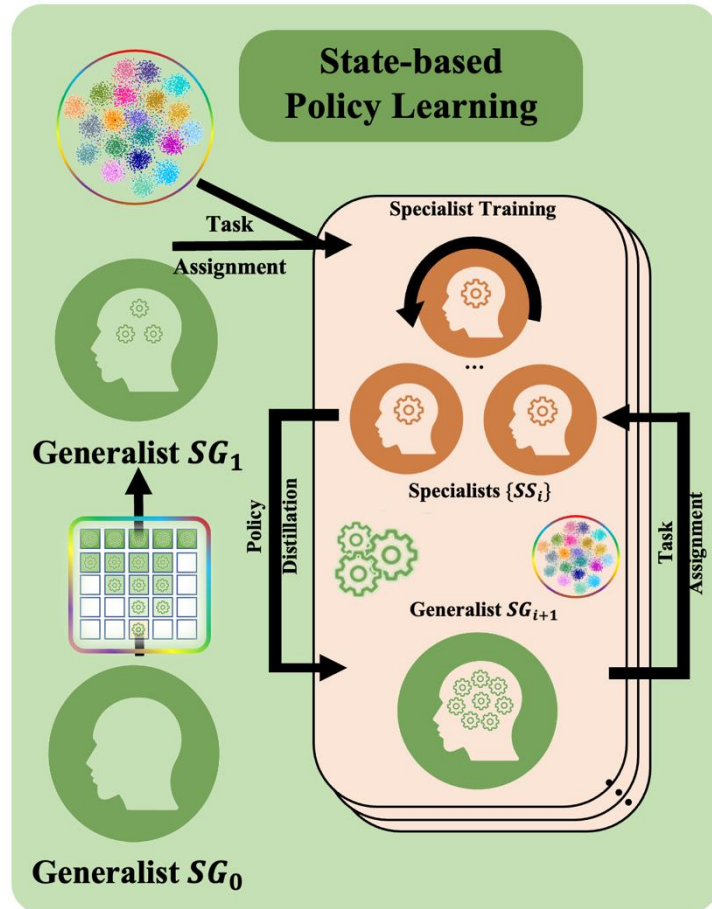
Experts can be Non-Human but Specialist Policies



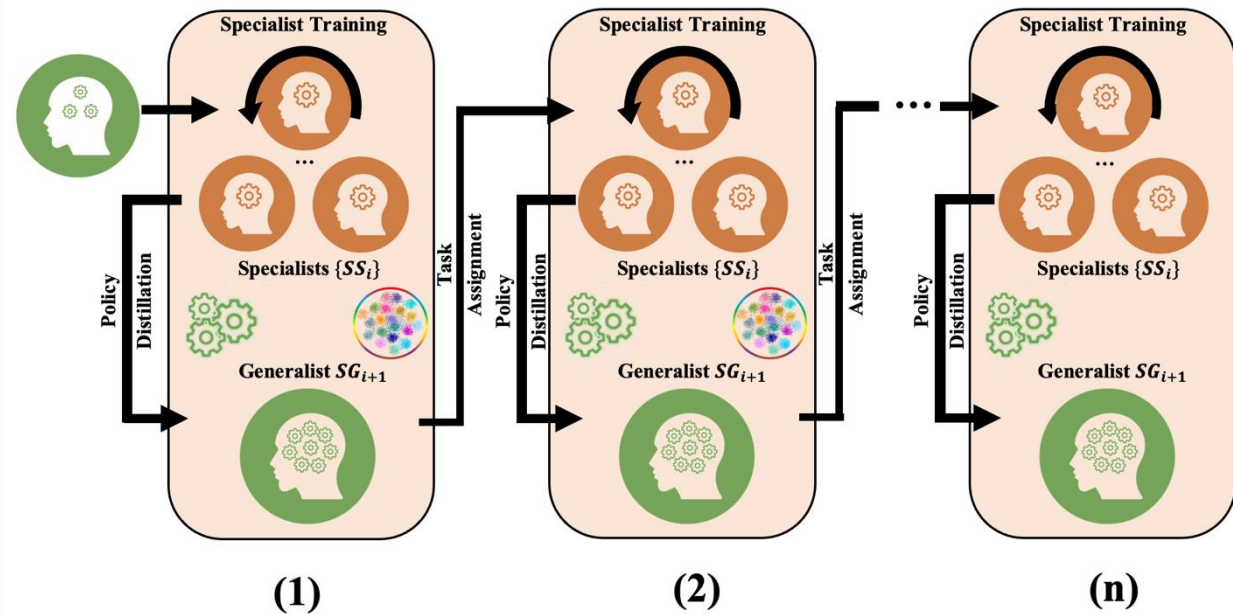
Bootstrapping from Specialist Agents Facilitates Large-scale Policy Learning



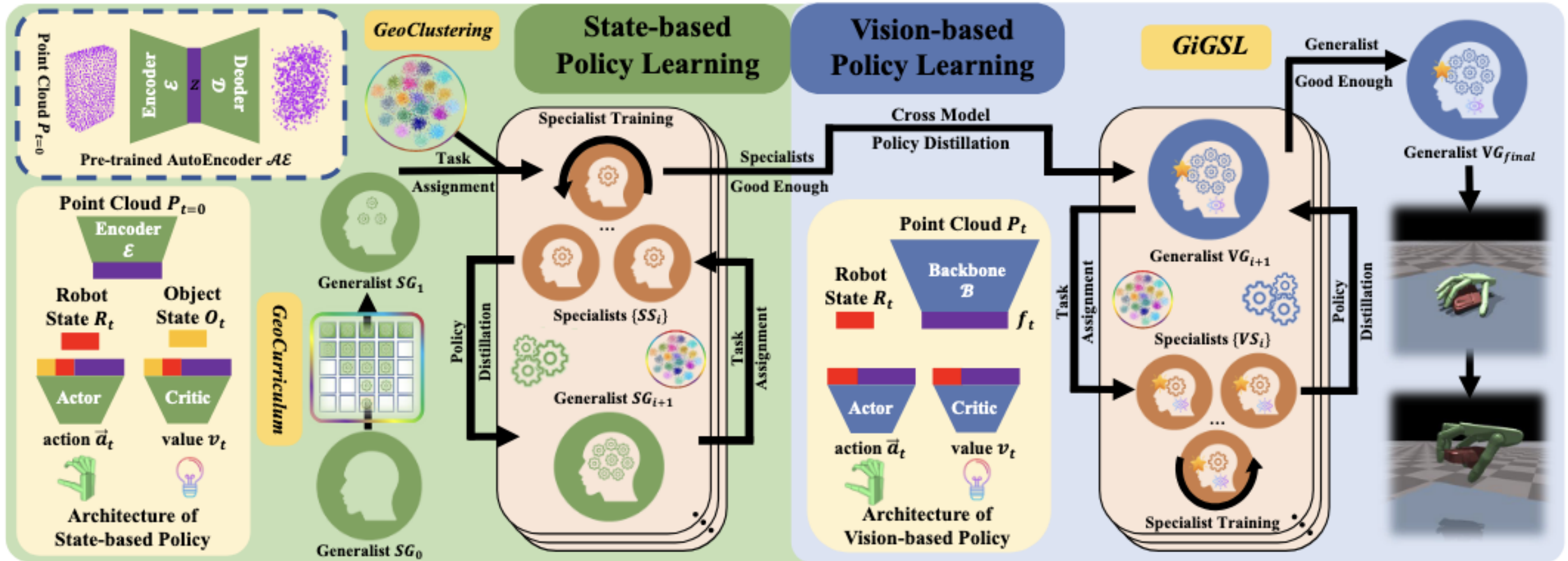
Bootstrapping from Specialist Agents Facilitates Large-scale Policy Learning



Geometry-aware iterative Generalist-Specialist Learning (GiGSL)



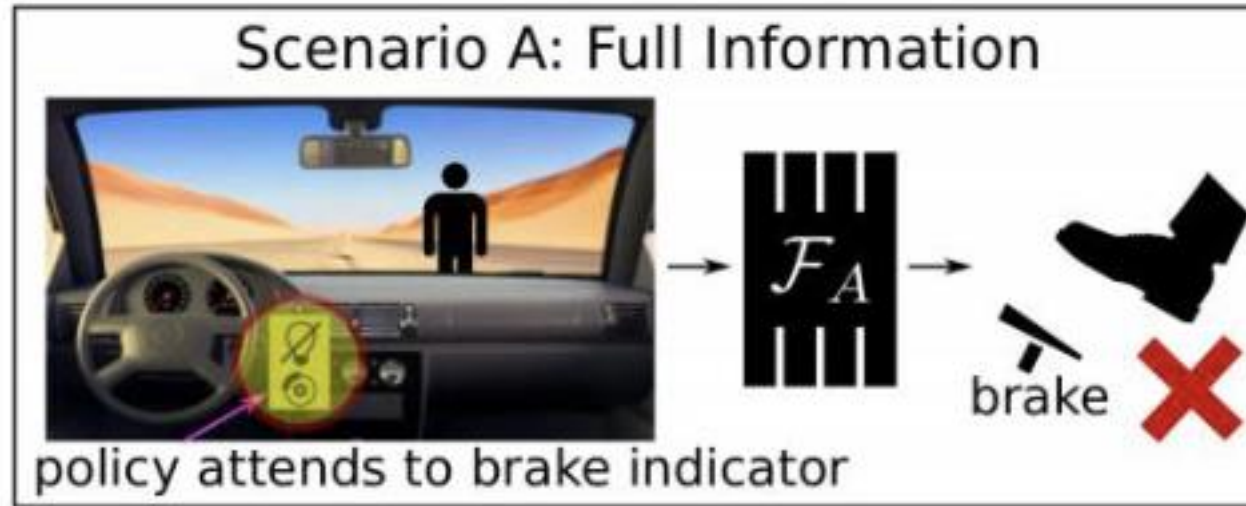
Bootstrap Policy Learning from Specialist Agents and Privileged Agents



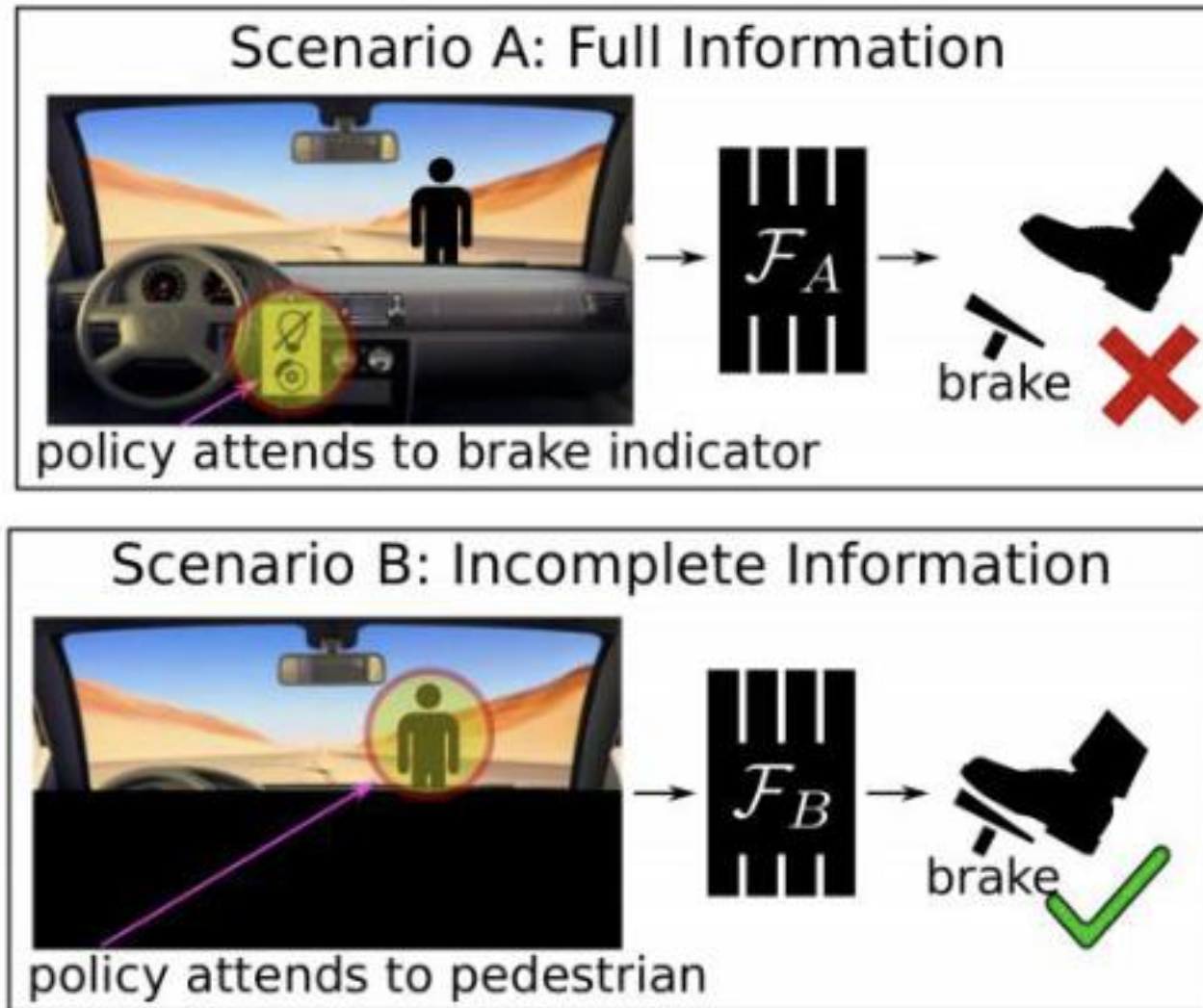
Is Behavior Cloning + DAGGER All You Need?

- Of course, No!
- More problems:
 - **Causal confusion.** Does your model look at the correct feature so that it generalizes to novel scene?

Causal Confusion: Look at Wrong Features



Causal Confusion: Look at Wrong Features



Is Behavior Cloning + DAGGER All You Need?

- Of course, No!
- More problems:
 - **Causal confusion.** Does your model look at the correct feature so that it generalizes to novel scene?
 - **Multi-modality behaviors.** What if there are multiple ways to achieve a goal? What is the better formulation of the model output?

Multi-modality Behaviors: The Same Task can be Done in Different Ways

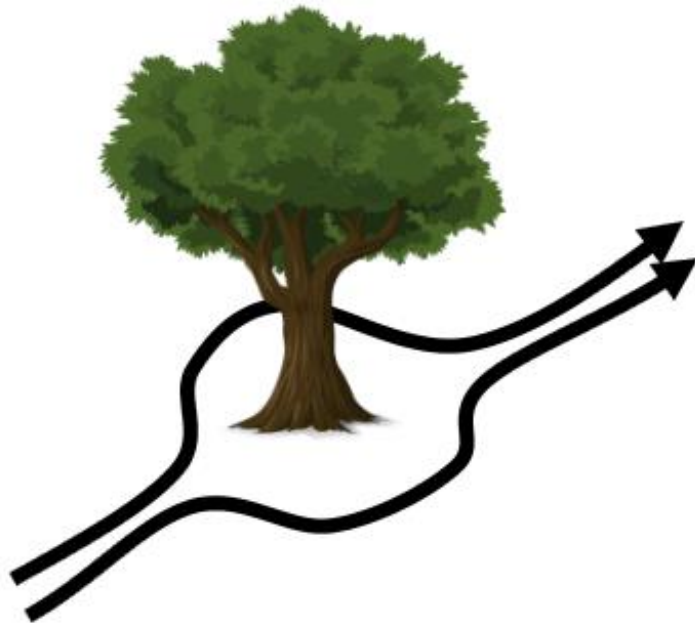
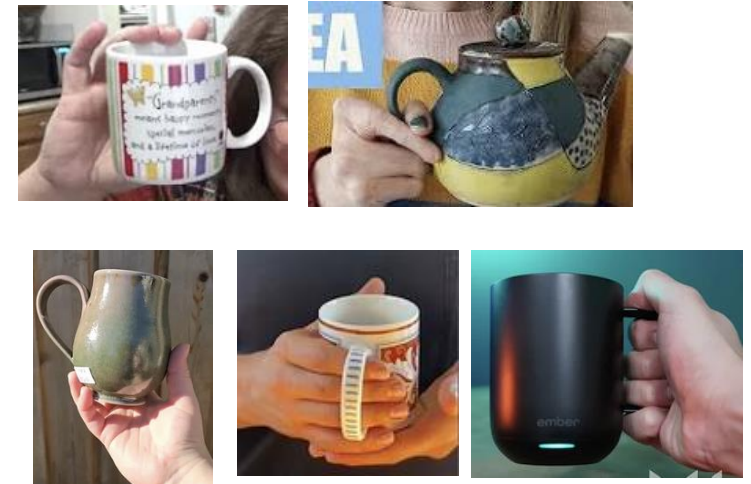


Image credit S. Levine]



Diffusion policy: Visuomotor policy learning via action diffusion

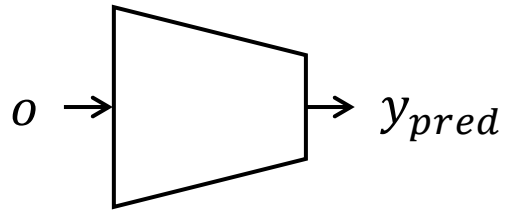


How to model multi-modal behaviors?

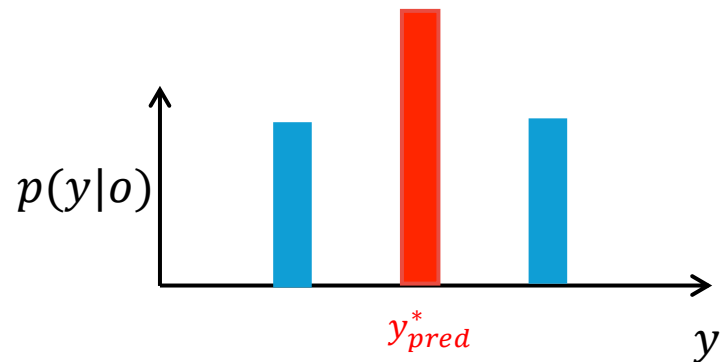
Deterministic Regression Approximates Action Average

Deterministic

- predicts the same output given the same input



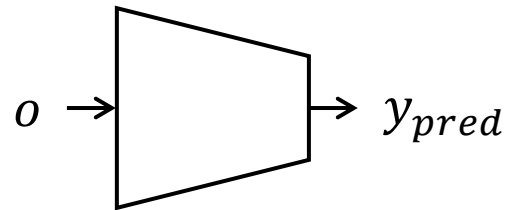
$$y_{pred}^* = \underset{y_{pred}}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^N |y_{pred} - y_n|^2$$



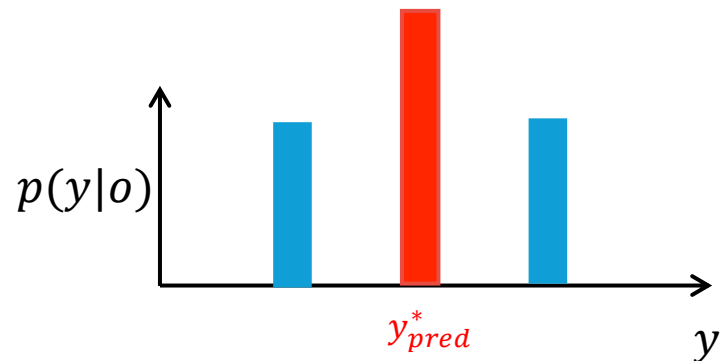
Probabilistic Regression Captures Distribution of Action

Deterministic

- predicts the same output given the same input

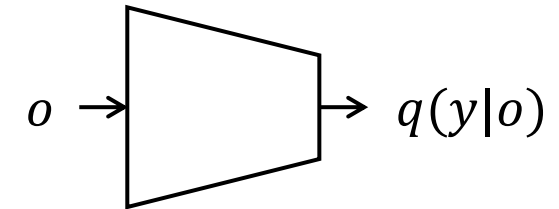


$$y_{pred}^* = \operatorname{argmin}_{y_{pred}} \frac{1}{N} \sum_{n=1}^N |y_{pred} - y_n|^2$$

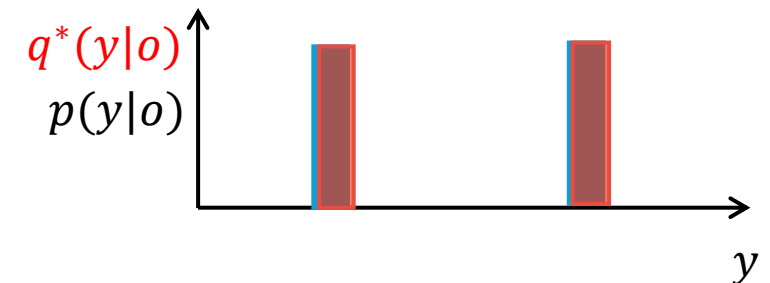


Probabilistic

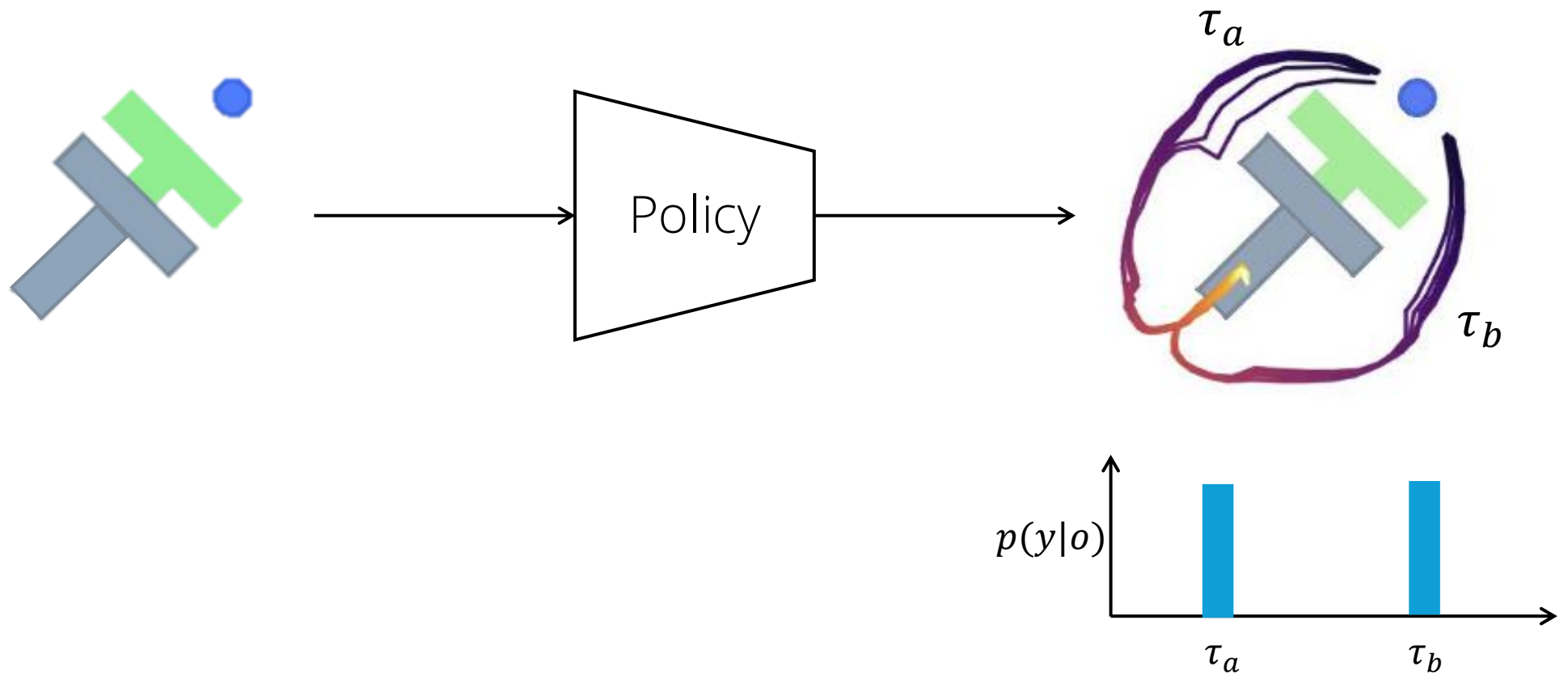
- predicts the same likelihood of all output given the same input



$$q^*(y|o) = \operatorname{argmin}_{q^*} D_{KL}(q(y|o), p(y|o))$$



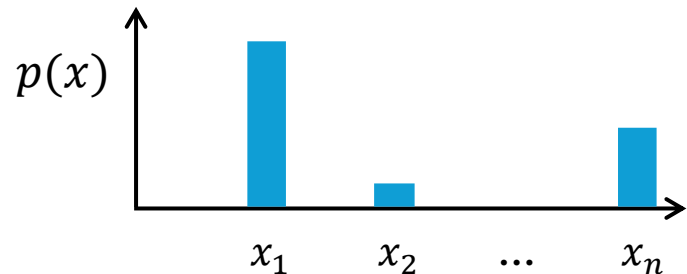
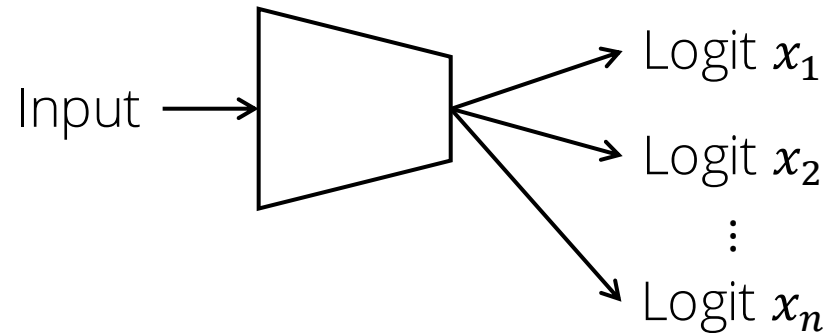
Model Multi-modality Behaviors with Probabilistic Models



Choices of Probabilistic Generative Models

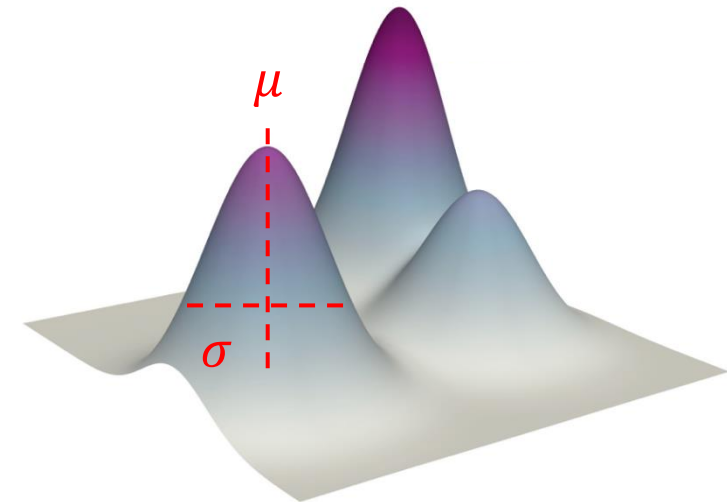
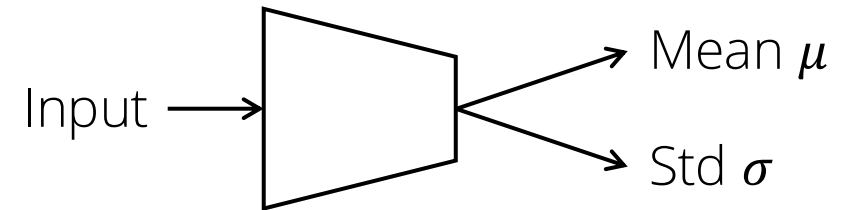
Categorical

- Predict the categorical probability of sample "x"
- Most LLMs etc



Gaussian

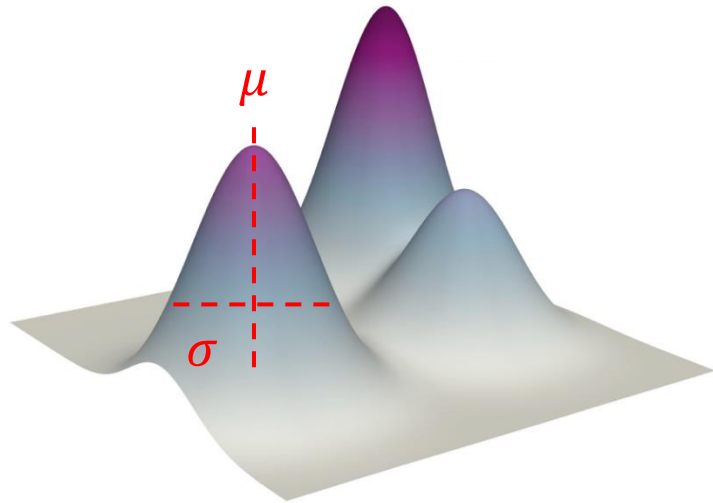
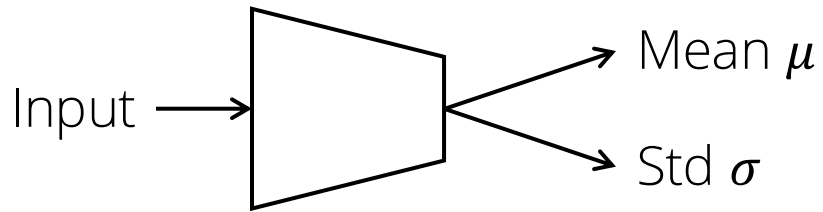
- Predict the mean and standard deviation to model the probability of sample "x"
- VAE, Diffusion models etc



Choices of Probabilistic Generative Models

Gaussian

- Predict the mean and standard deviation to model the probability of sample “x”
- VAE, Diffusion models etc



Mixture of Gaussians

$$\pi(\mathbf{a}|\mathbf{o}) = \sum_i w_i \mathcal{N}(\mu_i, \Sigma_i)$$

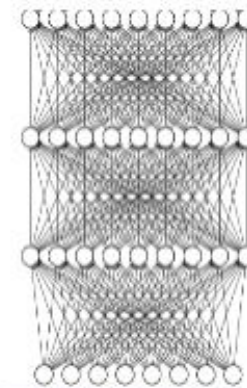
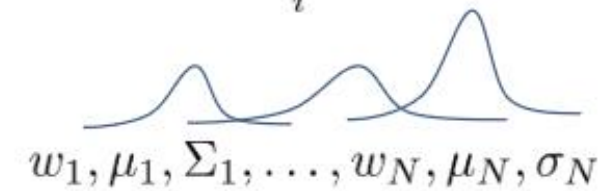
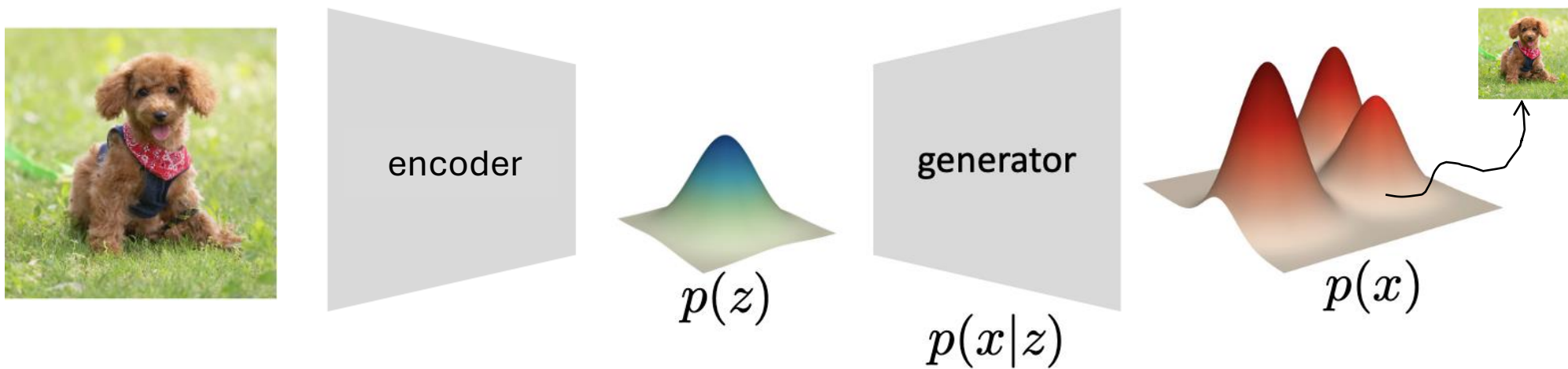
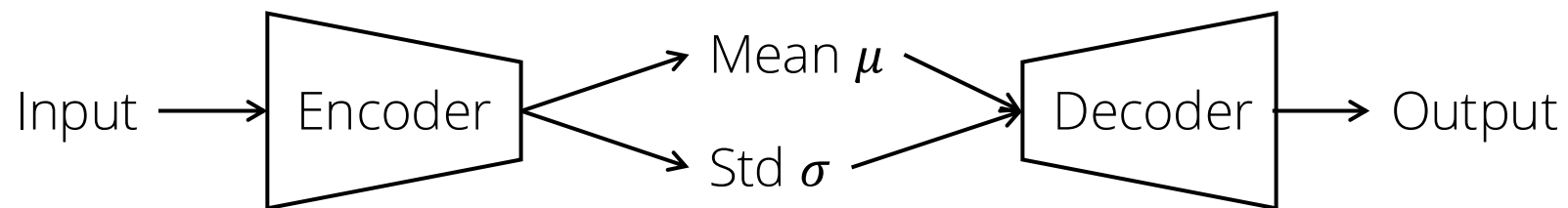
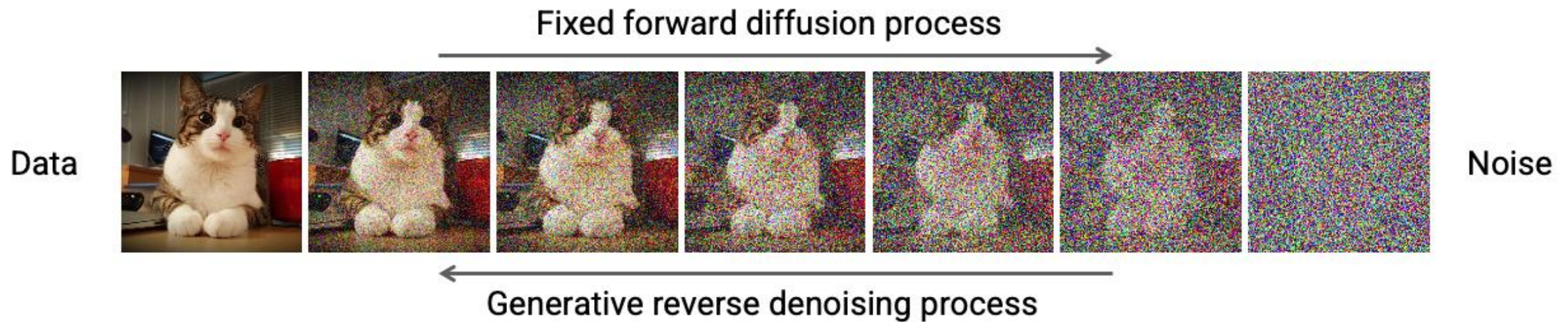


Image credit S. Levine

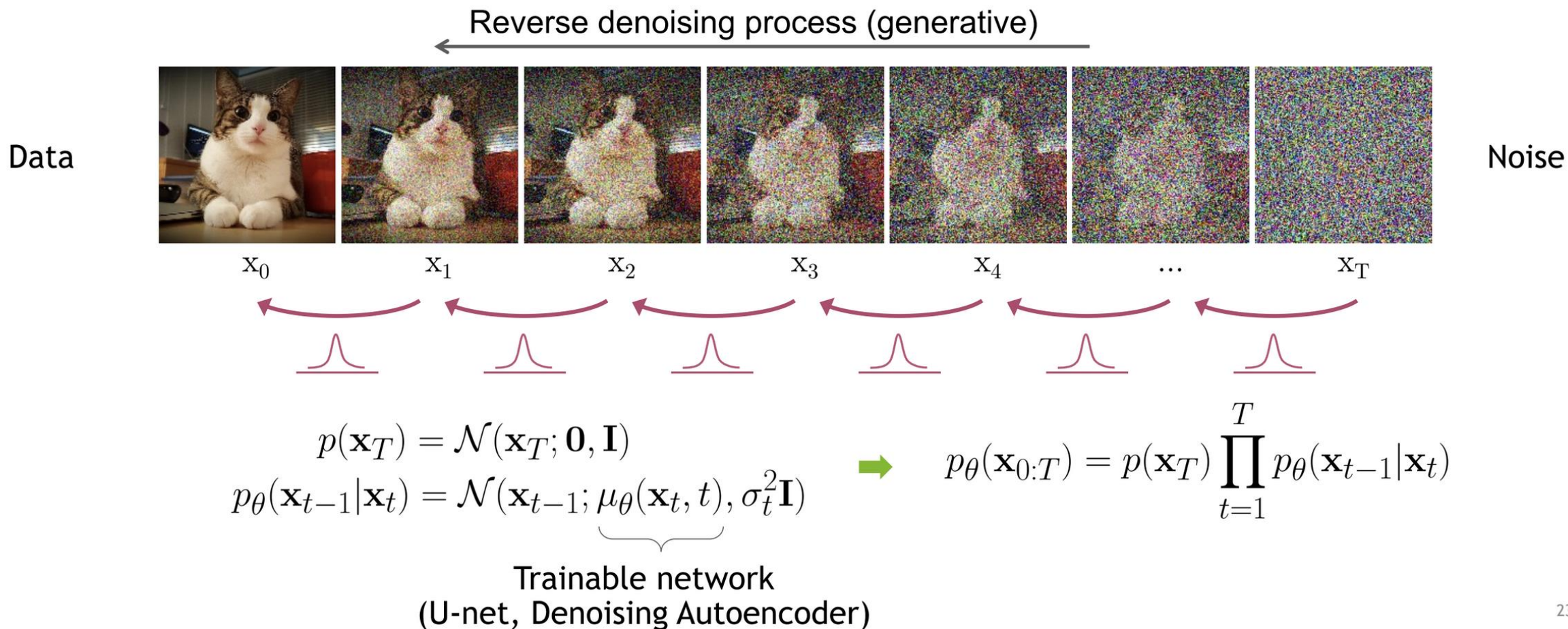
Example: Variational Auto-Encoder



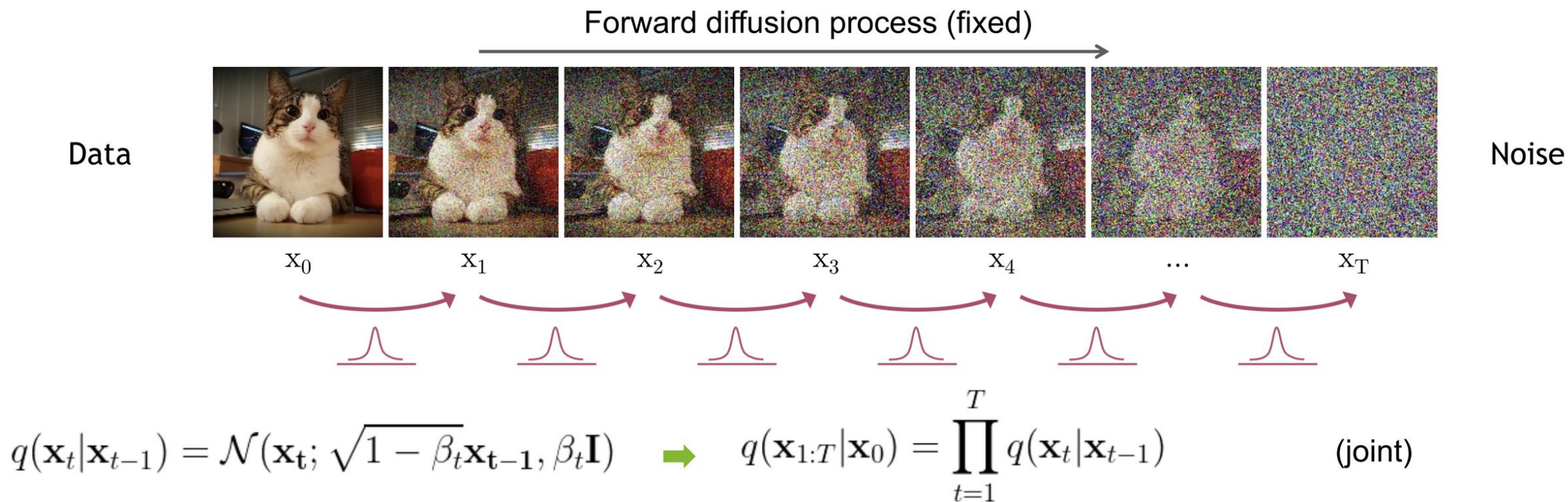
Example: Denoising Diffusion Probabilistic Models



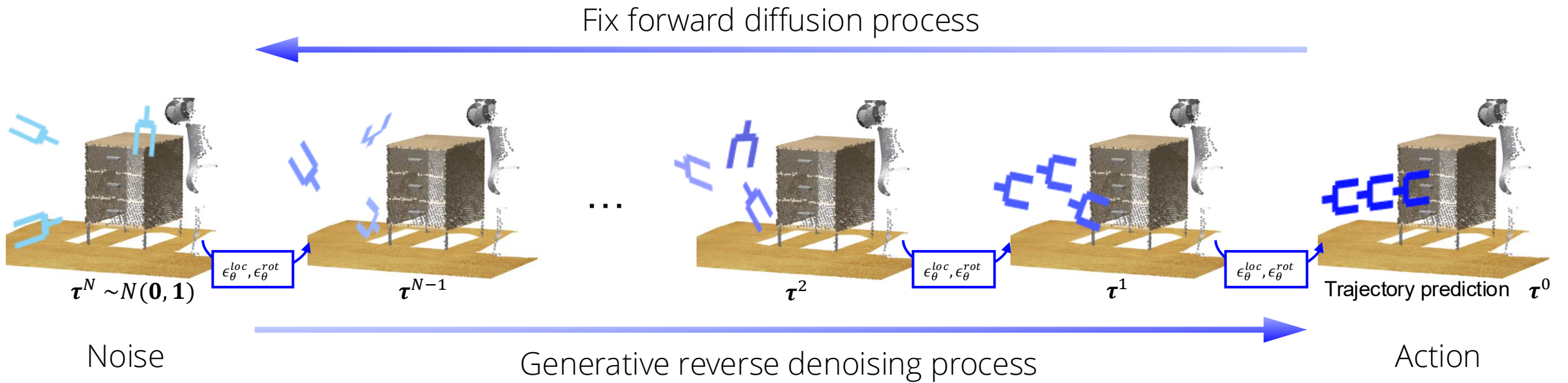
Denoising Diffusion Probabilistic Models (DDPM)



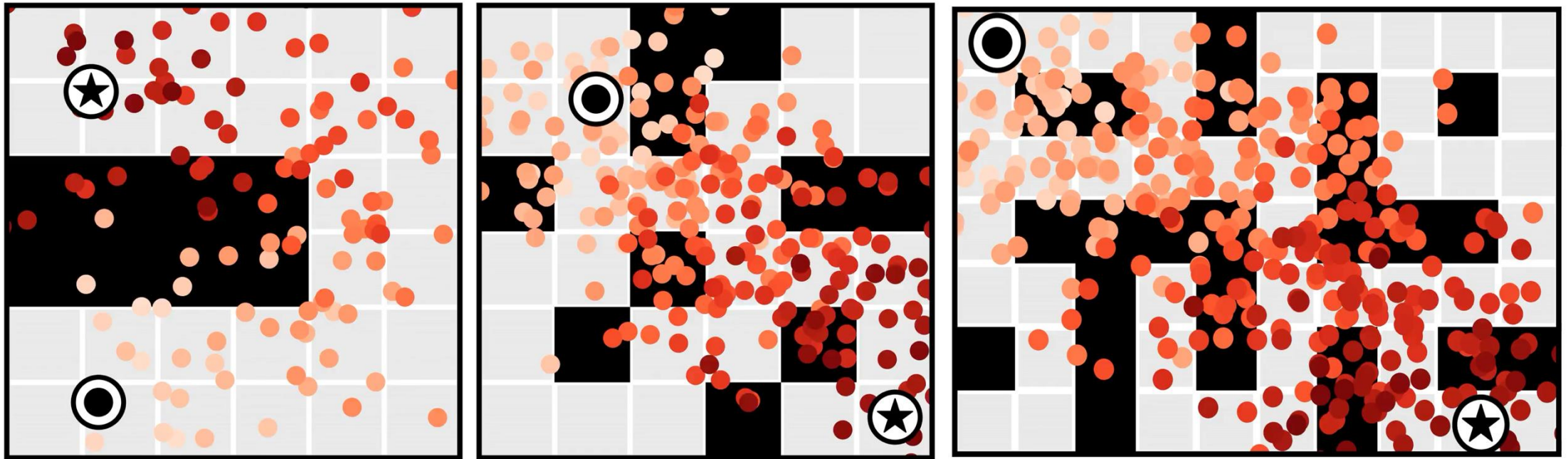
Denoising Diffusion Probabilistic Models (DDPM)



Learning DDPM Models as Robot Policies

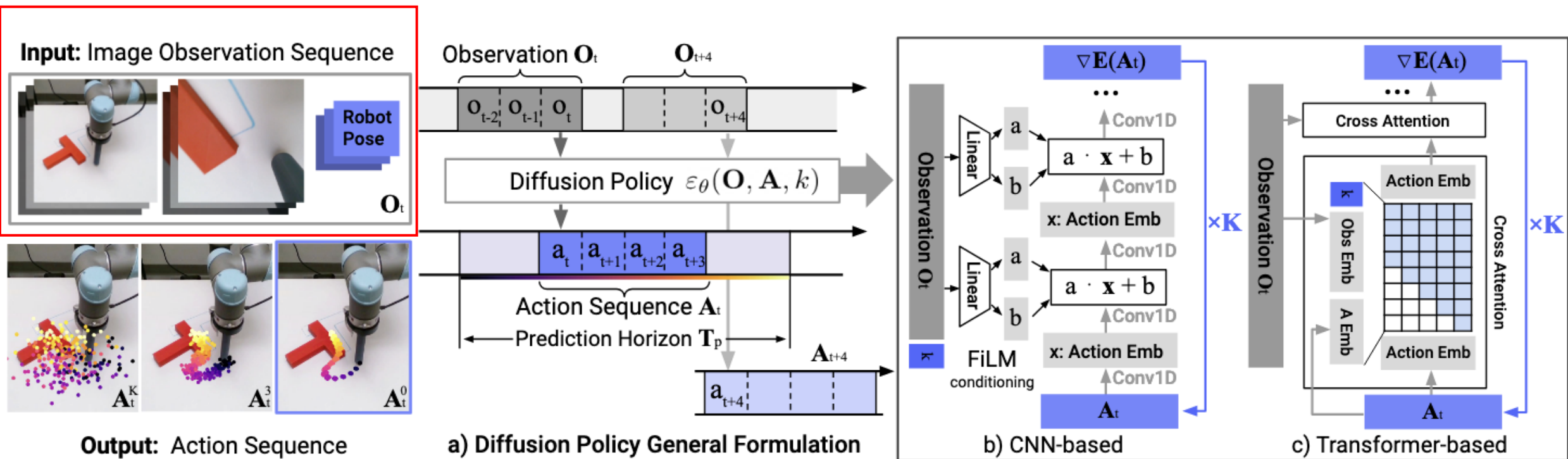


Learning DDPM Models as Robot Policies



DDPM Policy with 2D Inputs

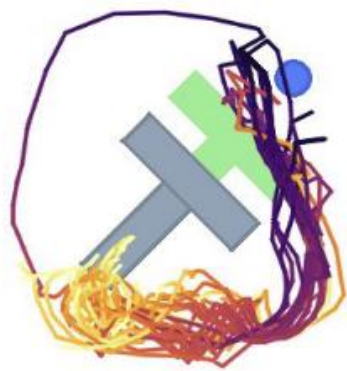
An input image is represented by a flattened feature vector



DDPM Policy with 2D Inputs



Diffusion Policy



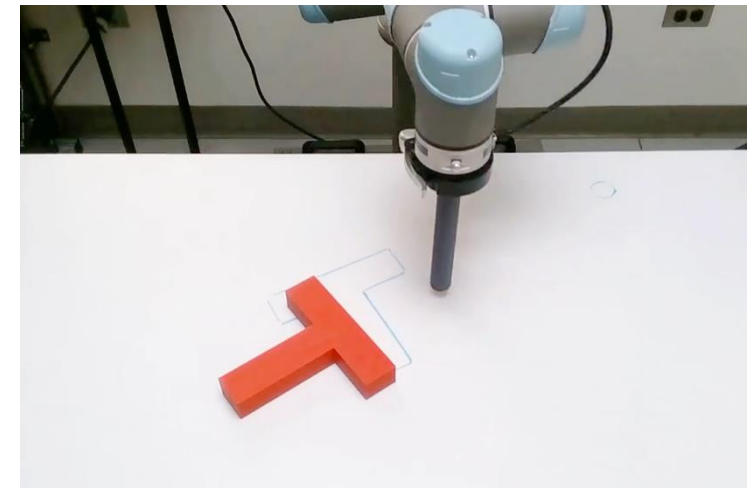
LSTM-GMM



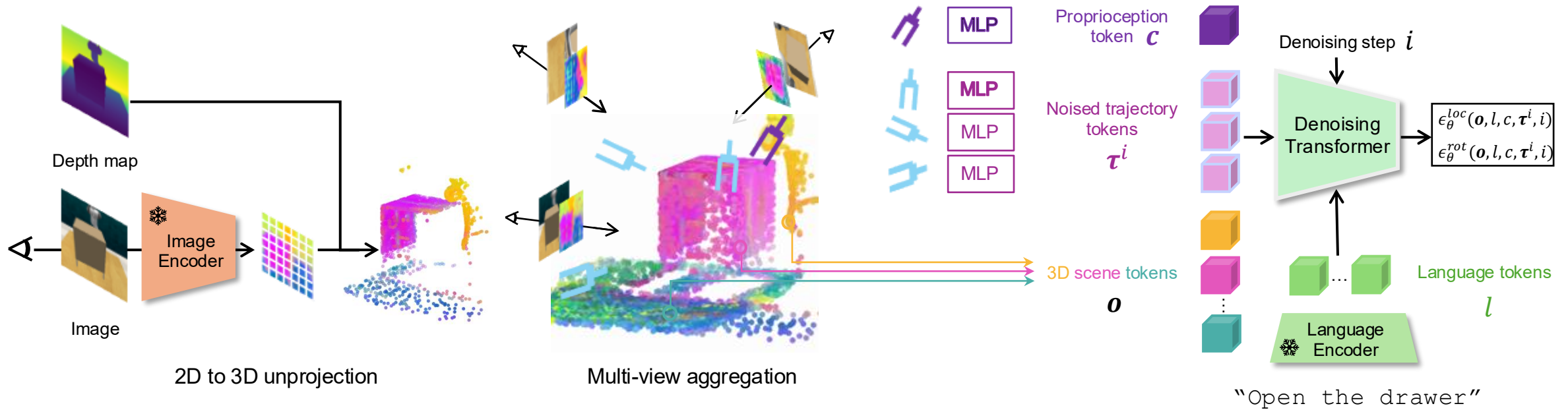
BET



IBC



DDPM Policy with 3D Inputs



DDPM Policy with 3D Inputs

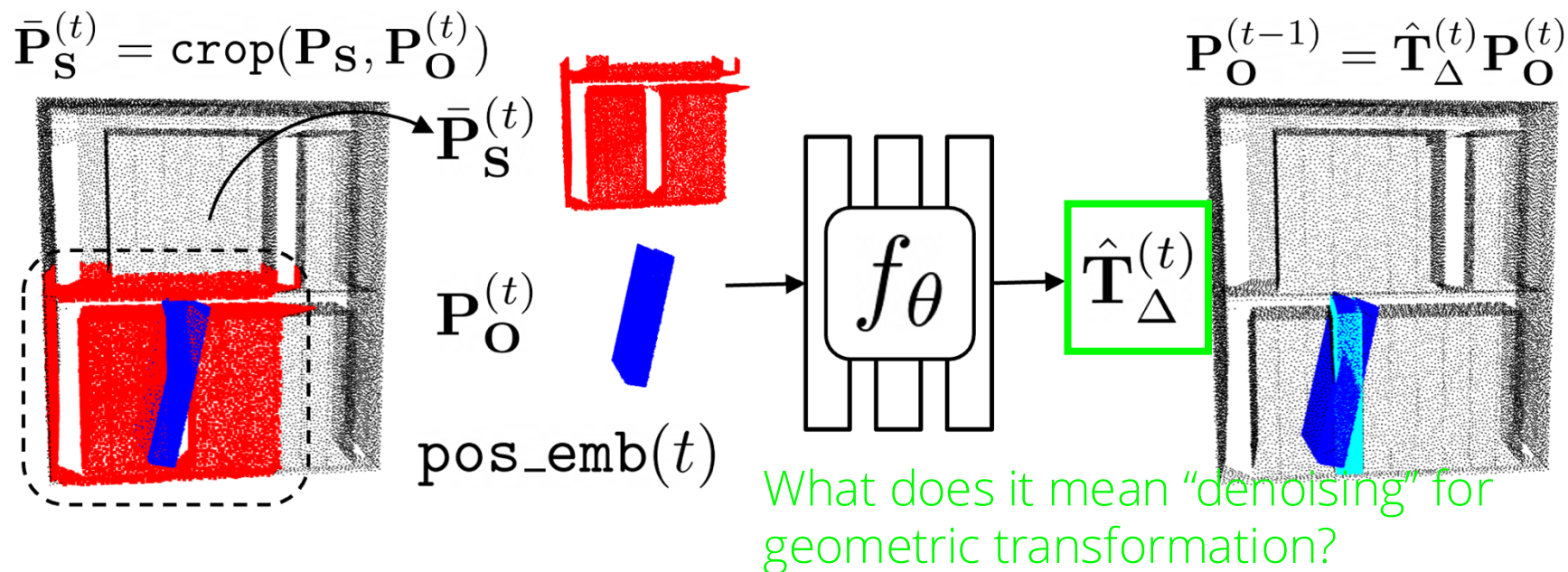
Calvin

Real-world



Demo Time

Can We Generate Other Pose Representations?



$\mathbf{P}_O^{(0)}$: (clean) object point cloud at diffusion step 0

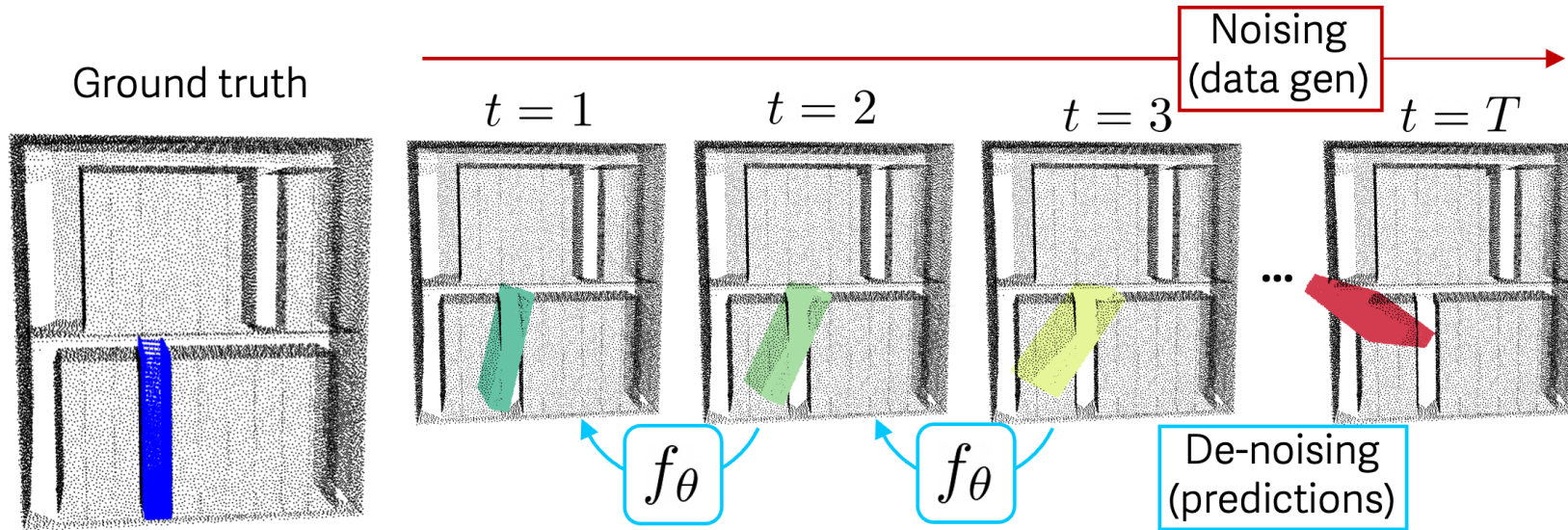
$\mathbf{P}_O^{(t)}$: (perturbed) object point cloud at diffusion step t

$\mathbf{P}_S$: the whole scene point cloud

$\mathbf{P}_S^{(t)}$: cropped scene point cloud around the object point cloud at diffusion step t

$\hat{\mathbf{T}}_\Delta^{(t)}$: de-perturbed transformation from diffusion step t to $t - 1$

De-noising Process

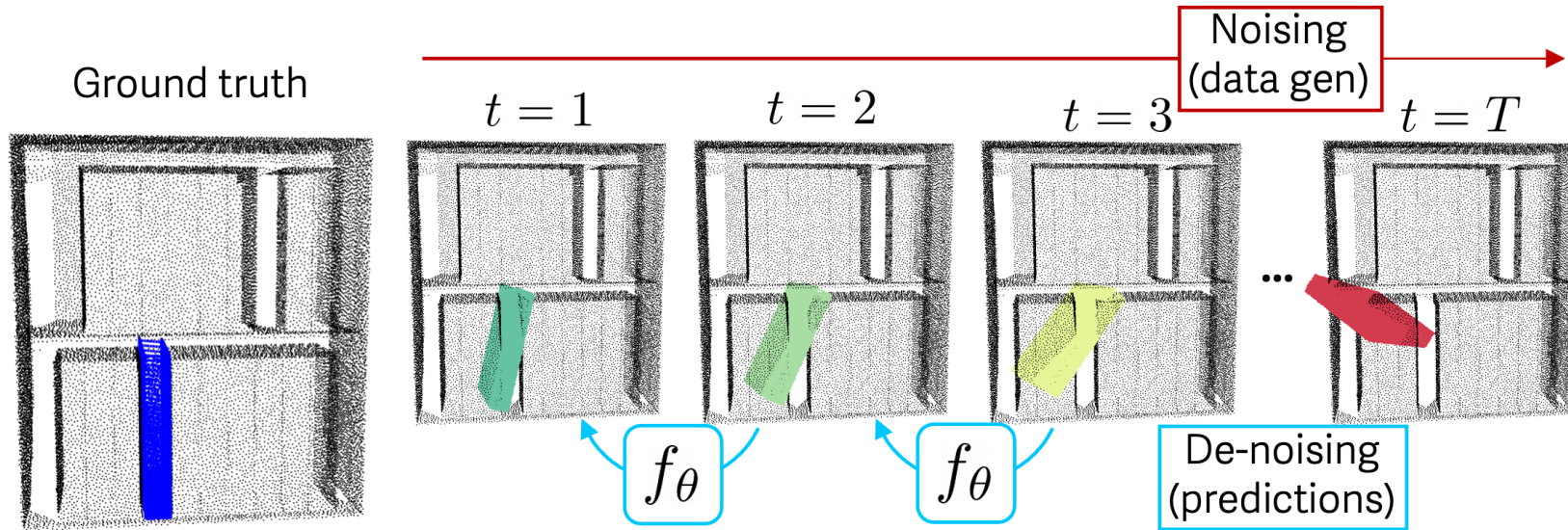


$$\mathbf{T}_{\Delta}^{(i)} = \mathbf{T}_{\Delta}^{\text{Rand}} f_{\theta} \left(\hat{\mathbf{P}}_{\mathbf{O}}^{(i)}, \mathbf{P}_{\mathbf{S}}, \text{pos_emb}(t) \right) \quad t = \text{i_to_t}(i)$$

$$\hat{\mathbf{T}}^{(i-1)} = \mathbf{T}_{\Delta}^{(i)} \hat{\mathbf{T}}^{(i)} \quad \hat{\mathbf{P}}_{\mathbf{O}}^{(i-1)} = \mathbf{T}_{\Delta}^{(i)} \hat{\mathbf{P}}_{\mathbf{O}}^{(i)}$$

De-noising from diffusion step t to $t - 1$

De-noising Process



Remember DDPM?

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

$$\mathbf{T}_\Delta^{(i)} = \mathbf{T}_\Delta^{\text{Rand}} f_\theta \left(\hat{\mathbf{P}}_O^{(i)}, \mathbf{P}_S, \text{pos_emb}(t) \right) \quad t = \text{i_to_t}(i)$$

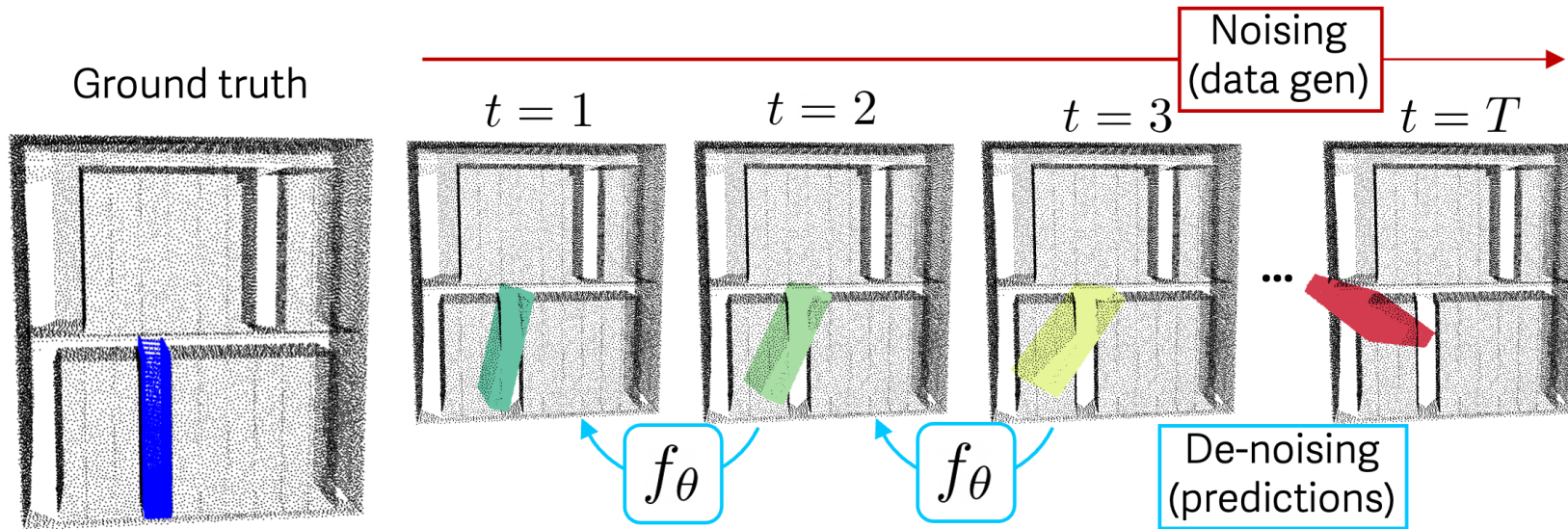
$$\hat{\mathbf{T}}^{(i-1)} = \mathbf{T}_\Delta^{(i)} \hat{\mathbf{T}}^{(i)} \quad \hat{\mathbf{P}}_O^{(i-1)} = \mathbf{T}_\Delta^{(i)} \hat{\mathbf{P}}_O^{(i)}$$

How to Generate Training Labels?

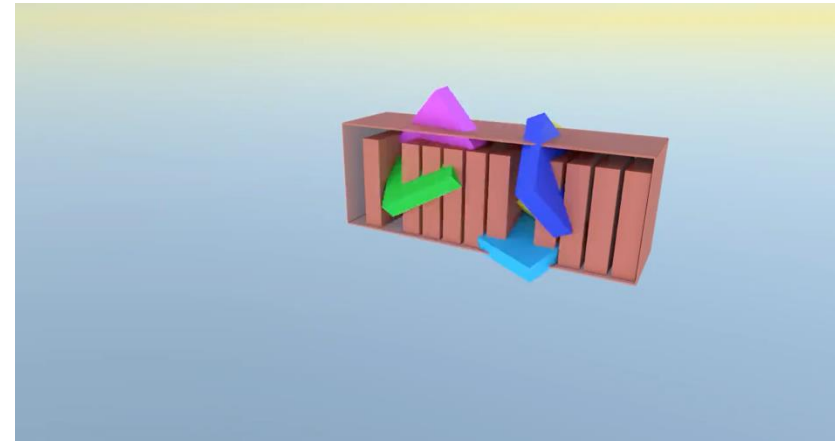
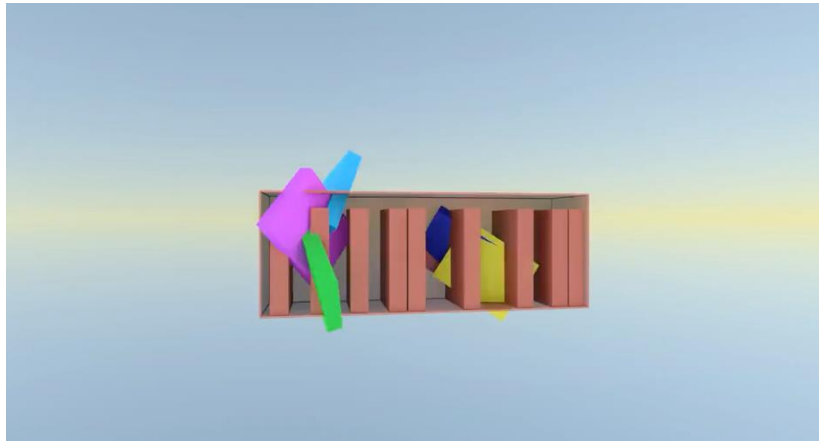
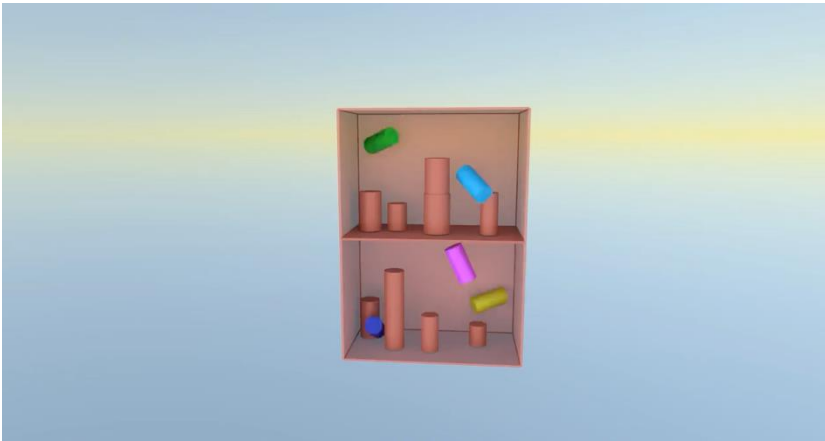
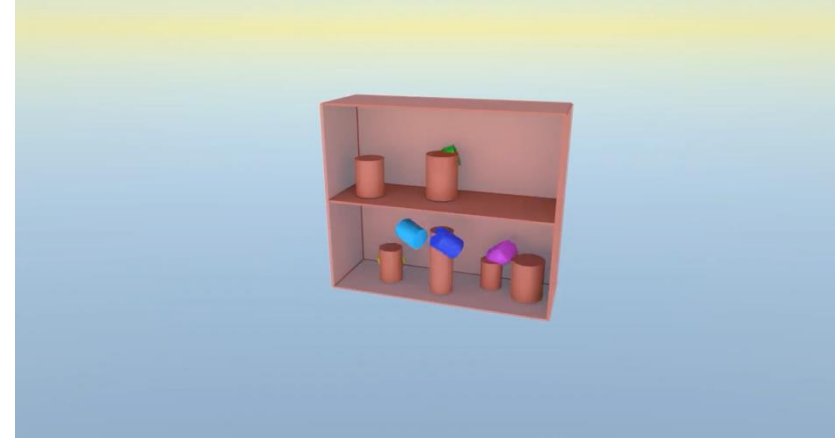
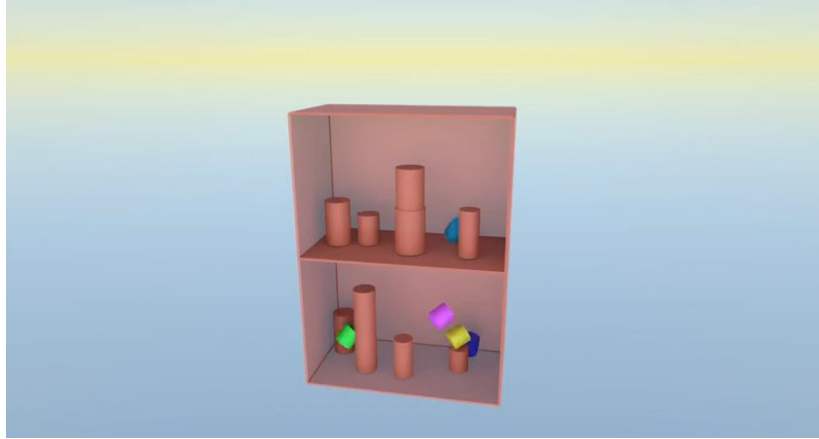
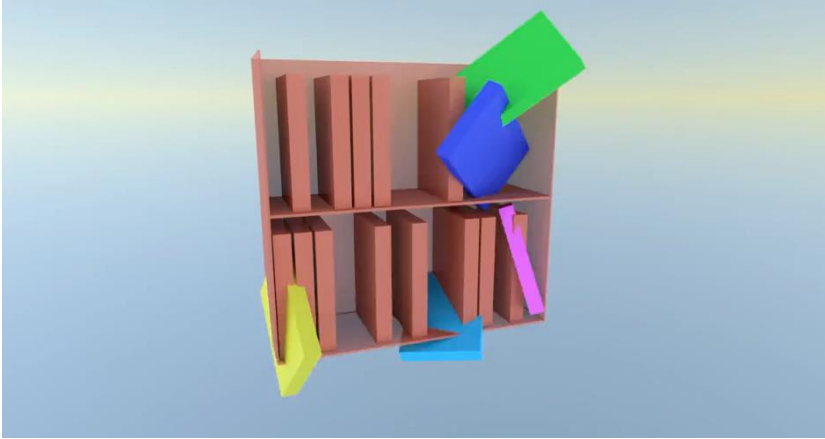
Sample a random transformation at diffusion step T ,
and find the interpolated intermediate transformation

$$\{\mathbf{T}_{\text{interp}}^{(s)}\}_{s=1}^t = \text{interp}(\mathbf{T}_{\text{noise, inv}}^{(t)}, t)$$

$$\mathbf{T}_{\Delta, \text{GT}}^{(t)} = [\mathbf{T}_{\text{interp}}^{(t-1)}]^{-1} \mathbf{T}_{\text{interp}}^{(t)}$$



Generated Results

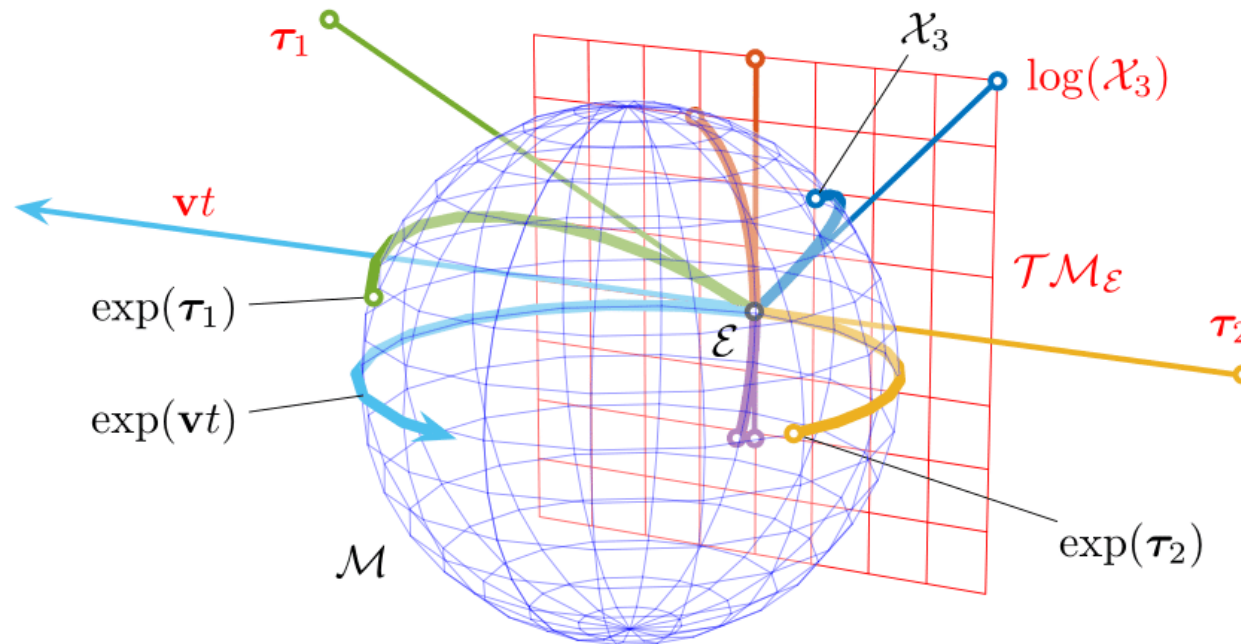


Can We Generate SE(3) Representations?

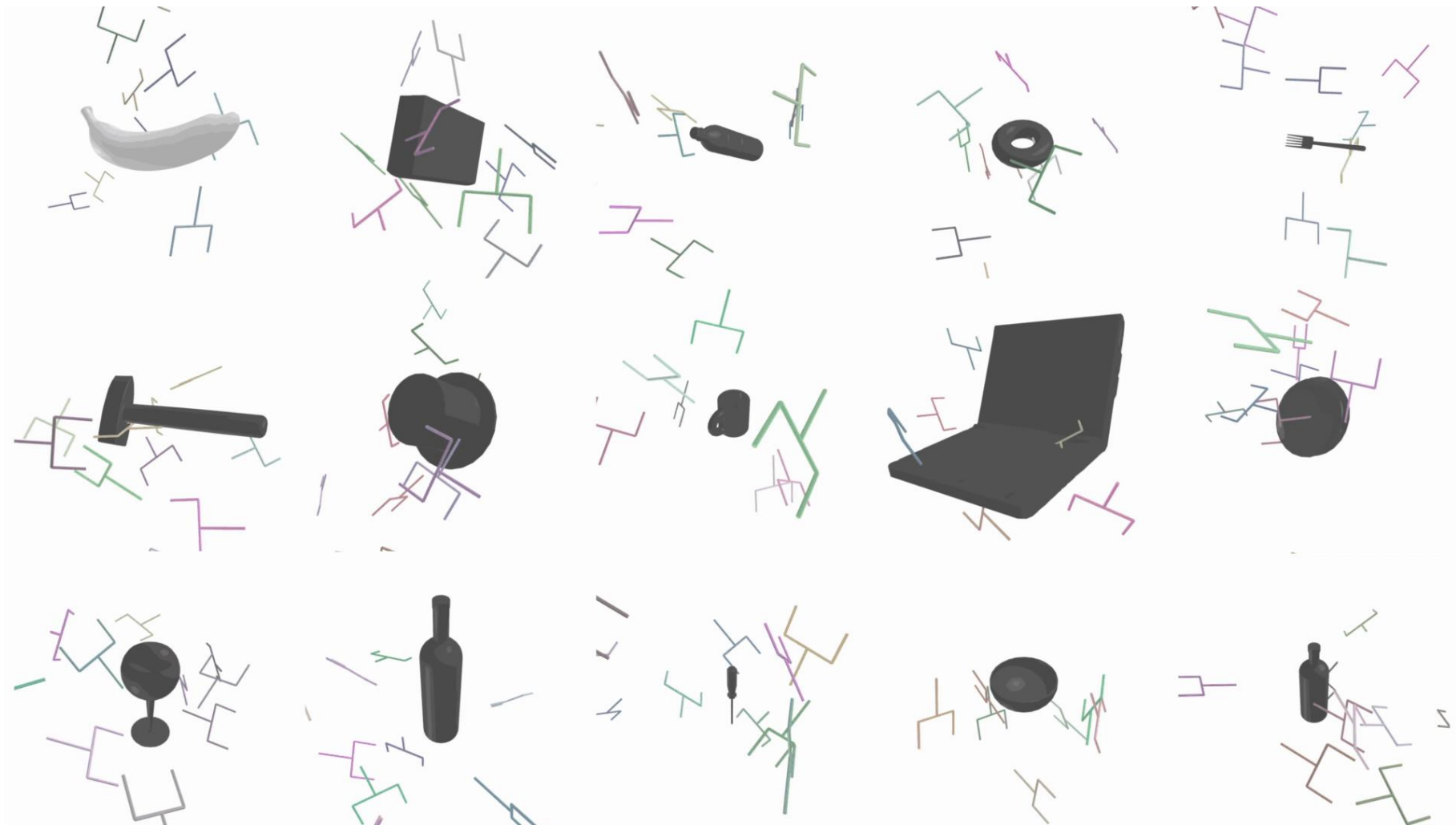
$$q(\mathbf{H}|\mathbf{H}_\mu, \mathbf{\Sigma}) \propto \exp\left(-0.5 \left\| \text{Logmap}(\mathbf{H}_\mu^{-1} \mathbf{H}) \right\|_{\mathbf{\Sigma}^{-1}}^2\right)$$

$\mathbf{H}_\mu \in SE(3)$ is the mean

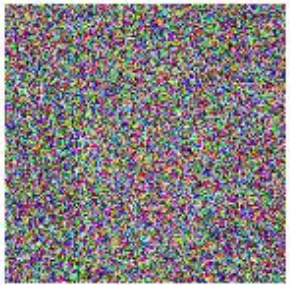
$\mathbf{\Sigma} \in \mathbb{R}^{6 \times 6}$ is the covariance matrix



Can We Generate SE(3) Representations?



Example: Flow Matching Policy



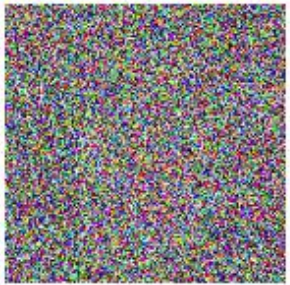
Noise



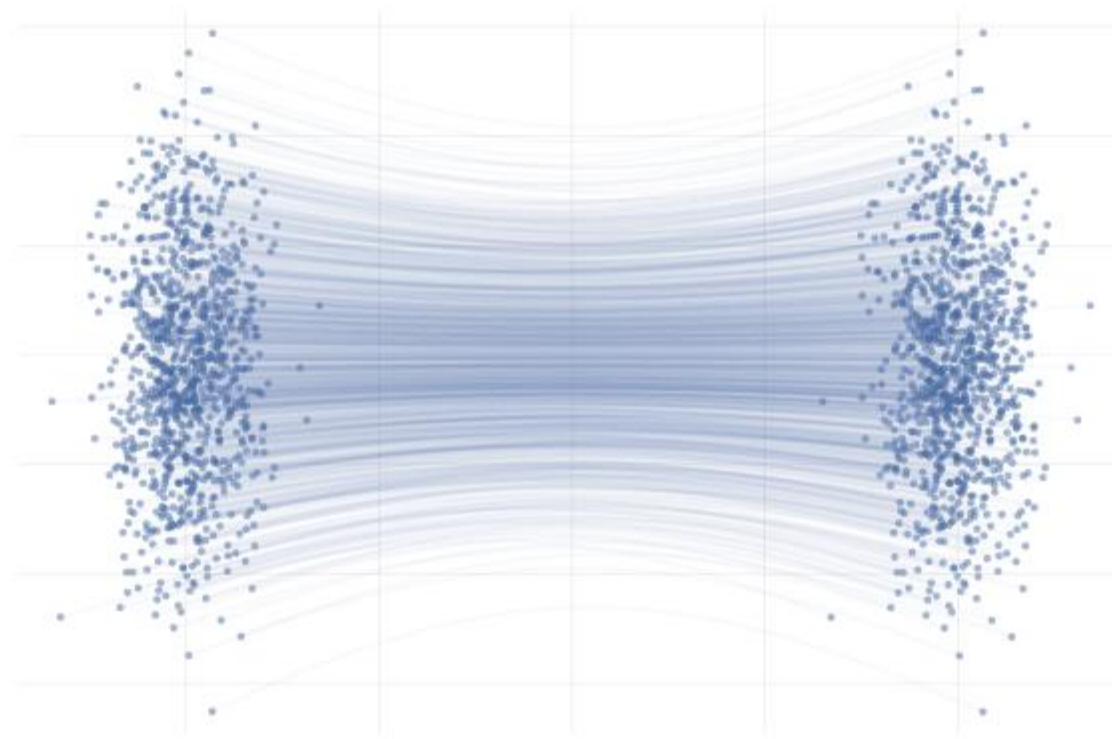
Data

Video source: <https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>

Flow Matching Policy Learn Velocity Field to Transport One Distribution to Another



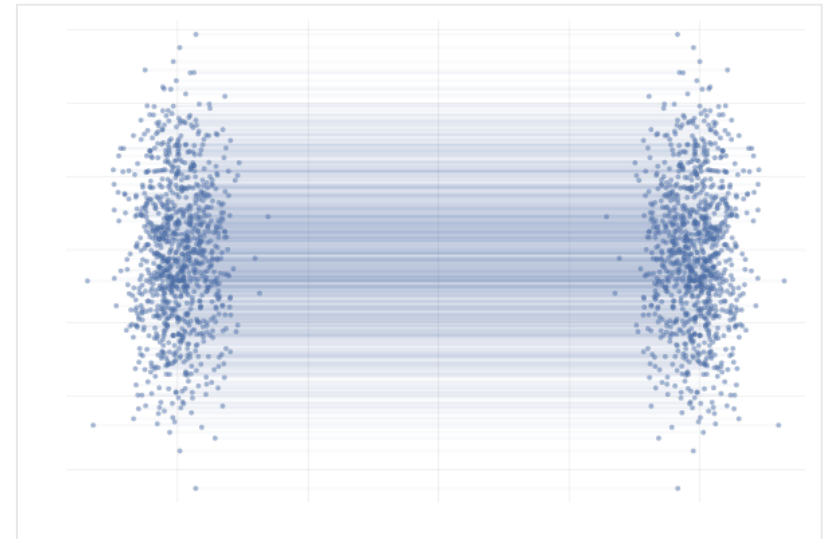
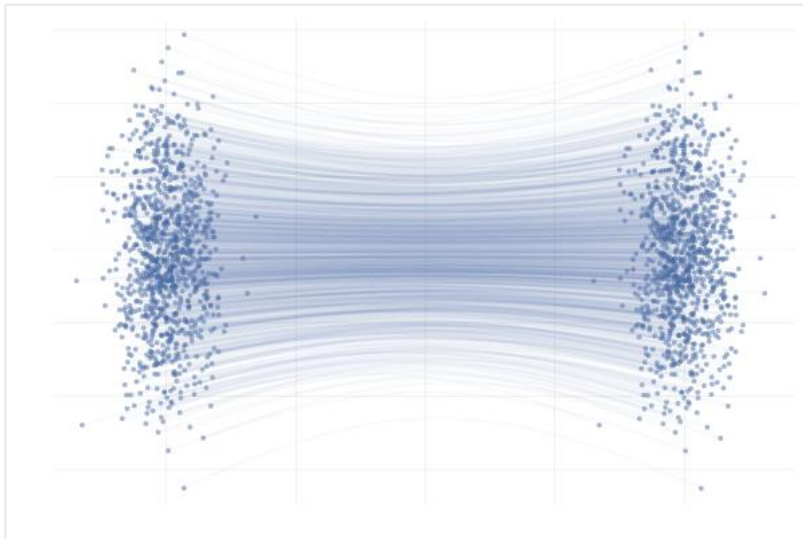
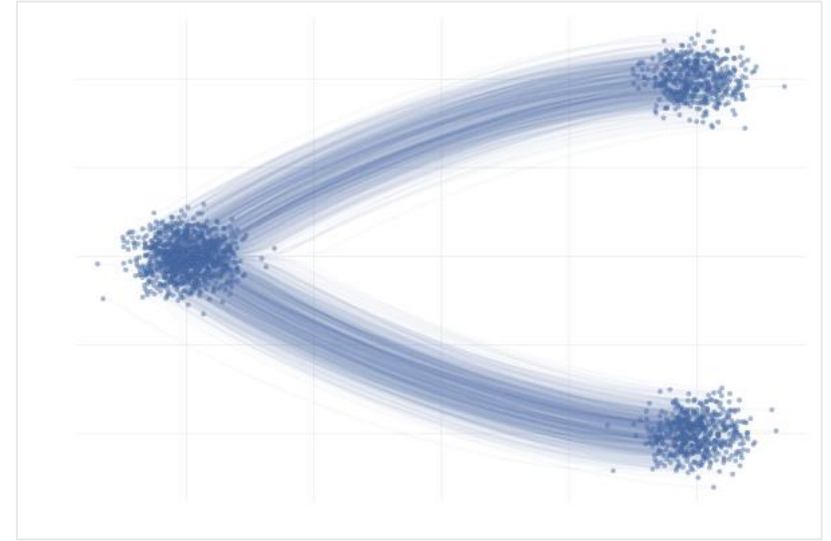
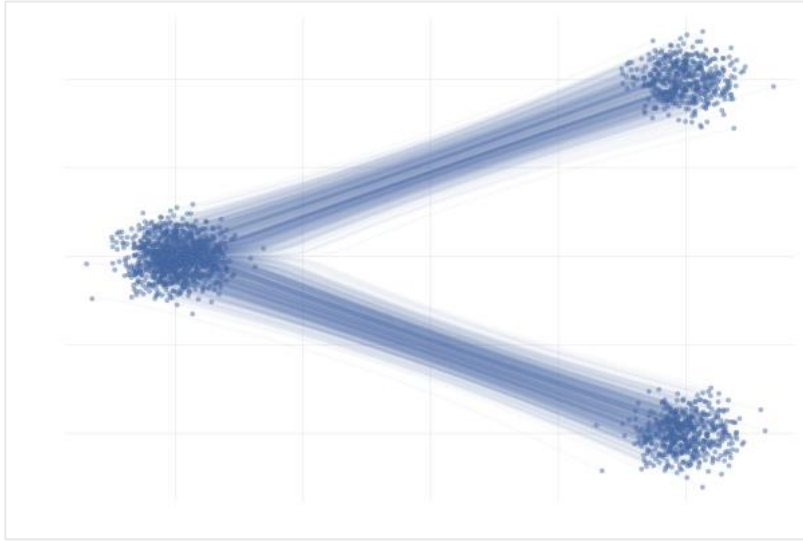
Noise



Data

Image source: <https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>

Transportation Velocity Field is not Unique

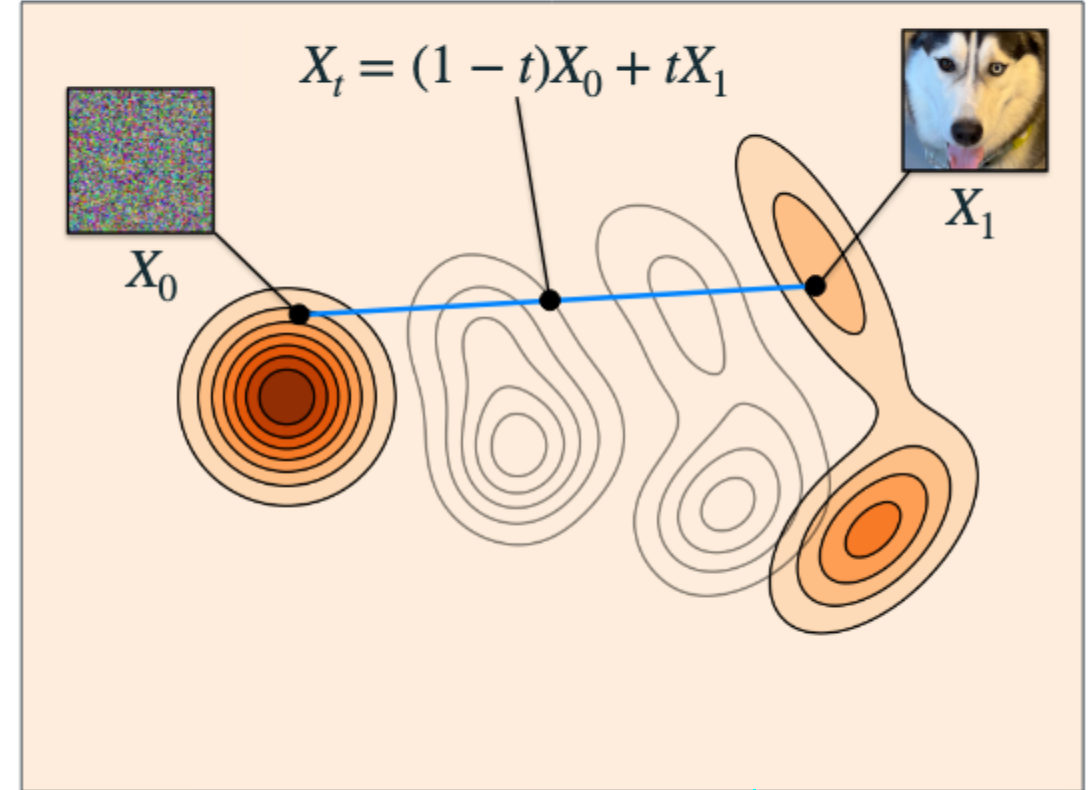


Simplest version of Flow Matching

- Arbitrary $X_0 \sim p, X_1 \sim q$
- Arbitrary coupling $(X_0, X_1) \sim \pi_{0,1}$

Why does it work?

- Build flow from conditional flows
- Regress conditional flows



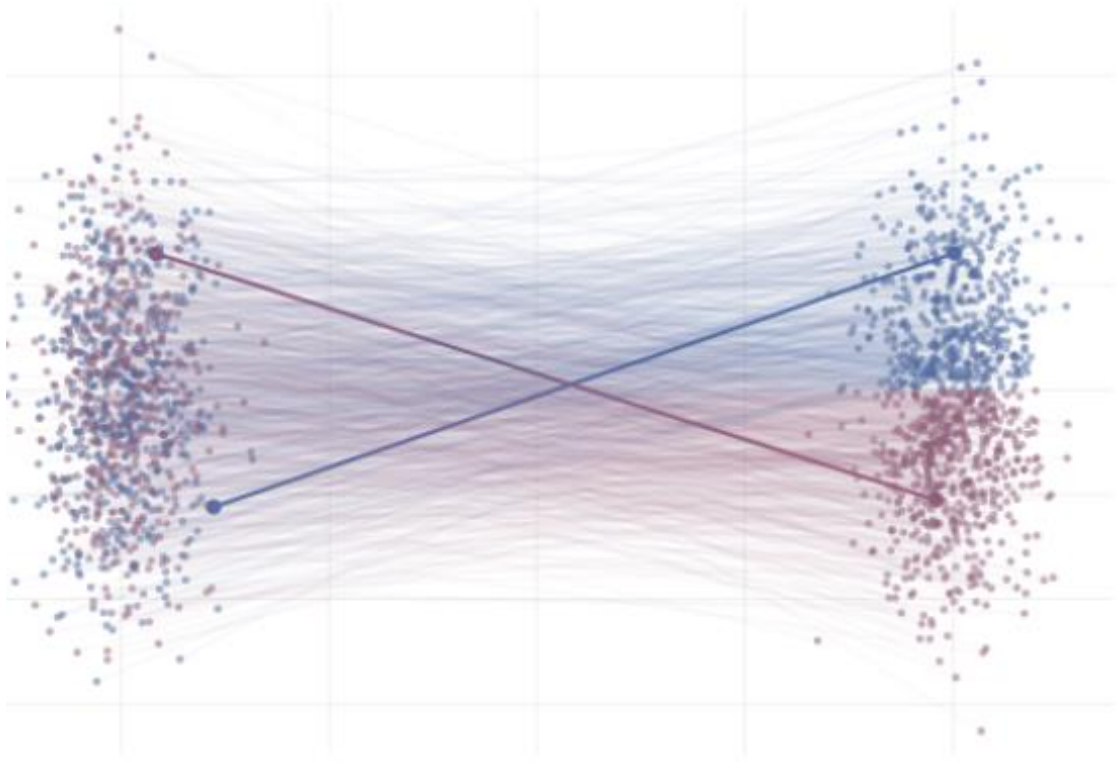
Optimal transport

$$\mathbb{E}_{t, X_0, X_1} \left\| \boxed{u_t^\theta(X_t)} - \boxed{(X_1 - X_0)} \right\|^2$$

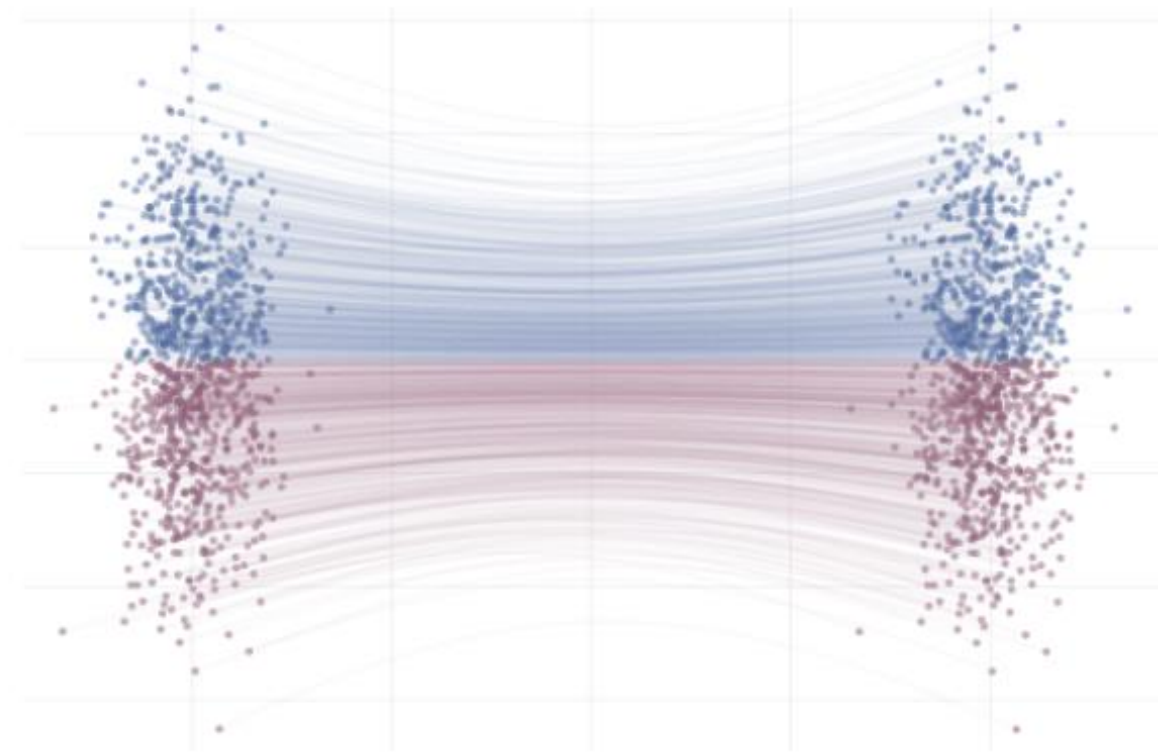
Neural network

Random Coupling of Two Distributions Results in Curved Velocity Field

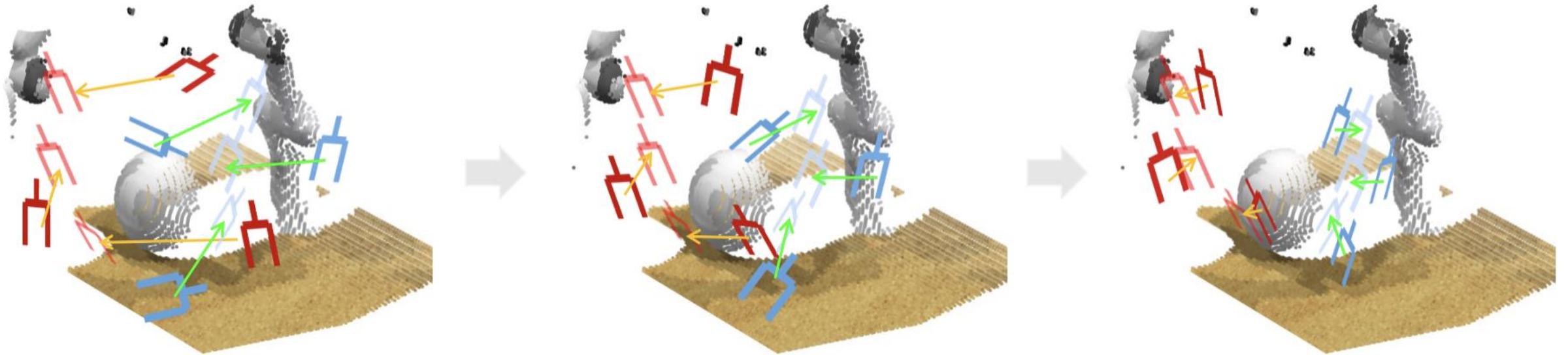
Training



Inference



Learning Flow Matching Models as Robot Policies



Question

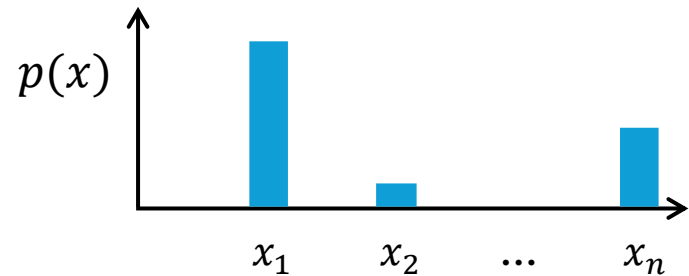
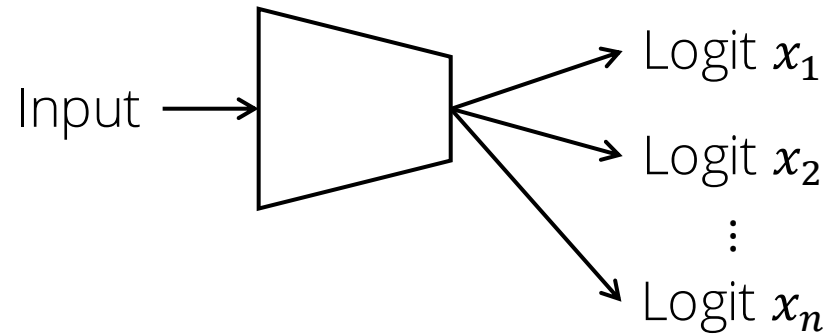
- How to reduce sampling steps during training?
- Is random coupling a good training recipe?
- What is the optimal prior distribution $\mathbf{X}_0$ to sample target distribution $\mathbf{X}_1$?

We'll have an additional video lecture by TAs (Jia-Wei and Chien-Sheng) on Flow Matching!

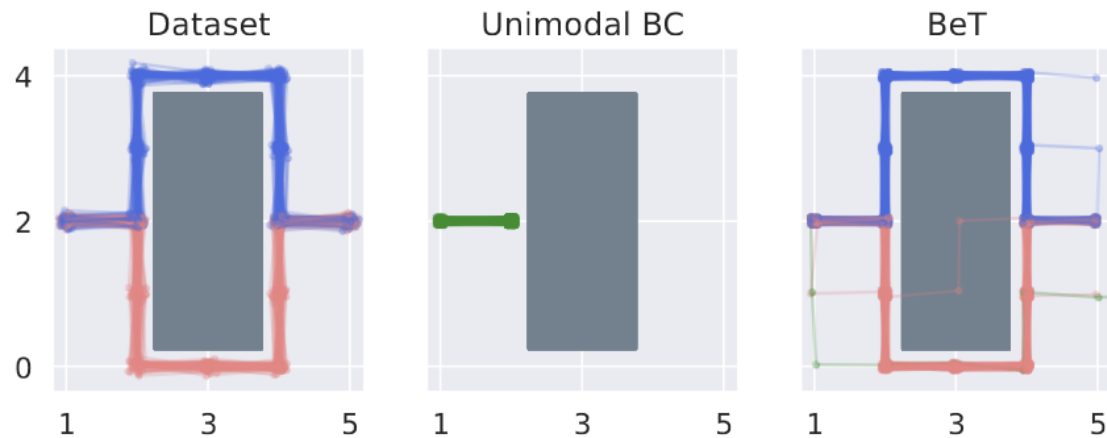
Choices of Probabilistic Generative Models

Categorical

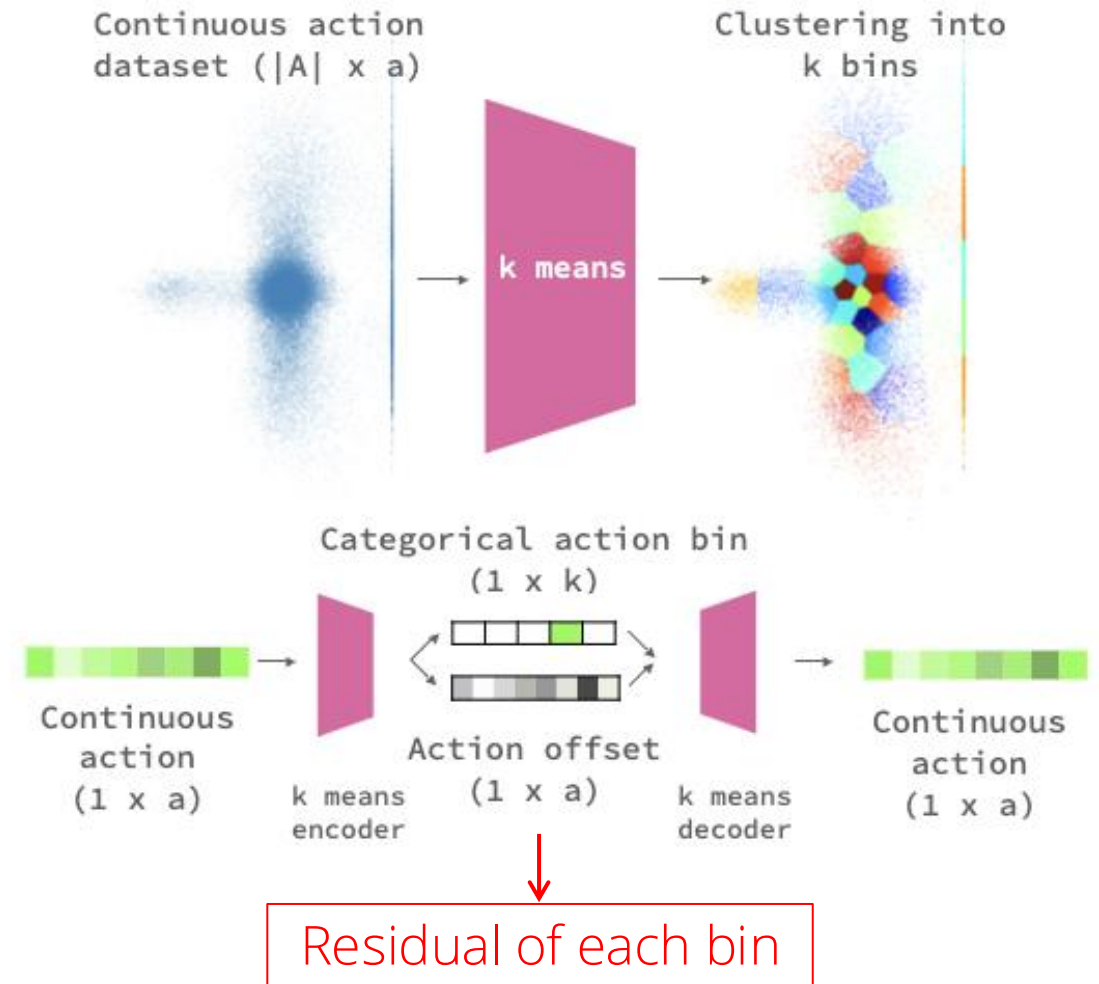
- Predict the categorical probability of sample "x"
- Most LLMs etc



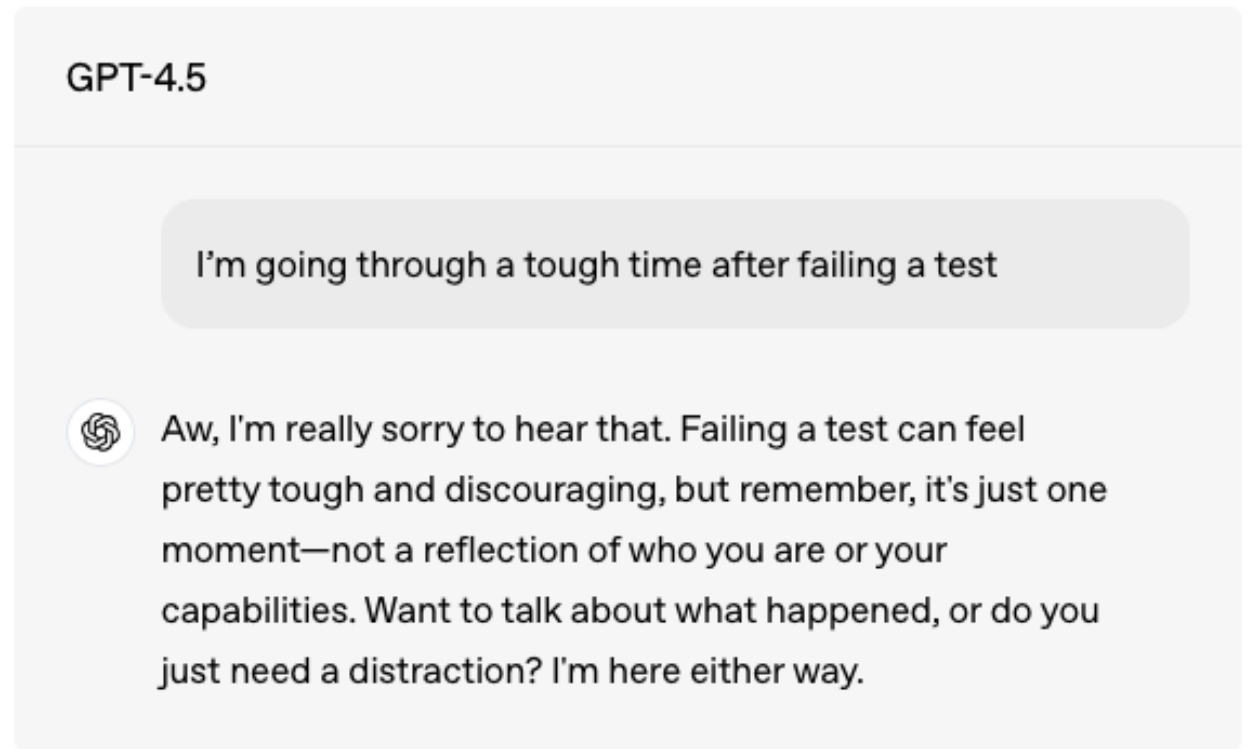
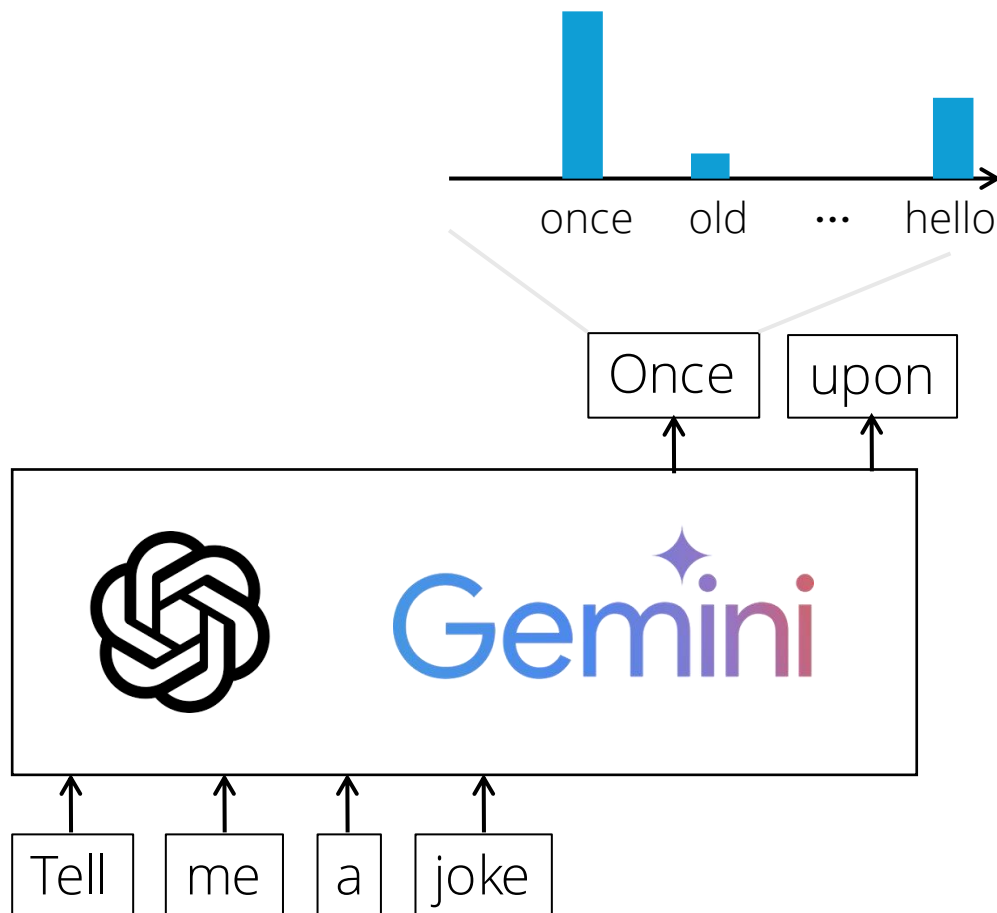
Example: Categorical Generative Models with Residuals



A. Continuous action binning

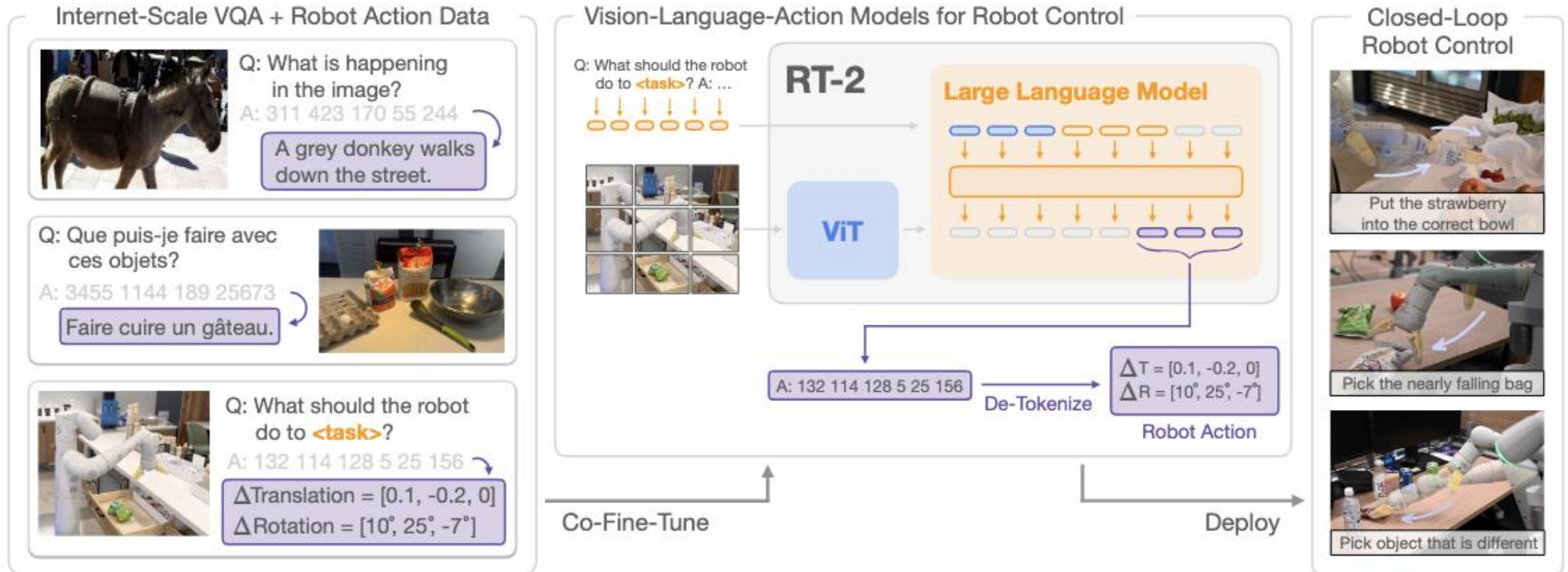


Example: Discrete Autoregressive Transformer as Categorical Generative Models

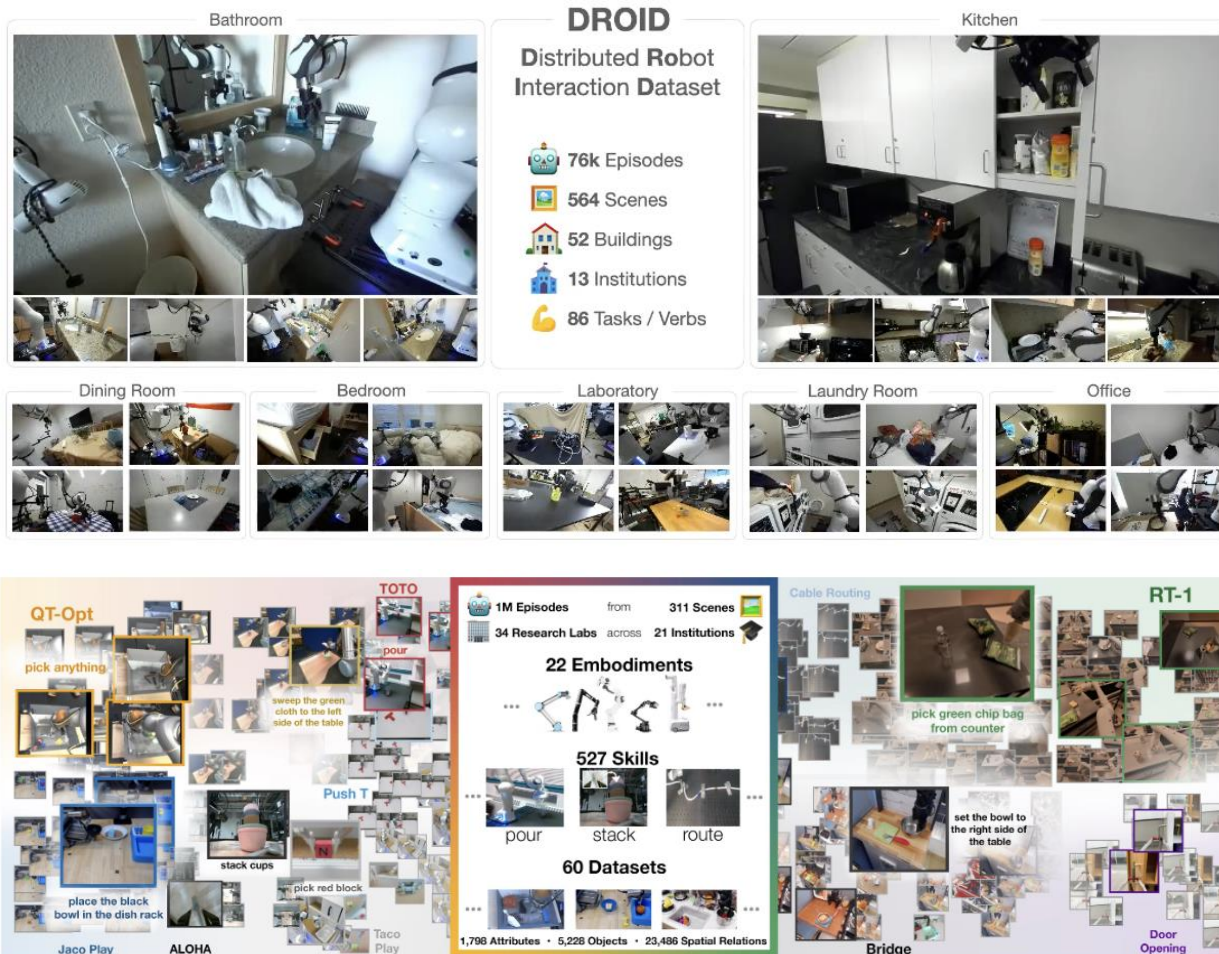


<https://openai.com/index/introducing-gpt-4-5/>

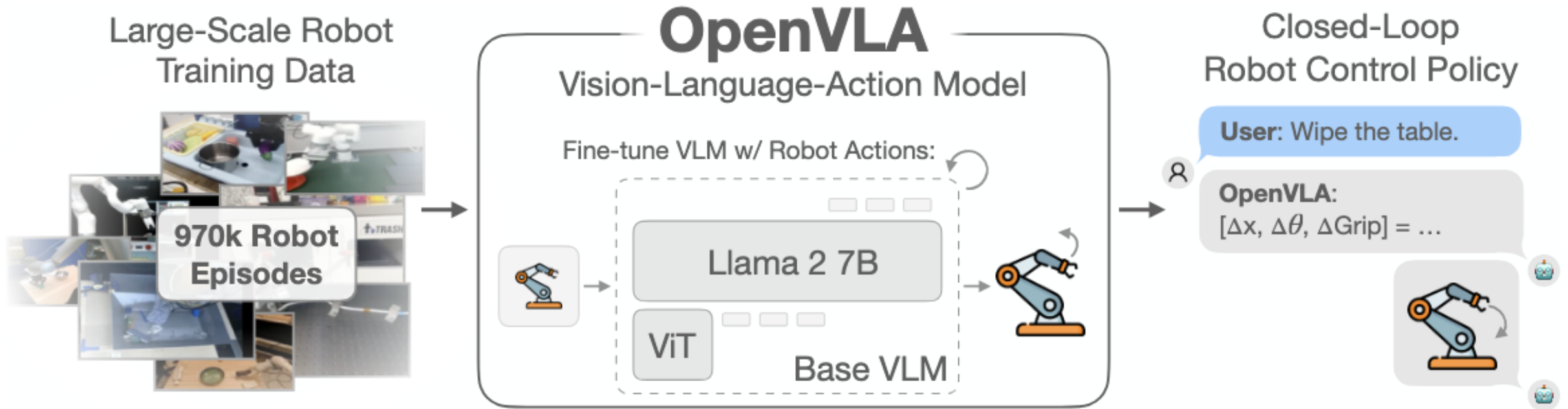
Vision-Language-Action Models (VLAs): Autoregressive Transformer as Robot Policies



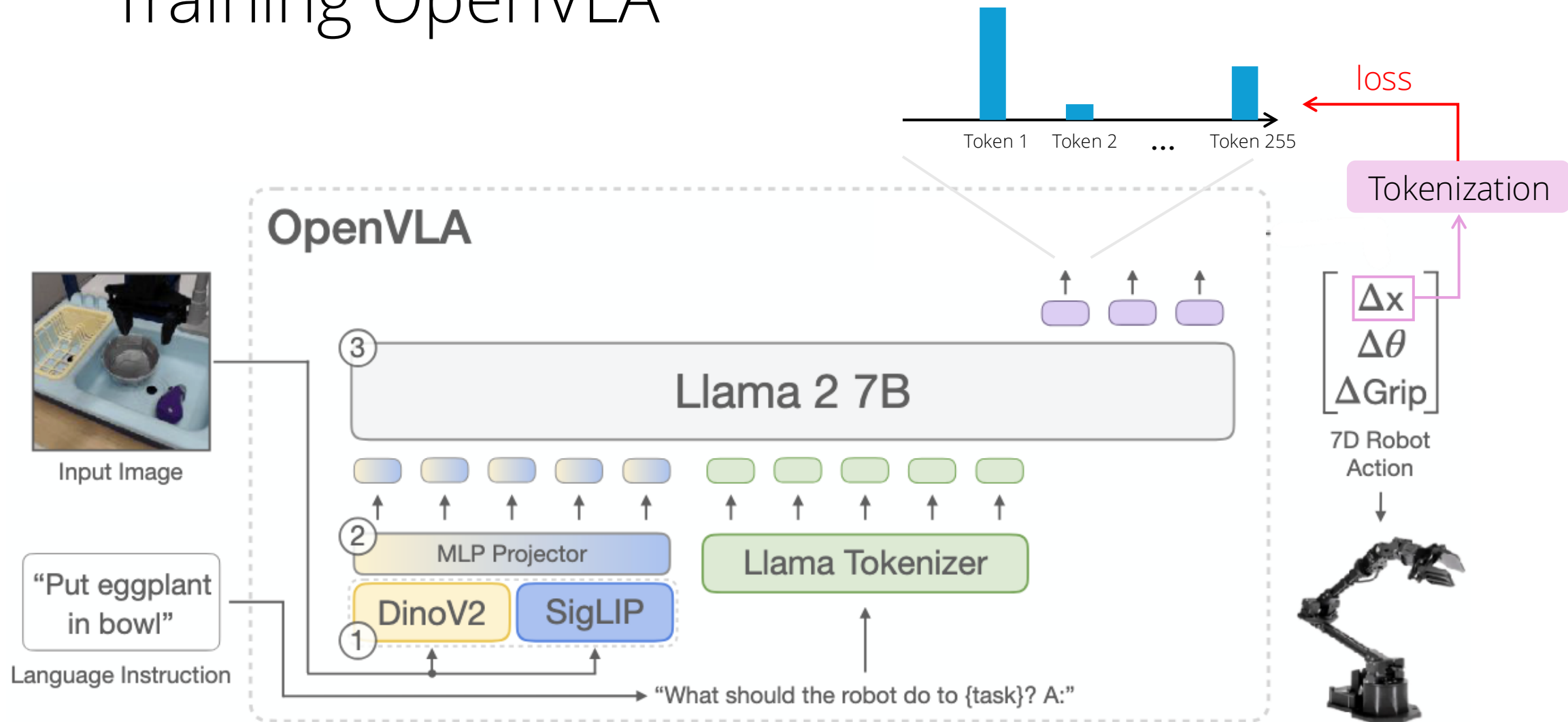
Large-Scale Robotic Datasets



Vision-Language-Action Models (VLAs): Autoregressive Transformer as Robot Policies



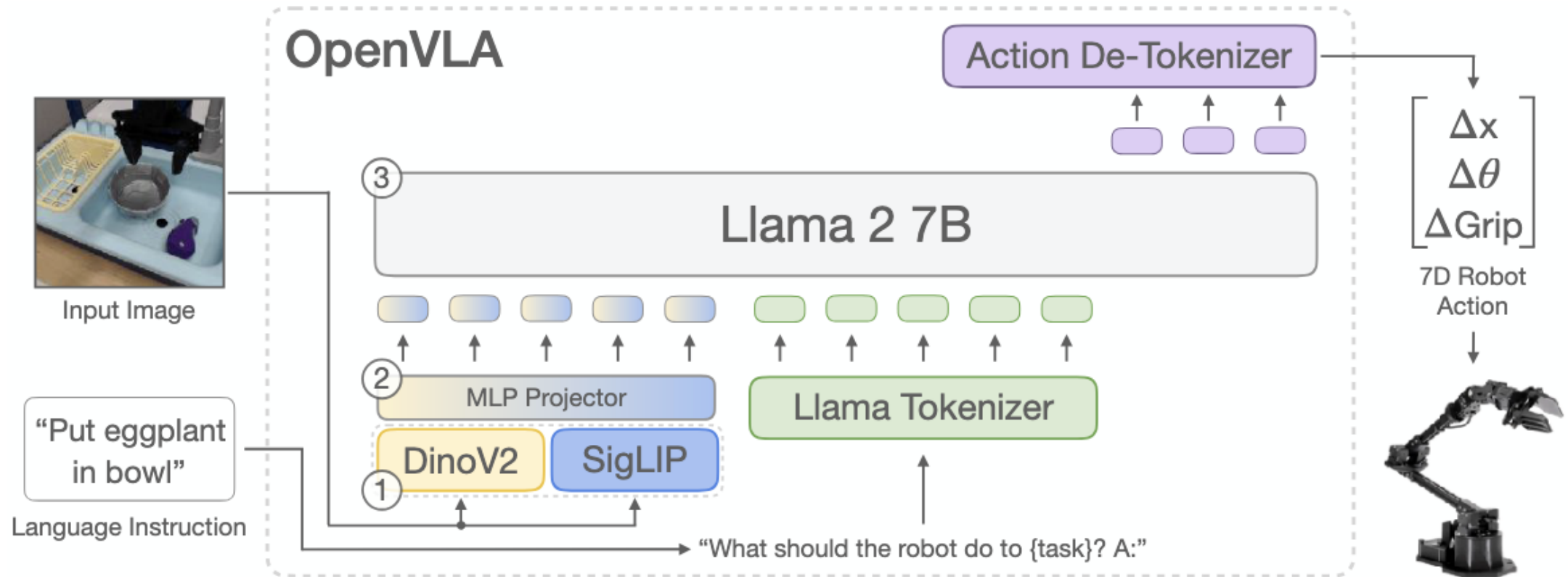
Training OpenVLA



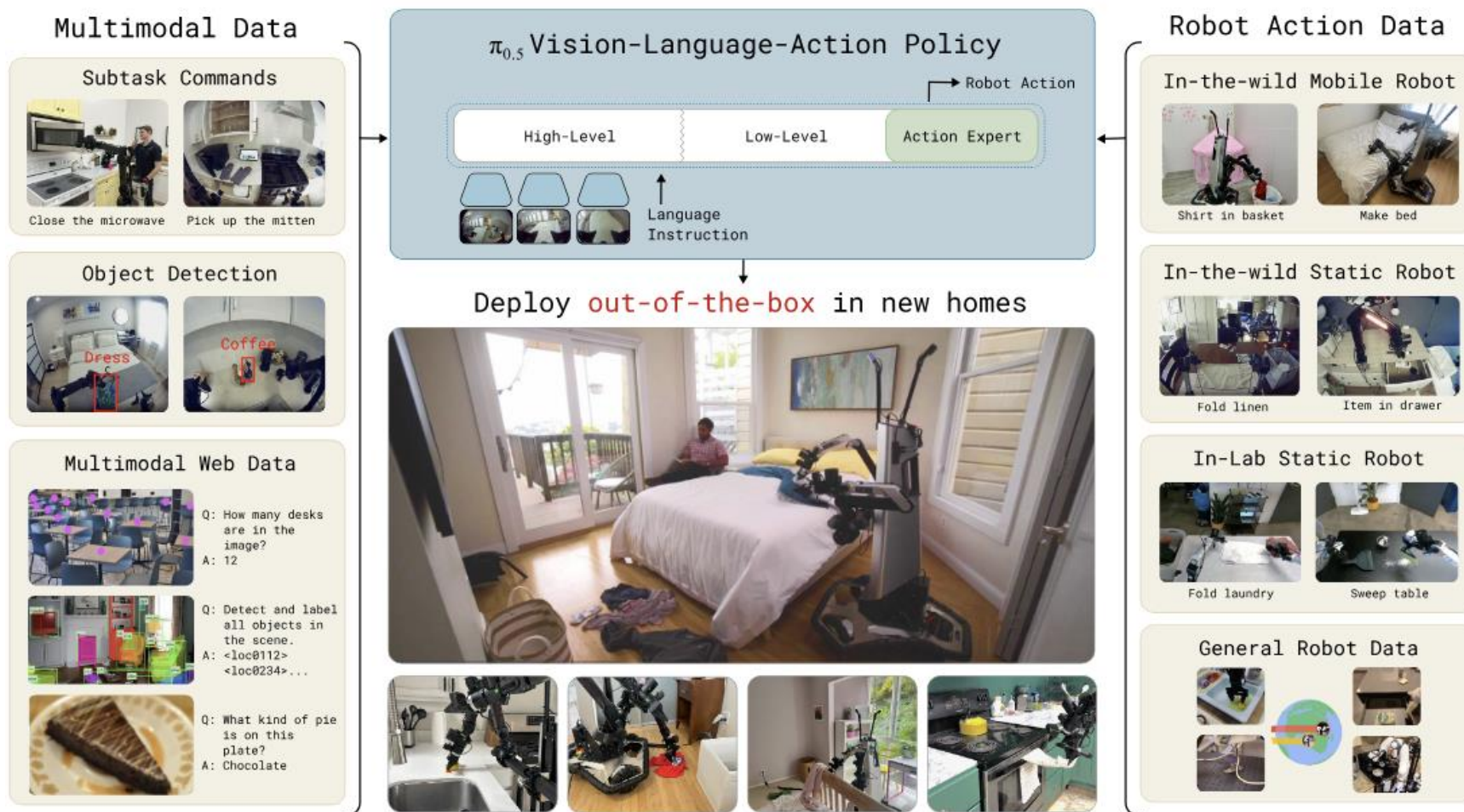
How to tokenize real-valued actions?

Normalization and mapping to rarely used tokens

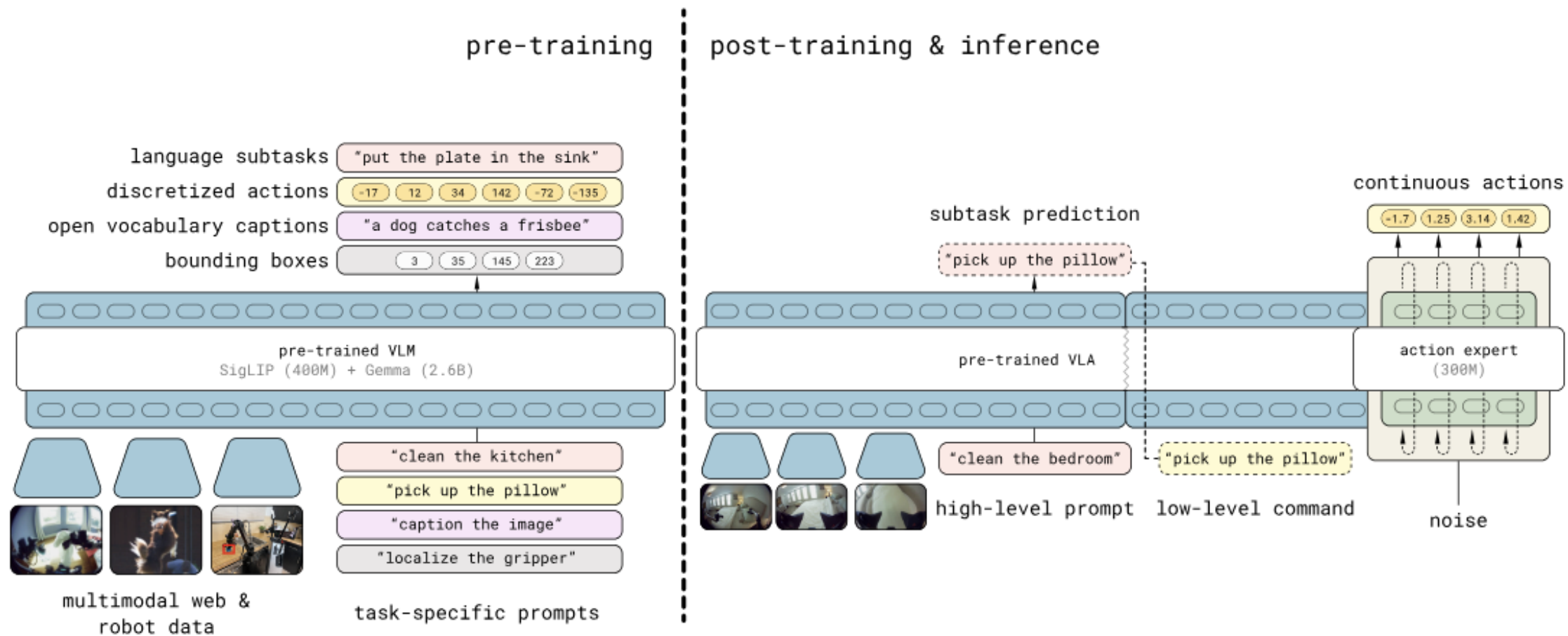
Inference with OpenVLA



Autoregressive Transformers Jointly Handle Robotic, Vision, and Language Tasks



Autoregressive Transformers Jointly Handle Robotic, Vision, and Language Tasks



Is Behavior Cloning + DAGGER All You Need?

- Of course, No!
- More problems:
 - **Causal confusion.** Does your model look at the correct feature so that it generalizes to novel scene?
 - **Multi-modality behaviors.** What if there are multiple ways to achieve a goal? What is the better formulation of the model output?
 - **No generalization to new scenes.** The model performs poorly in unseen / slightly difference scene

Robotics is Not Solved: SOTA Policies Don't Generalize

Succeed in the real world



OpenVLA: An Open-Source Vision-Language-Action Model

Fail in the digital twin

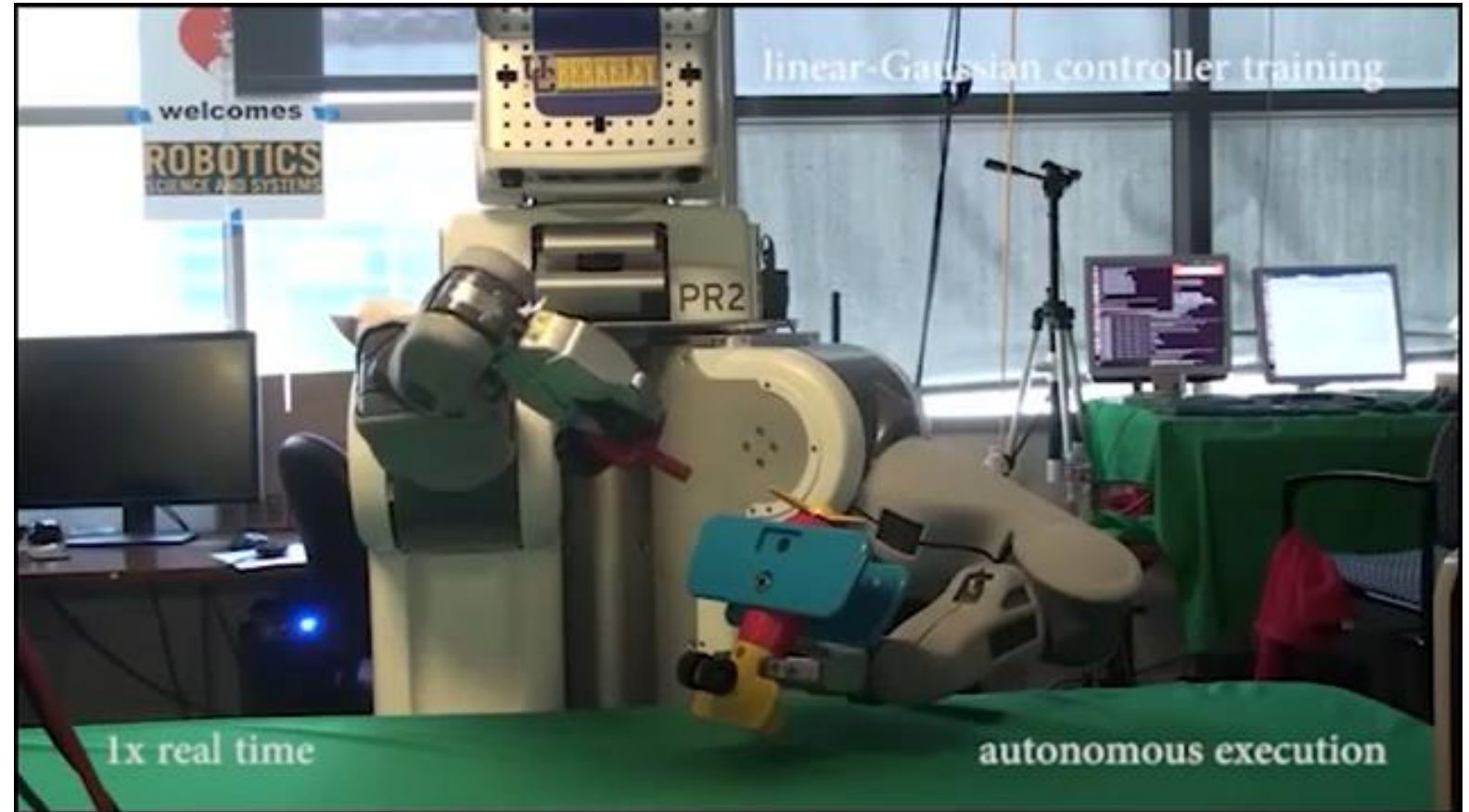


Solution: Exploration in the Environment

- Try out new behaviors with the hope of discovering optimal solutions

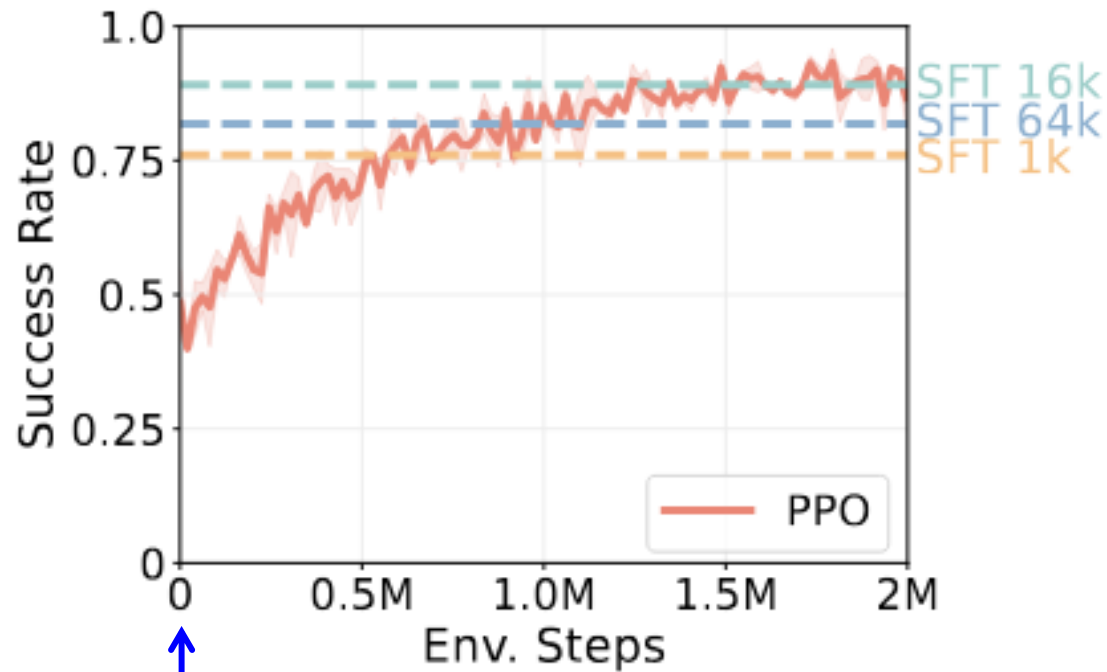


<https://www.youtube.com/watch?v=kFPcoclV5Vo>



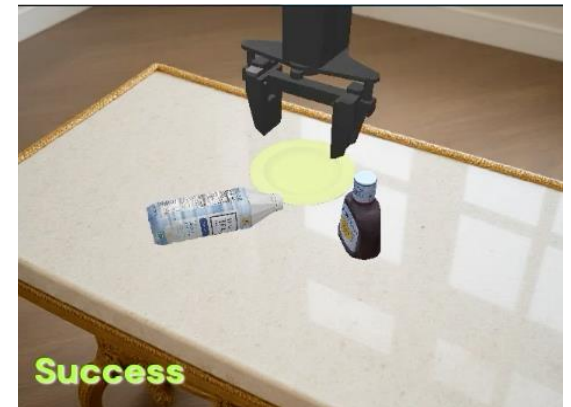
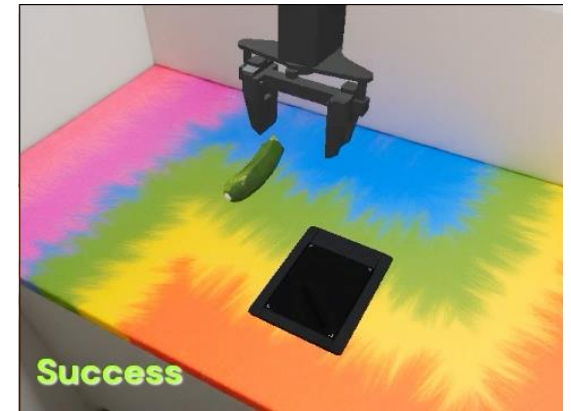
<https://www.youtube.com/watch?v=JeVppkoloXs&t=11s>

Exploration Improves Trained Policies



Before RL
finetuning

We'll cover in the next week!

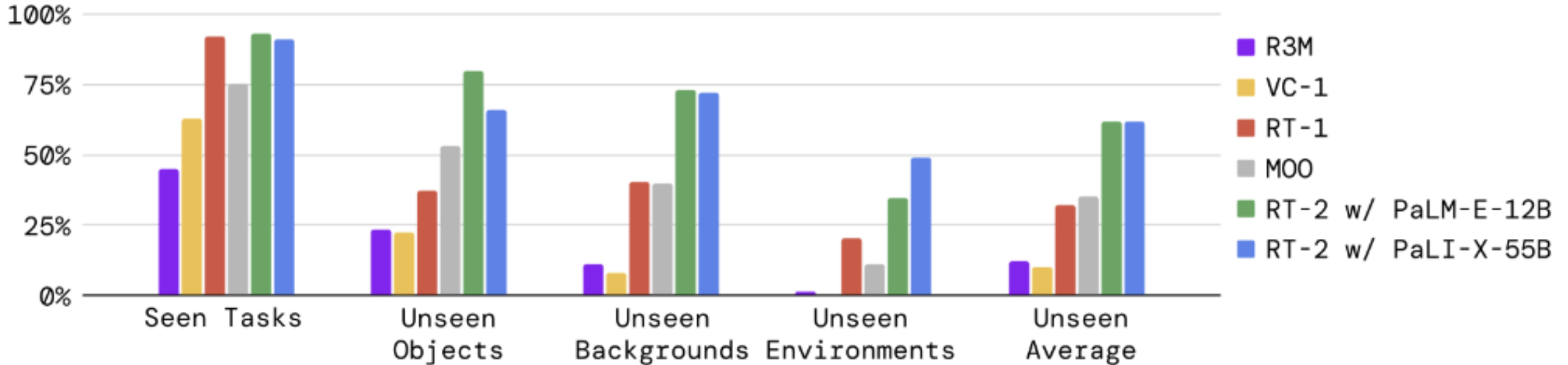


Is Behavior Cloning + DAGGER All You Need?

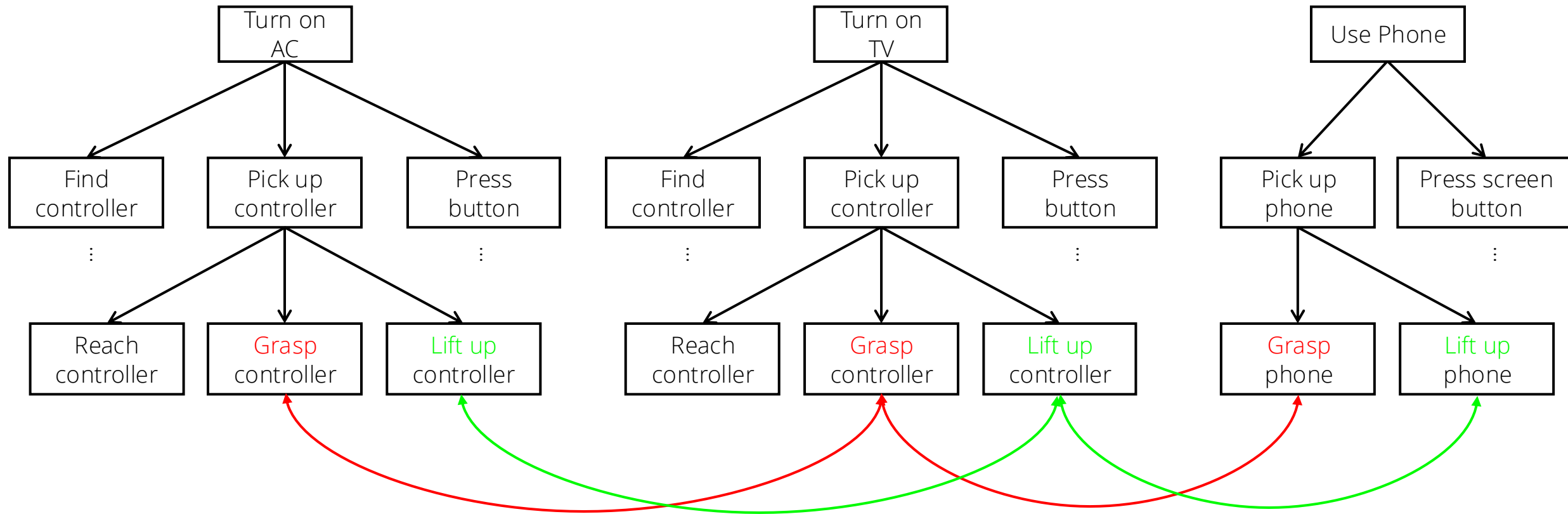
- Of course, No!
- More problems:
 - **Causal confusion.** Does your model look at the correct feature so that it generalizes to novel scene?
 - **Multi-modality behaviors.** What if there are multiple ways to achieve a goal? What is the better formulation of the model output?
 - **No generalization to new scenes.** The model performs poorly in unseen / slightly difference scene
 - **No generalization of new tasks.** Once your models learn to turn on the air conditioner, can it turn on the tv without/with minimal learning?

No generalization of new tasks

Even the largest and the best model don't generalize to unseen tasks...



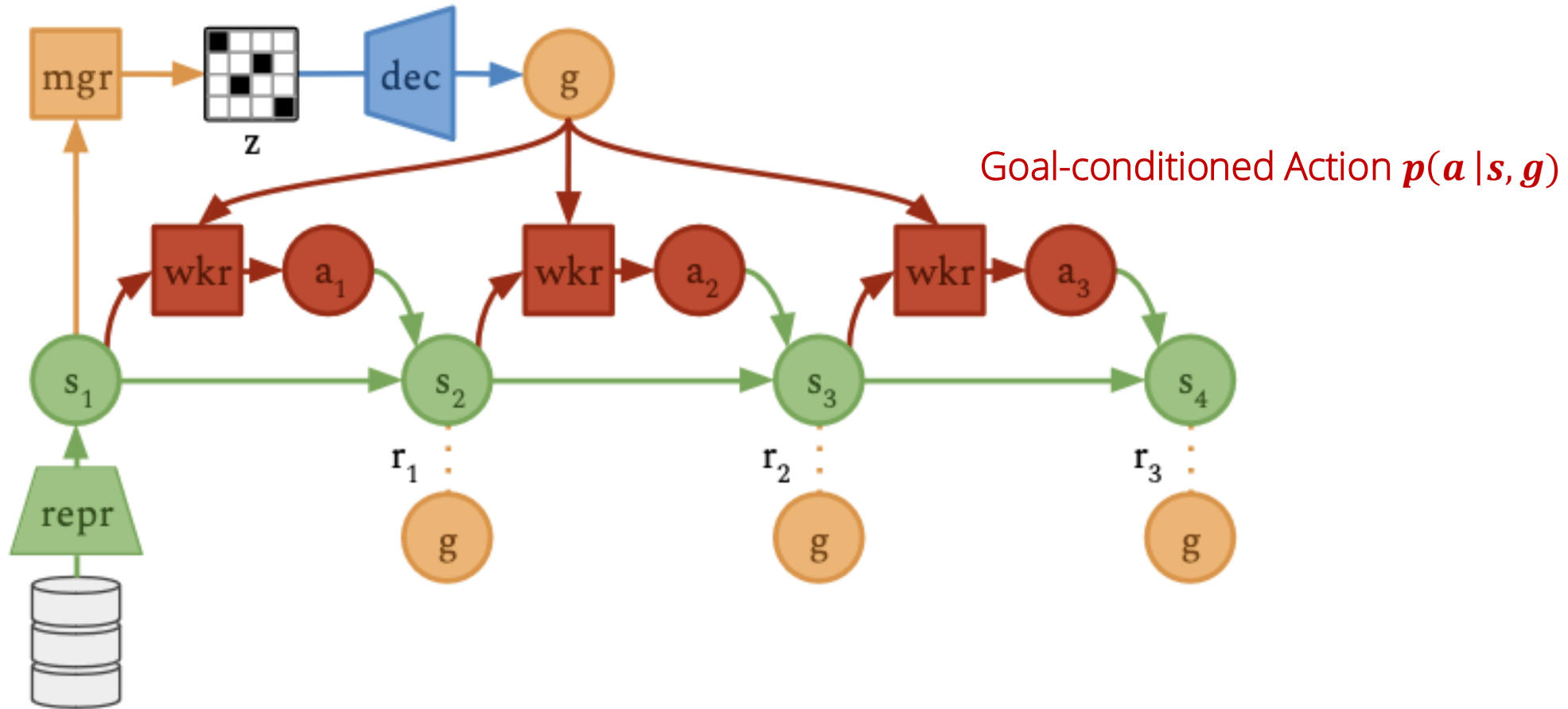
Knowledge Can be Shared Across Tasks



Lower-level skills are more similar among tasks
than higher-level ones

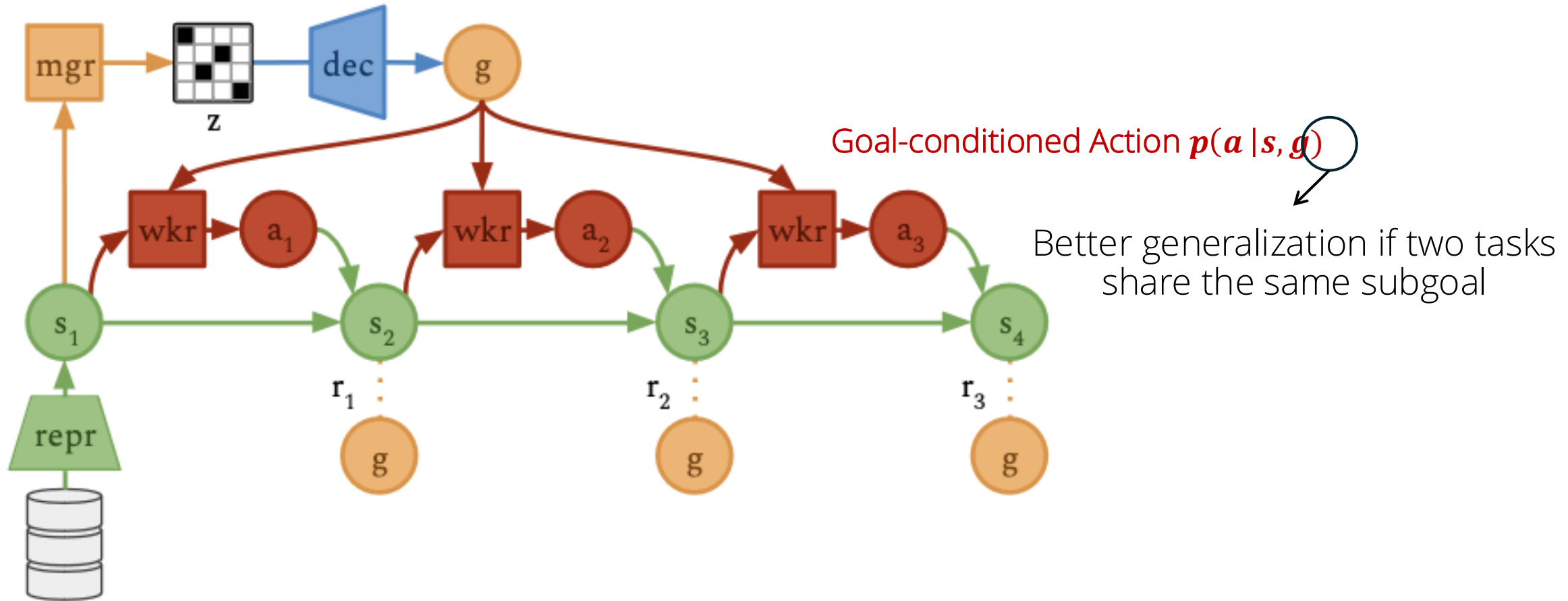
Learn to Achieve Subgoal

Subgoal E.g. Grasp controller

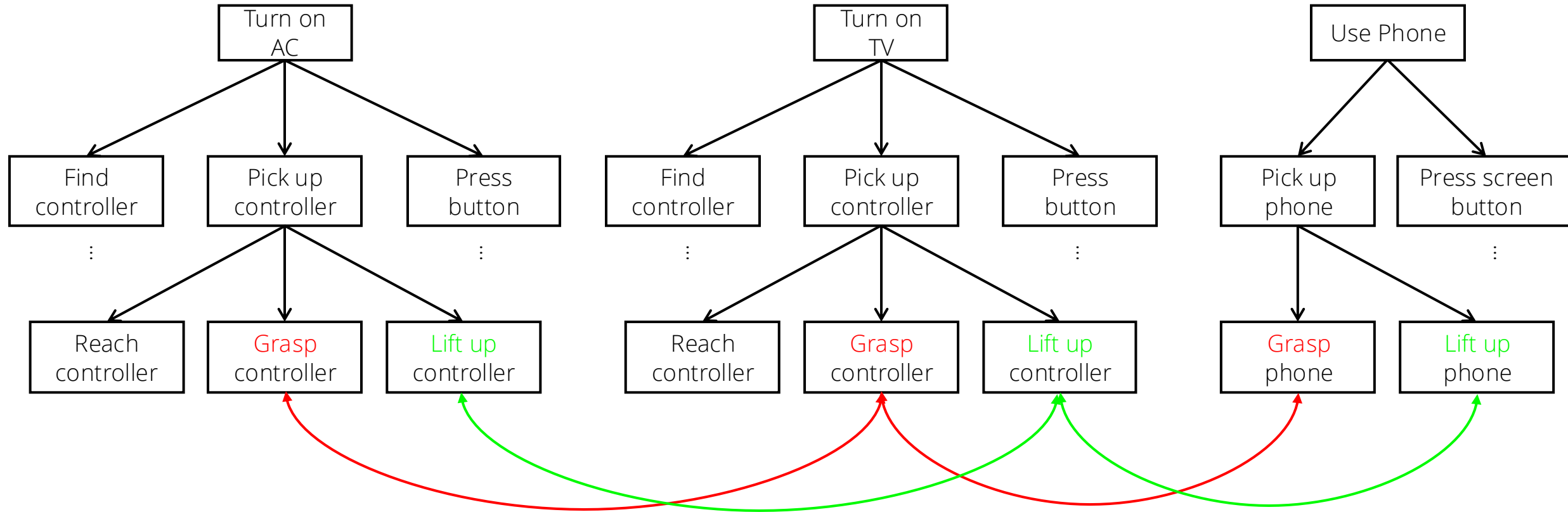


Learn to Achieve Subgoal

Subgoal E.g. Grasp controller

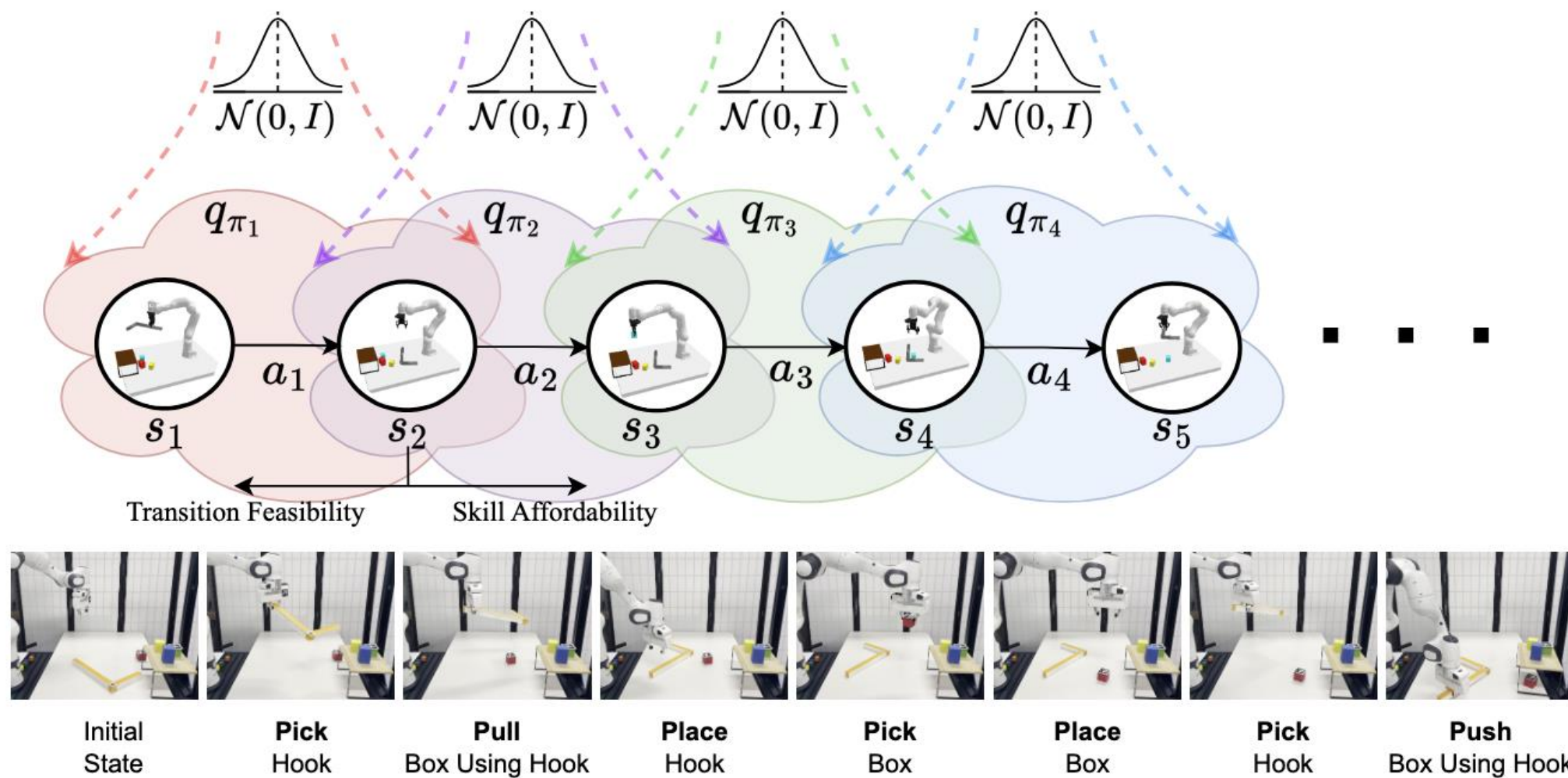


An Open Research Question: How to Generalize to Similar Skills?



Lower-level skills are more similar among tasks
than higher-level ones

An Open Research Question: How to Learn New Tasks By Composing Multiple Learned Skills?

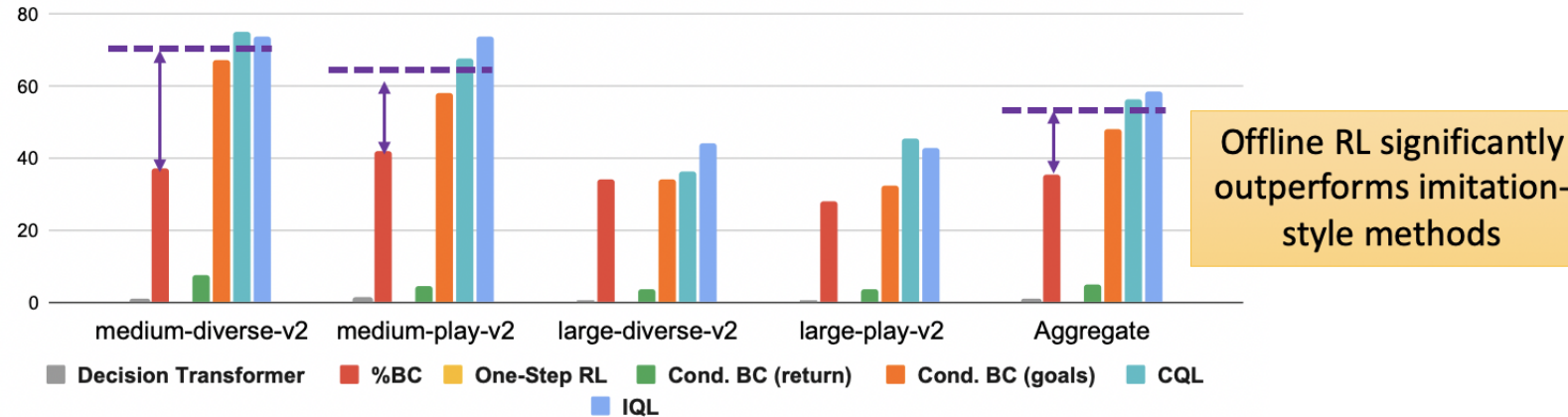


Is Behavior Cloning + DAGGER All You Need?

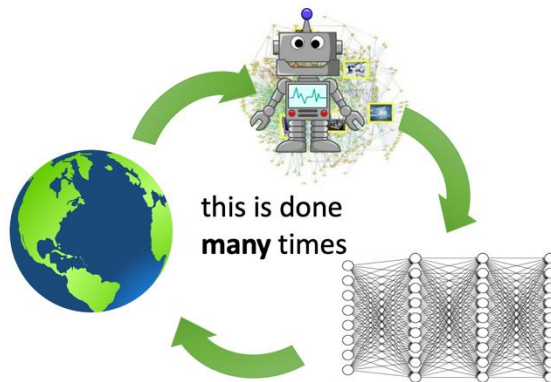
- Of course, No!
- More problems:
 - **Causal confusion.** Does your model look at the correct feature so that it generalizes to novel scene?
 - **Multi-modality behaviors.** What if there are multiple ways to achieve a goal? What is the better formulation of the model output?
 - **No generalization to new scenes.** The model performs poorly in unseen / slightly difference scene
 - **No generalization of new tasks.** Once your models learn to turn on the air conditioner, can it turn on the tv without/with minimal learning?
 - **Sub-optimal demonstration.** What if the experts are mediocre? Can you be better than your teacher?

Offline RL Goes Beyond Suboptimal Demonstrations

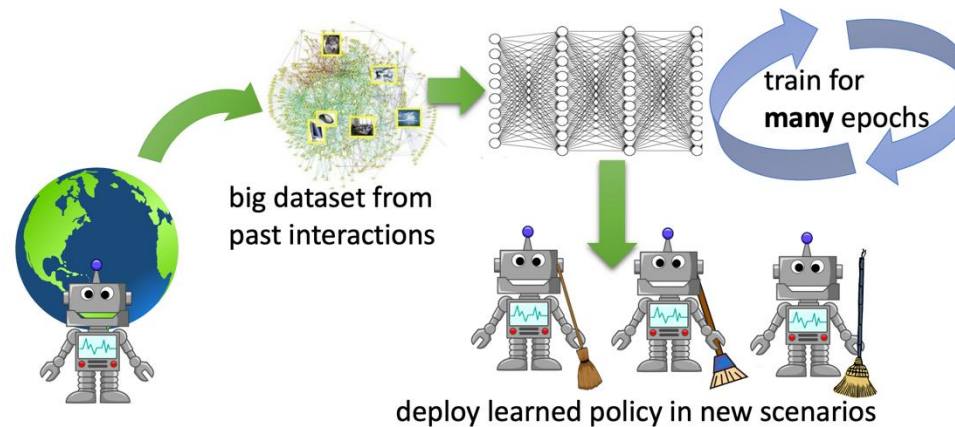
Results on AntMaze Tasks in D4RL



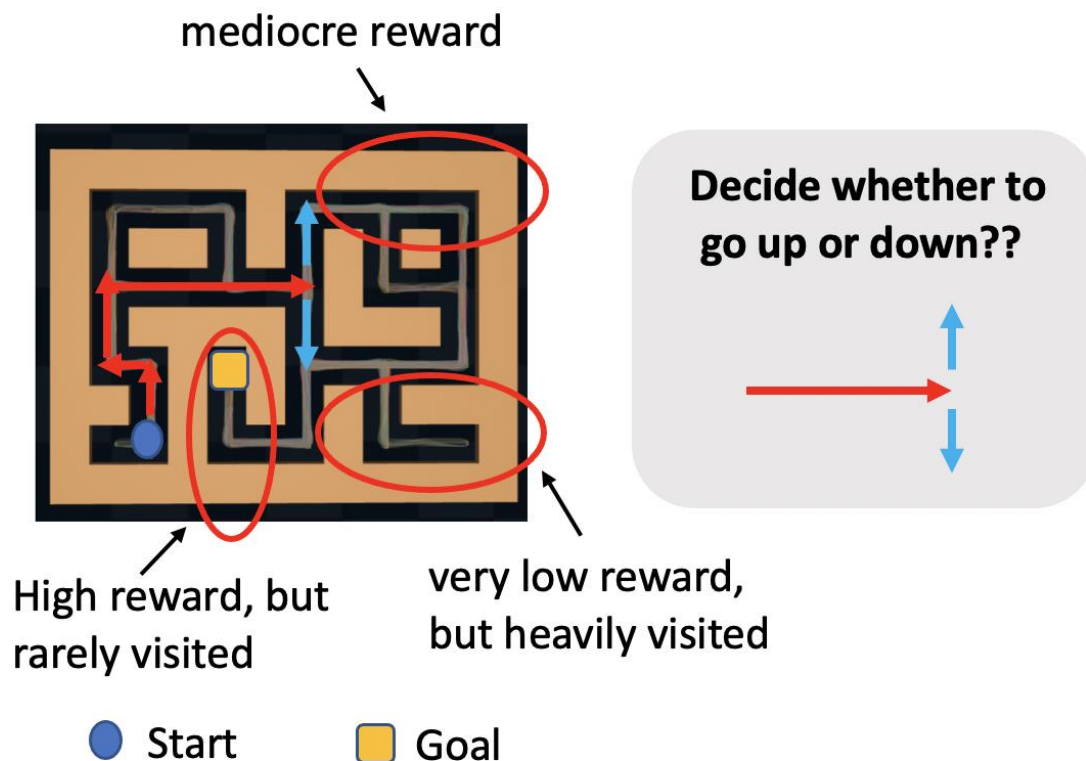
reinforcement learning



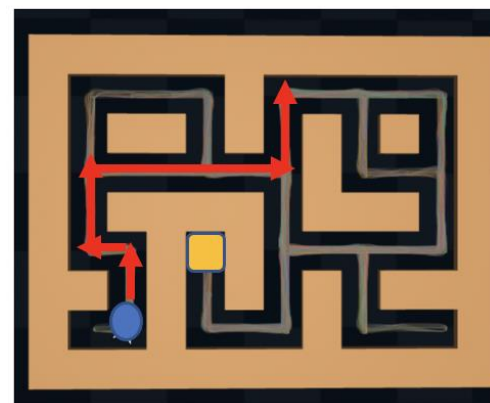
offline reinforcement learning



BC is Biased by the Mode of Behaviors

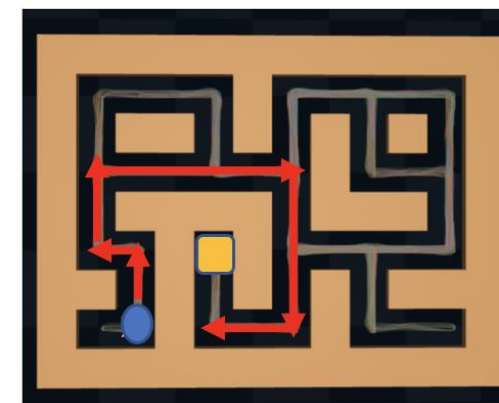


BC-like methods



Only finds mediocre reward

Full offline RL

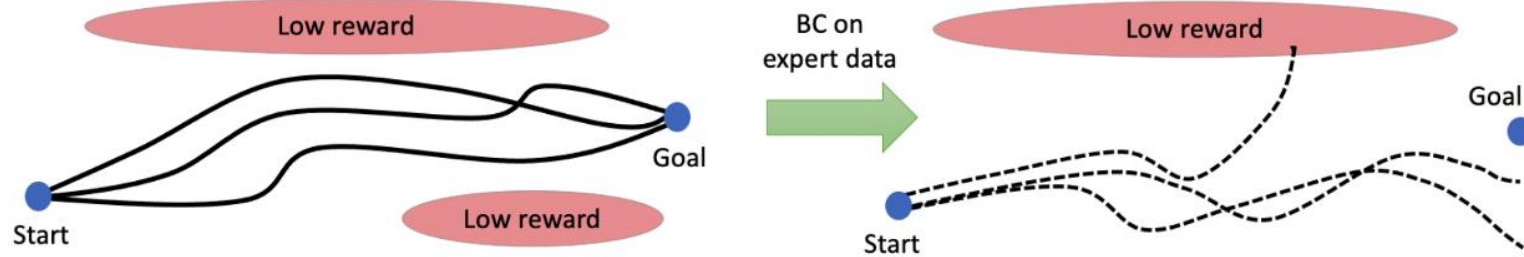


Finds the optimal behavior

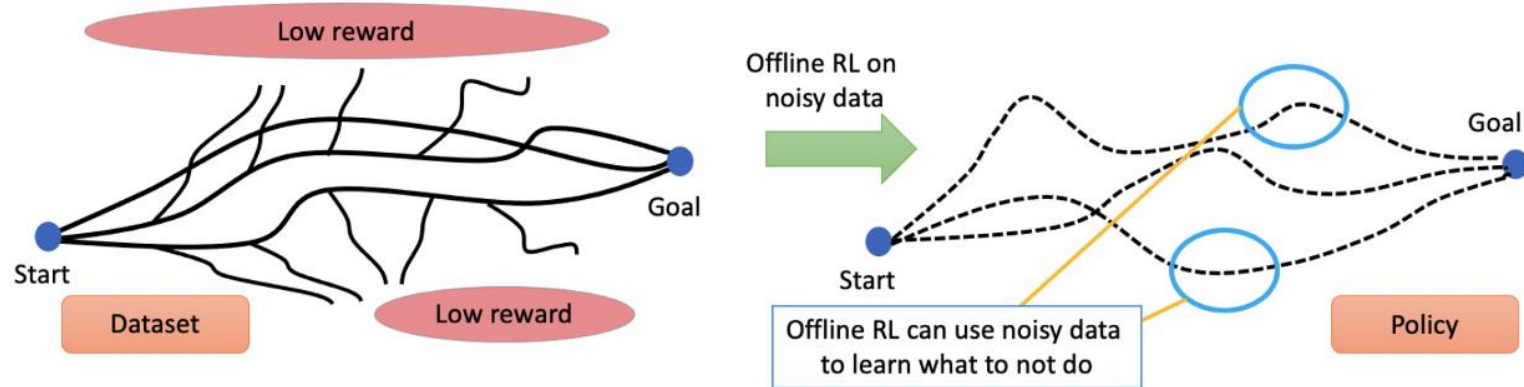
Takeaway: optimal solution under behavior policy future can be suboptimal

Offline RL Learns What to Not Do from Low-Reward Trajectories

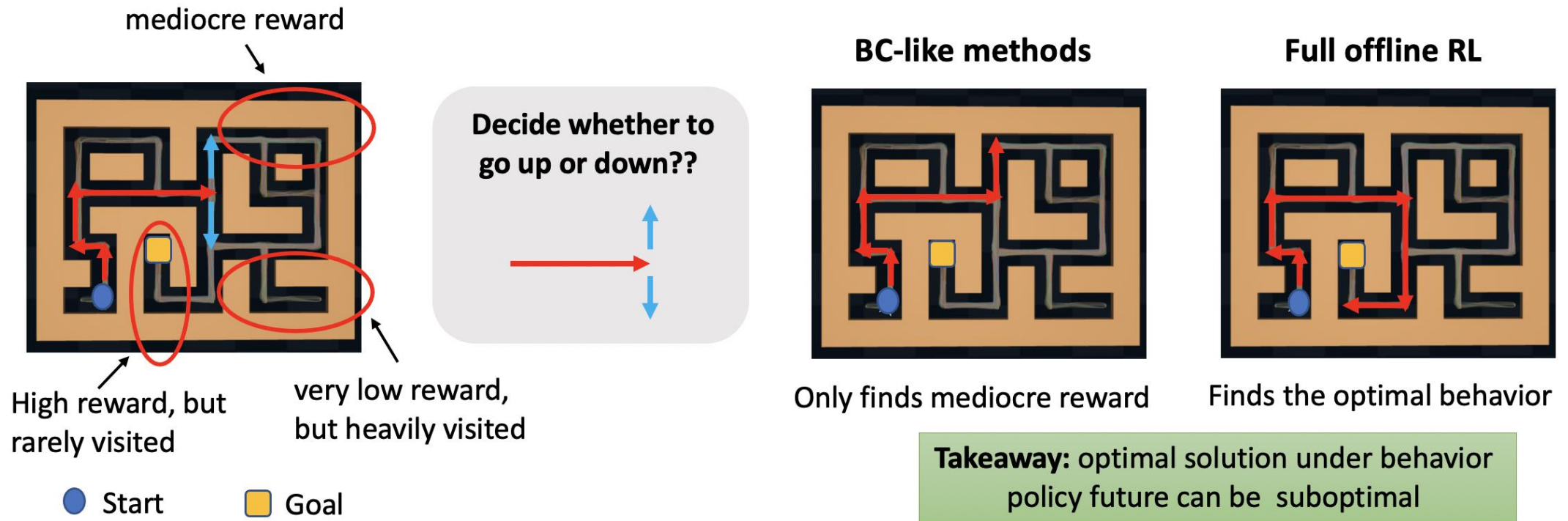
Behavior cloning on expert:



Offline RL with noisy data:



Offline RL Stitches Good Segments from Low-Reward Trajectories



We'll cover in the next lectures!